



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Telekomunikacji Multimedialnej

Praca dyplomowa magisterska

KONFIGUROWALNE MODUŁY WYŚWIETLACZY ALFANUMERYCZNYCH LCD Z INTERFEJSEM RÓWNOLEGŁYM I SZEREGOWYM

Szymon Dobrowolski, 151702
Rafał Ryske, 151679

Promotor
dr inż. Krzysztof Arnold

POZNAŃ 2026

Streszczenie

Praca magisterska skupia się na analizie i optymalizacji metod sterowania alfanumerycznymi wyświetlaczami LCD, stanowiąc kompleksowe studium integrujące w tym zakresie zagadnienia sprzętowe i programowe systemów mikroprocesorowych. W projekcie położono szczególny nacisk na oszczędne wykorzystanie wyprowadzeń mikrokontrolera przy zachowaniu efektywnej komunikacji procesora z wyświetlaczami. W pracy opisano mechanizmy komunikacji ze sterownikiem HD44780 i przeanalizowano różnice między klasyczną transmisją równoległą a transmisją szeregową. W tym celu wykorzystano magistralę I²C oraz SPI, badając układy ekspanderów portów z rodziny PCF8574, MCP23017 oraz MCP23S17. Omówiono także projekty zintegrowanych, dwuwarstwowych obwodów drukowanych. Przygotowano autorskie biblioteki w języku C z bezpośrednim dostępem do rejestrów ekspanderów i przeprowadzono szczegółową analizę opóźnień programowych oraz charakterystyk dynamicznych w transmisji danych. W pracy przedstawiono również narzędzia sprzętowo-programowe, udowadniając, że wydajne interfejsy szeregowo z powodzeniem mogą zastąpić komunikację równoległą bez strat dla poprawnej i szybkiej transmisji danych.

Słowa kluczowe: ATmega 328p, expander I/O, ESP32, I2C, mikrokontroler, MCP23017, MCP23S17, obwód drukowany, PCB, PCF8574, SPI, sterownik HD44780, wyświetlacz LCD

Abstract

This Master's thesis focuses on the analysis and optimization of control methods for alphanumeric LCDs, constituting a comprehensive study that integrates the hardware and software aspects of microprocessor systems in this regard. Particular emphasis was placed on the economical use of microcontroller pins while maintaining effective communication between the processor and the displays. The thesis describes the communication mechanisms with the HD44780 controller and analyzes the differences between classic parallel and serial transmissions. For this purpose, I²C and SPI buses were utilized to examine port expander chips from the PCF8574, MCP23017, and MCP23S17 families. The designs of integrated, two-layer printed circuit boards (PCBs) are also discussed. Custom C language libraries with direct access to the expanders' registers were developed, and a detailed analysis of software delays and dynamic characteristics in data transmission was conducted. Furthermore, the thesis presents hardware and software tools, proving that efficient serial interfaces can successfully replace parallel communication without compromising correct and high-speed data transmission.

Keywords: ATmega328p, I/O expander, ESP32, I2C, microcontroller, MCP23017, MCP23S17, printed circuit board, PCB, PCF8574, SPI, HD44780 controller, LCD display

Spis treści

1	Wstęp	7
1.1	Geneza pracy	7
1.2	Podział obowiązków projektowych	8
2	Wyświetlacze alfanumeryczne LCD	9
2.1	Charakterystyka wyświetlaczy	9
2.2	Interfejsy wyświetlaczy	11
2.3	Transmisja poleceń i danych	13
2.4	Konwertery I ² C/LCD	15
3	Ekspandery wejść/wyjść równoległych	17
3.1	Zadania ekspanderów w systemach mikroprocesorowych	17
3.2	Układy PCF8574	17
3.3	Układy MCP23x17	19
4	Projekt modułu LCD z 8-bitowymi ekspanderami PCF8574	25
4.1	Założenia techniczne	25
4.2	Koncepcja modułu	26
4.3	Opis modułu	27
4.4	Płytką PCB z soldermaską	28
4.5	Płytką PCB z frezowaniem ścieżek	32
5	Projekt modułu LCD z 16-bitowym ekspanderem MCP23017	35
5.1	Założenia techniczne	35
5.2	Koncepcja modułu	36
5.3	Opis modułu	36
5.4	Płytką PCB z soldermaską	37
5.5	Płytką PCB z frezowaniem ścieżek	38
6	Projekt modułu LCD z 16-bitowym ekspanderem MCP23S17	41
6.1	Założenia techniczne	41
6.2	Koncepcja modułu	42
6.3	Opis modułu	42
6.4	Płytką PCB z soldermaską	43
6.5	Płytką PCB z frezowaniem ścieżek	45
7	Projekt modułu LCD z 16-bitowymi ekspanderami MCP23x17	47

7.1	Założenia techniczne	47
7.2	Koncepcja modułu	48
7.3	Opis modułu	50
7.4	Płytką PCB z soldermaską	51
7.5	Płytką PCB z frezowaniem ścieżek	53
8	Projekt systemu rozproszonego z łącznością radiową	55
8.1	Założenia techniczne	55
8.2	Koncepcja modułu	55
8.3	Opis modułu	58
8.4	Badania parametrów propagacyjnych	58
8.5	Płytką PCB	67
9	Testowanie ekspanderów wejść/wyjść	71
9.1	Środowisko pomiarowe	71
9.2	Testy funkcjonalne układów PCF8574	72
9.3	Testy funkcjonalne układów MCP23017	77
9.4	Testy funkcjonalne układów MCP23S17	78
10	Uruchamianie modułu LCD z 8-bitowymi ekspanderami PCF8574	83
10.1	Środowisko pomiarowe	83
10.2	Testy modułu z 8-bitową transmisją równoległą danych	84
10.3	Testy modułu z 4-bitową transmisją równoległą danych	88
10.4	Testy modułu z transmisją szeregową danych	90
10.5	Testy modułu z transmisją szeregową danych i sygnałów sterujących	94
11	Uruchamianie modułu LCD z 16-bitowymi ekspanderami MCP23x17	97
11.1	Środowisko pomiarowe	97
11.2	Testy modułu z 8-bitową transmisją równoległą danych	98
11.3	Testy modułu z 4-bitową transmisją równoległą danych	98
11.4	Testy modułu z transmisją danych przez łącze I ² C	98
11.5	Testy modułu z transmisją danych i sygnałów sterujących przez łącze I2C	99
11.6	Testy modułu z transmisją danych przez łącze SPI	100
11.7	Testy modułu z transmisją danych i sygnałów sterujących przez łącze SPI	101
12	Uruchamianie modułu bezprzewodowego z układem CC1101	103
12.1	Środowisko pomiarowe	103
12.2	Kod testowy	104
13	Badania parametrów podsystemu z modułem LCD	117
13.1	Analiza porównawcza opóźnień programowych i charakterystyk dynamicznych wyświetlaczy LCD	117
13.2	Badania poprawności transmisji przewodowej w systemie rozproszonym	123
14	Podsumowanie	127
	Literatura	129

Rozdział 1

Wstęp

1.1 Geneza pracy

W obecnej dobie technologicznej wyświetlacze stanowią fundamentalny element komunikacji na linii człowiek-maszyna (ang. *Human-Machine Interface, HMI*). Przybierają one wielorakie formy i stopnie skomplikowania: od ekranów o wysokiej rozdzielczości w inteligentnych smartfonach, aż po proste, monochromatyczne panele w kasach sklepowych. W trakcie codziennej interakcji z urządzeniem elektronicznym użytkownika interesuje z reguły wyłącznie natychmiastowo i błędnie wyświetlony wynik. Rzadko zdajemy sobie sprawę, jak złożony ciąg zdarzeń musi zaistnieć wewnątrz układu cyfrowego, zanim na ekranie zaobserwujemy żadaną informację.

Każdy znak pojawiający się na matrycy ciekłokrystalicznej jest w rzeczywistości wynikiem dziełatek niskopoziomowych operacji bitowych, rygorystycznej synchronizacji sygnałów strobojących i translacji poleceń przez zaawansowane kontrolery sprzętowe. W świecie systemów wbudowanych (ang. *embedded systems*), gdzie zasoby pamięci i mocy obliczeniowej bywają mocno ograniczone, proces ten staje się jeszcze większym wyzwaniem inżynierskim.

Ponadto, nieustanne dążenie do miniaturyzacji elektroniki sprawia, że klasyczne, równoległe sterowanie wyświetlaczami alfanumerycznymi pochłania zbyt wiele cennych wyprowadzeń (GPIO) mikrokontrolera. Naturalnym rozwiązaniem tego problemu jest przeniesienie ciężaru komunikacji na popularne magistrale szeregowo, takie jak I²C czy SPI, z wykorzystaniem dedykowanych układów pośredniczących – ekspanderów portów.

Głównym celem niniejszej pracy jest kompleksowa analiza procesu sterowania matrycami LCD w standardzie HD44780, poczynając od interfejsów równoległych, aż po zoptymalizowane rozwiązania szeregowo oparte na układach z rodziny PCF8574, MCP23017 i MCP23S17. Obszarem zainteresowań jest wykazanie, jak pełną inżynierskich niuansów tematyką może być ta z pozoru prozaiczna wizualizacja znaków.

Aby w pełni zweryfikować te procesy, opracowano fizyczne stanowiska testowe w postaci zintegrowanych obwodów drukowanych, pozwalających na pełnoprawną obsługę wielowierszowych matryc. Co istotne, stworzone moduły zaprojektowano z myślą o wysokiej uniwersalności, dzięki czemu mogą one posłużyć również jako cenne pomoce dydaktyczne w laboratoriach studenckich. Przeprowadzona w kolejnych rozdziałach analiza porównawcza, uwzględniająca przepływność magistrali, rygory czasowe i niezawodność sygnałową, stanowić ma kompendium wiedzy o optymalnym łączeniu warstwy programowej ze sprzętową w celu uzyskania maksymalnej efektywności układu.

1.2 Podział obowiązków projektowych

Szymon Dobrowolski

Dokonanie przeglądu not katalogowych mikrokontrolerów z rdzeniem AVR i specyfikacji modułów Arduino. Analiza architektury, obsługi programowej i protokołów transmisji dla wskazanych ekspanderów I/O. Zapoznanie się z opisem katalogowym standardowego sterownika HD44780U. Dokonanie przeglądu konwerterów I2C/LCD oraz SPI/LCD. Zapoznanie się z projektami wstępnymi konfigurowalnych wyświetlaczy LCD, zaproponowanymi przez promotora. Zaprojektowanie konfigurowalnego modułu LCD 2x16/4x20 z interfejsem równoległym oraz interfejsami I²C i SPI. Wykonanie badań symulacyjnych oprogramowania. Przeprowadzenie podstawowych testów na wersjach prototypowych modułów. Wykorzystanie w testach platformy Arduino. Przeprowadzenie badań, weryfikujących zgodność opóźnień programowych z wymaganiami sterownika HD44780U oraz parametrami dynamicznymi wyświetlaczy i ekspanderów I/O.

Współudział w opracowaniu projektów PCB modułów według wskazówek promotora. Udział w uwzględnieniu w projektach PCB wersji z frezowanym polem masy i z warstwą soldermaski.

Rafał Ryske

Zapoznanie się z zasadą działania, protokołami komunikacyjnymi i programową inicjalizacją wbudowanych modułów TWI oraz SPI. Przegląd wyświetlaczy alfanumerycznych LCD, ze szczególnym uwzględnieniem modułów 2x16 i 4x20. Przeanalizowanie interfejsów i rozkładu wyprowadzeń typowych alfanumerycznych wyświetlaczy LCD. Wybór reprezentatywnych wyświetlaczy o zgodnym rozkładzie wyprowadzeń. Zaprojektowanie modułu LCD 2x16/4x20 z interfejsem równoległym, interfejsem I²C i konfiguracją połączeń na płytce modułu. Wykorzystanie wskazanych 8-bitowych ekspanderów I/O. Wykorzystanie wskazanych 16-bitowych ekspanderów I/O. Opracowanie programów testowych dla zaprojektowanych modułów. Zbadanie poprawności komunikacji jednostki centralnej z konfigurowalnymi modułami LCD dla różnych długości połączeń.

Współudział w opracowaniu projektów PCB modułów według wskazówek promotora. Udział w uwzględnieniu w projektach PCB wersji z frezowanym polem masy i z warstwą soldermaski.

Rozdział 2

Wyświetlacze alfanumeryczne LCD

2.1 Charakterystyka wyświetlaczy

Rozważania dotyczące wyświetlaczy należy zacząć od tego jakie rozwiązania są oferowane przez producentów oraz potrzeb projektowych. Pierwszym wyborem jest technologia w jakiej wyprodukowano ekran. W większości przypadków sprowadza się to do decyzji pomiędzy wyświetlaczami LCD, a OLED. Obydwa rozwiązania różnią się pod względem parametrów wyświetlania (tutaj przeważa ekran OLED), oraz ceną. Młodzi adepci elektroniki częściej decydują się na zakup wyświetlaczy LCD do celów dydaktycznych. Niebagatelny jest tutaj stosunek ceny do parametrów fizycznych ekranu - jego wielkości. W związku z popularnością tej technologii, jest ona rozważana w dalszej części pracy.

Typowy wyświetlacz ciekłokrystaliczny do prawidłowej pracy potrzebuje podświetlenia, sterownika oraz filtrów polaryzacyjnych. Elementem, który powoduje zmianę danych i komend z mikrokontrolera na napięcia sterujące matrycą, jest sterownik graficzny. Dzięki temu urządzeniu możliwa jest manipulacja polaryzacją struktury ciekłokrystalicznej - zmiana odcieni, barw. Najczęściej spotykanymi rodzajami sterowników są między innymi: *HD44780*, *ILI9341* i *ST7789*. Implementacja danego układu scalonego w wyświetlaczu definiuje jego sposób komunikacji, rodzaje obsługiwanych matryc, liczbę możliwych znaków czy sekcję zasilania układu [9].

Wybór ekranu jest nieodłącznie związany z doбором pod optymalną liczbę znaków. Tak jak w większości przypadków rynek wyświetlaczy pozwala tu na dostęp do wielorakich rozwiązań. Dla sterowników *HD44780* dostępnych jest kilka rozmiarów matryc znakowych. Najpopularniejszymi są 16x2, 20x4 i 40x2¹. Sterowniki *ILI9341* oraz *ST7789* są przeznaczone do wizualizacji różnego rodzaju grafik. Z tego względu ekrany współpracujące z powyższymi układami są charakteryzowane liczbą dostępnych pikseli [5].

W trakcie projektowania urządzenia, należy zwrócić uwagę na umiejscowienie wyświetlacza na płytce. W przypadku wyboru ekranu z sterownikiem *HD44780* błędne dobranie wyprowadzeń, może wiązać się z nieodpowiednim wyświetlaniem znaków - do góry nogami. Może to zabrzmieć prozaicznie, lecz tworząc duży projekt takie przeoczenie niesie za sobą ogromne konsekwencje - w najgorszym przypadku zmarnowanie funduszy. Młodsze sterowniki pozwalają na programowe manipulowanie obrazem, przez co problem z błędnym zaprojektowaniem miejsca na wyświetlacz, przy dobrze napisanym kodzie praktycznie nie występuje[27].

W zależności od technologii wykonania sterownika wyświetlacza, występuje w nich różne napięcie zasilania oraz przyjmowane stany logiczne. *HD44780* musi być zasilany napięciem 5 V, natomiast

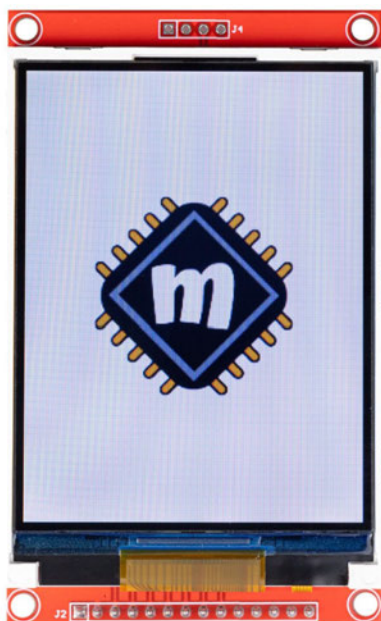
¹gdzie pierwsza liczba to liczba kolumn, natomiast druga to liczba wierszy



RYSUNEK 2.1: Przykład wyświetlacza 16x2 z wyprowadzeniami na dolnej części wyświetlacza [29]



RYSUNEK 2.2: Przykład wyświetlacza 16x2 z wyprowadzeniami na górnej części wyświetlacza [4]



RYSUNEK 2.3: Wyświetlacz TFT sterowany układem ILI9341 [22]



RYSUNEK 2.4: Zdjęcie wyświetlacza 40x2 ze złączami po lewej stronie [30]

nowszy klon tego układu *HD44780U* może przyjmować napięcie 2,7 - 5,5 V. Z tego względu nowsza odsłona zadziała zarówno z mikrokontrolerami z rodziny *ATMEL AVR* (5 V) [1] jak i z *ESP32* (3,3 V) [11]. Sterownik *ILI9341* przyjmuje napięcie zasilania na poziomie 2,5 - 3,3 V. W przypadku mikrokontrolerów pracujących na wyższym napięciu, może być konieczne zastosowanie konwertera poziomów napięć logicznych [13].

Każdy wyświetlacz wymaga podświetlenia. Bez niego odczytanie informacji jest bardzo trudne, często niemożliwe. Większość ekranów pozwala na zmienianie jasności. Jest to możliwe poprzez użycie zewnętrznego potencjometru lub zmiennej szerokości impulsu - *PWM*. Takie rozwiązanie pozwala na dostosowanie wyświetlacza do miejsca pracy.

Kolejną możliwością dostosowania wyświetlacza, jest regulacja kontrastu. W przypadku modułów z 16-pinowym wyprowadzaniem, zazwyczaj odbywa się to zmianą napięcia na pinie *V0*. Malejące napięcie powoduje zwiększenie kontrastu i zanikanie znaków z matrycy. Narastające napięcie zmniejsza kontrast i znaki zlewają się w białe lub czarne prostokąty. Idealne napięcie to około 0,5 - 1 V, można je uzyskać z napięcia dzielnika, lub potencjometru, którym można liniowo wyregulować kontrast, pod dany kąt patrzenia na wyświetlacz. W przypadku chęci skorzystania z cyfrowej regulacji kontrastu, pin *V0* można podłączyć do złącza *PWM*, podobnie jak w przypadku sterowania jasnością [11].

2.2 Interfejsy wyświetlaczy

Interfejs to punkt styku lub zestaw reguł umożliwiający komunikację między różnymi systemami, urządzeniami lub programami. Pozwala na szybką wymianę danych oraz współpracę elementów [23].

Głównym podziałem interfejsów jest klasyfikacja ze względu na sposób przesyłu danych. W taki sposób rozróżniamy magistrale jako równoległe i szeregowy. Interfejs równoległy charakteryzuje się znacznie większą prędkością transmisji, lecz potrzebuje wielu połączeń. Magistrale szeregowe ograniczają liczbę złącz, natomiast wymagają złożonego protokołu komunikacji. Projektując urządzenie do pracy z interfejsem szeregowym należy zapoznać się z ograniczeniami prędkości transmisji danego rozwiązania. Najbardziej popularnymi interfejsami szeregowymi są *I²C* oraz *SPI* [24].

I²C lub *IIC* (ang. *Inter-Integrated Circuit*) jest przykładem magistrali szeregowej, wykorzystującej 2 linie do komunikacji. Znaczącą przewagą nad innymi interfejsami jest możliwość wymiany danych pomiędzy wieloma urządzeniami bez potrzeby prowadzenia indywidualnych połączeń. Wszystko odbywa się na tej samej magistrali odpowiednio rozszerzając protokół komunikacji. Dzięki takiemu rozwiązaniu interfejs zyskał znaczną popularność wśród osób zaczynających przygodę z elektroniką. Niewątpliwą wadą magistrali *I²C* jest jej maksymalna szybkość transmisji. Dla dwukierunkowej wymiany danych jest to 3,4 Mbit/s. Nie każde urządzenie obsługujące tę magistralę ma możliwość komunikacji z taką szybkością. Zważając na ograniczenia interfejsu *I²C* znajduje on swoje zastosowanie na krótkich dystansach. Magistrala *IIC* jest chętnie wykorzystywana w układach pomiarowych, z dużą liczbą sensorów [21, 28].

Alternatywą do wolnego *I²C* jest interfejs *SPI*. Specjalistyczne układy wykorzystujące tę magistralę są w stanie komunikować się z prędkościami przekraczającymi 100 Mbit/s. Z drugiej strony *SPI* w swojej specyfice posiada szereg wad, które trzeba rozpatrzyć w trakcie projektowania. Znaczącą różnicą od wymienionego wcześniej *I²C* jest dwukrotnie większa liczba połączeń. Wspólna dwukierunkowa szyna danych zostaje rozbита na dwie jednokierunkowe tj. *MISO* (ang. *Master Input Slave Output*) oraz *MOSI* (ang. *Master Output Slave Input*). Sygnał zegarowy jest prze-

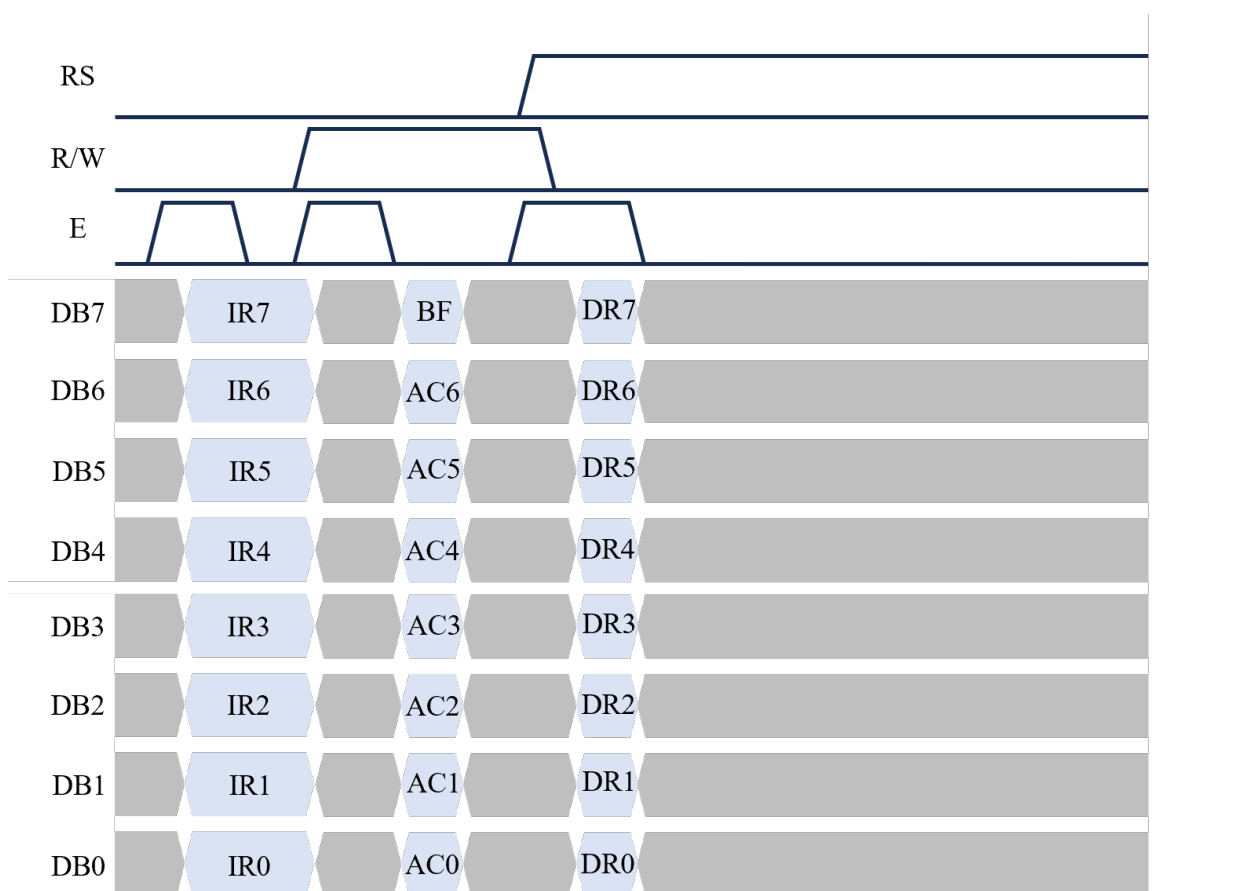
syłany linią SCK (ang. *Source Clock*). Najważniejszą zmianą pomiędzy wymienionymi interfejsami jest wybór urządzenia podrzędnego. Dużą zaletą magistrali I²C jest jej minimalistyczne podejście, gdzie jedną szyną wywołuje się wymianę danych z żądanym układem slave. Gdy chcemy zmienić rozmówcę wszystko odbywa się tą samą linią. Sprawa ma się zupełnie inaczej przy SPI. Ostatnim z złącz tego interfejsu jest SS (ang. *Slave Select*). Każde z urządzeń podrzędnych ma indywidualne połączenie z swojego SS do odpowiednio przygotowanego pinu w urządzeniu nadrzędnym. Przykładowo tworząc system z 3 czujnikami i mikrokontrolerem dla interfejsu SPI trzeba zająć 6 pinów, gdzie dla I²C to 2 połączenia. Stanowi to ogromną różnicę, zwłaszcza przy projektowaniu płytki drukowanej. Magistrala SPI jest częstym rozwiązaniem w szybkich pamięciach flash, wyświetlaniu wymagającej szybkiego odświeżania grafiki czy specjalistycznych układach pomiarowych [20, 26].



RYSUNEK 2.5: 16-pinowe złącze wyświetlacza [29]

Na rysunku 2.5 przedstawiono 16-pinowe złącze wyświetlacza pracującego z interfejsem równoległym. Każdy z pinów pełni określoną rolę. VSS to wyprowadzenie masy układu. VDD to zasilanie (typowo 5 V). V0 to regulacja kontrastu. A to anoda diody podświetlenia ekranu, natomiast K to katoda. Przez wyprowadzenia D0 - D7 odbywa się komunikacja równoległa, przesyłane są tu dane oraz komendy. Numer podany, stojący przy literze D odpowiada indeksowi bitu w bajtowej ramce danych. Na załączonym rysunku widnieją również trzy porty sterujące RS, RW, oraz E, które zostaną omówione w dalszej części rozdziału.

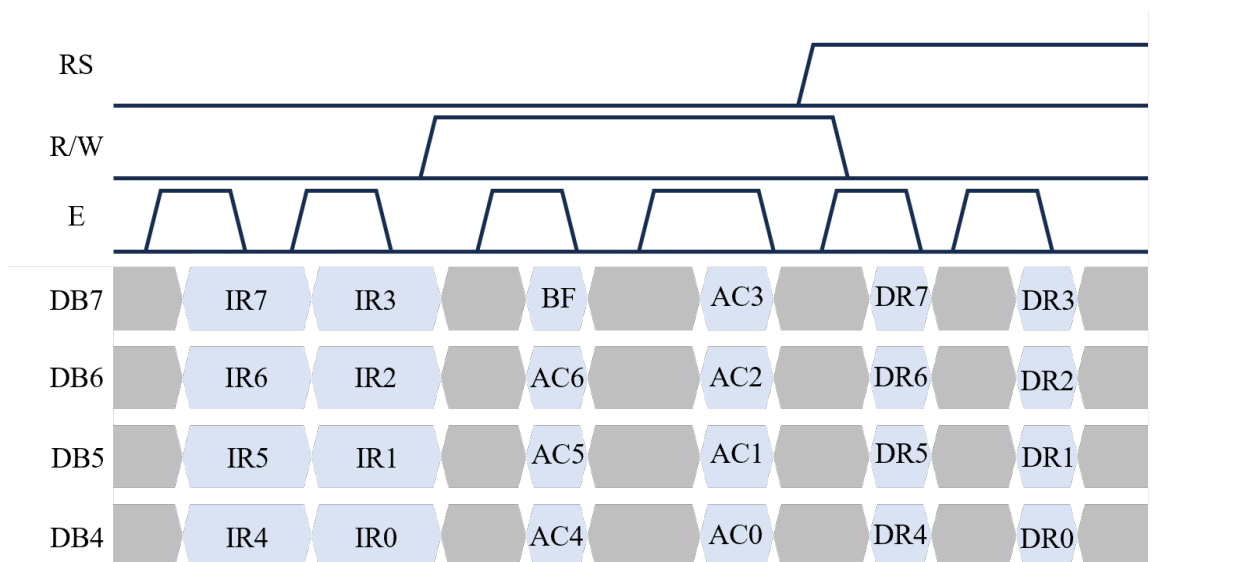
2.3 Transmisja poleceń i danych



RYСУNEK 2.6: Wykres czasowy, przedstawiający transmisję równoległą 8-bitową [11]

W 8-bitowej równoległej transmisji danych wykorzystywany jest cały zestaw wyprowadzeń danych wyświetlacza (DB7 - DB0). Przesyłany jest cały bajt danych w jednym impulsie Enable. Występują też trzy linie sterujące:

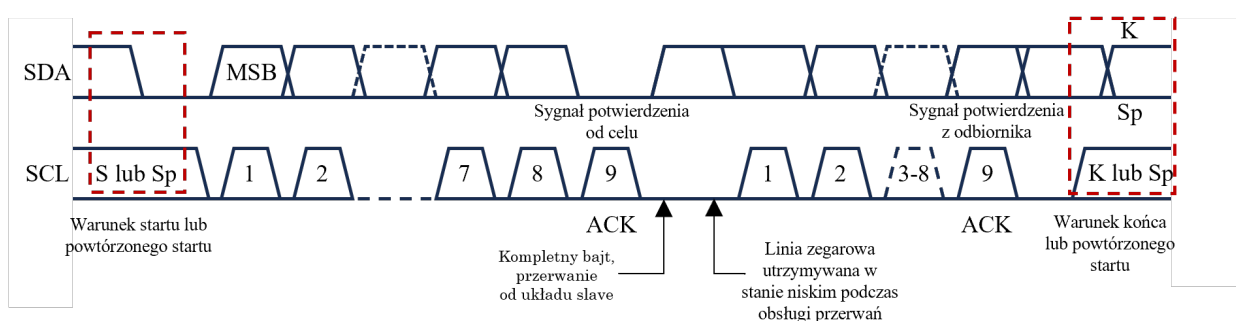
- RS (ang. *Register Select*) - pozwala na pracę w trybie instrukcji (na rysunku 2.6 oznaczone jako IR7 - IR0) lub trybie danych (oznaczone jako DR7 - DR0). Instrukcje to inaczej polecenia, które ma wykonać sterownik wyświetlacza (np. wyczyść matrycę, lub ustaw kursor na danej pozycji, itp.). Dane to kod ASCII litery, która ma być wyświetlona lub definicja pikseli dla własnego znaku. Dla niskiego stanu (0 V) wyświetlacz przyjmuje instrukcje. Jeżeli jest podany stan wysoki (5 V), to wyświetlacz przyjmuje dane.
- R/W (ang. *Read/Write*) - gdy jest podany niski stan, wyświetlacz jest w trybie zapisu. Przyjmuje wówczas dane oraz komendy. Stan wysoki ustawia sterownik w tryb odczytu, wyświetlacz, przesyła wtedy na pinie D7 flagę zajętości (na rysunku 2.6 - BF) oraz na pozostałych (D6-D0) licznik adresu (AC6 - AC0), przy czym, że pin RS jest ustawiony na 0. W przypadku, gdy RS ma stan 1, wyświetlacz odsyła kod ASCII wybranego znaku.
- E (ang. *Enable*) - sygnał wyzwalający przyjęcie komendy lub danych. Sterownik odczytuje dane z pinów w momencie opadającego zbocza (zmiany stanu z wysokiego na niski).



RYSUNEK 2.7: Wykres czasowy, przedstawiający transmisję równoległą 4-bitową [11]

Twórcy układu *HD44780* przewidzieli, że stosowanie 8 linii do transmisji poleceń dla wyświetlacza znacząco wpływa na dostępność portów w mikrokontrolerze, dlatego zdecydowali się wprowadzić tryb 4-bitowy, który częściowo ten problem rozwiązuje. Dane są wówczas fizycznie przesyłane przez piny D7 - D4, natomiast pozostałe wyprowadzenia (D3 - D0) pozostają niepodłączone.

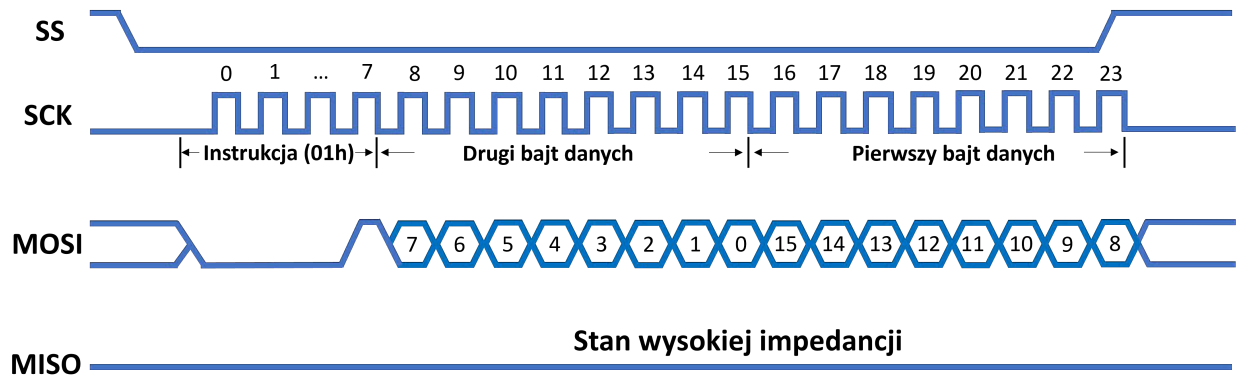
Przesył komend lub danych odbywa się poprzez podział bajtu na dwie części. Jak można zauważyć na rysunku 2.7, w trakcie pierwszego impulsu sygnału Enable przesyłana jest starsza część bajtu instrukcji (IR7 - IR4) lub danych (DR7 - DR4). Następnie, przy drugim impulsie Enable, przesyłana jest młodsza część bajtu instrukcji (IR3 - IR0) lub danych (DR3 - DR0). Należy podkreślić, że stan linii RS oraz R/W nie może ulec zmianie w trakcie przesyłania obu połówek jednego bajtu [11].



RYSUNEK 2.8: Wykres czasowy, przedstawiający transmisję szeregową dla I²C [21]

W standardzie I²C wykorzystywane są tylko dwie linie SDA (dane) oraz SCL (zegar). Aby wywołać konkretne urządzenie *master* wysyła warunek startu. Zegar jest w stanie wysokim, w tym czasie linia danych zmienia swój stan z wysokiego na niski. W takim wypadku odbiorniki interpretują to jako początek transmisji. Pierwszy bajt wysłany przez nadajnik to siedem bitów adresu oraz najmniej znaczący, zarezerwowany bit operacji (R/W - zapis, odczyt). Po nadaniu ósmego bitu *master* zwalnia linię SDA pozwalając jej wrócić do stanu wysokiego. Wygenerowany zostaje

dziewiąty impuls zegara. W tym czasie odbiornik musi ściągnąć linię SDA do masy, aby potwierdzić odebranie danych (nadać ACK ang. *Acknowledge*).



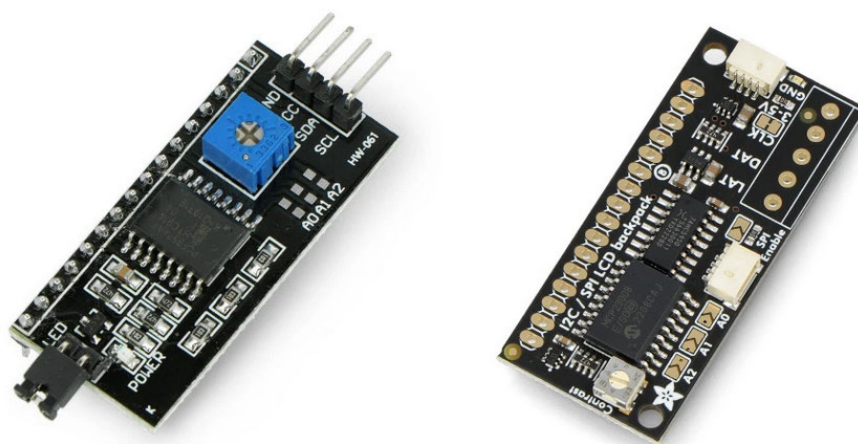
RYSUNEK 2.9: Wykres czasowy, przedstawiający transmisję szeregową dla SPI [20]

Zapis w magistrali SPI odbywa się poprzez *aktywację układu* linią SS - (ang. *Slave Select*) ustawioną w stan niski. Zaraz po tym *master* wysyła odpowiednią instrukcję zapisu szyną MOSI (01h), a następnie jeden lub dwa bajty właściwych danych, które trafiają do odpowiedniego rejestru urządzenia. W trybie zapisu układ slave nie wysyła żadnych odpowiedzi, a *master* ignoruje wszelkie sygnały przychodzące. Po zakończeniu transmisji nadajnik dezaktywuje linię SS. Zegarem steruje *master*. To on ustala prędkość transmisji. Odbiornik nigdy nie generuje swojego sygnału SCK. Jeżeli master przerwie nadawanie zegara, z powodu obsługi innych zadań, wtedy slave czeka, aż sygnał zegara wróci, aby mógł dokończyć obiór danych. Na rysunku 2.9 przedstawiony jest tryb nadawania *SPI Mode 0*. Oznacza to, że polaryzacja spoczynkowa zegara to stan niski. Natomiast dane odczytywane są na pierwszym narastającym zboczach zegara [9].

2.4 Konwertery I²C/LCD

W trakcie wyboru wyświetlacza można natrafić na ekrany z dodatkowym modulem wlutowanym do wyprowadzeń płytki. Zazwyczaj dołożony moduł jest konwerterem X/LCD. Głównym urządzeniem na dodatkowej płytce jest tzw. ekspander wejść/wyjść. Zastosowanie konwertera pozwala na sterowanie z wyświetlaczem zaprojektowanym na interfejs równoległy z użyciem komunikacji szeregowej. Takie rozwiązanie stanowi pogodzenie charakterystyki wyświetlacza z minimalizacją połączeń.

Konwerter oparty na protokole I²C pozwala na sterowanie wyświetlaczem w trybie 4 - bitowym. Redukuje on liczbę niezbędnych połączeń do dwóch - SDA, dla danych, oraz SCL dla zegara. Posiada potencjometr do regulacji kontrastu, trzy pola lutownicze do ustalenia adresu układu PCF8574 oraz zworkę do włączania zasilania podświetlenia. Na wyjściu ma 16 pinów odpowiadających wyświetlaczowi ze sterownikiem HD44780.

RYSUNEK 2.10: Konwertery I²C oraz I²C/SPI [3]

Rozbudowaną wersją jest konwerter z interfejsami I²C/SPI. Za pomocą zworki (pola lutowicznego) można wybrać tryb transmisji I²C lub SPI. Interfejs I²C, obsługuje układ *MCP23008*. Podobnie jak poprzedni konwerter umożliwia ustawienie adresu układu. W przypadku wybrania trybu SPI wykorzystywany jest układ *74LS595D*. Na płytce posiadają one wspólne wejścia oraz wyjścia, co wymusza na programiście wybranie tylko jednego z tych układów. Tutaj również występuje regulacja kontrastu przy pomocy potencjometru, a włączenie podświetlenia odbywa się z jednego z pinów wyjściowych układów *MCP23008* oraz *74LS595D* [26].

Rozdział 3

Ekspandery wejść/wyjść równoległych

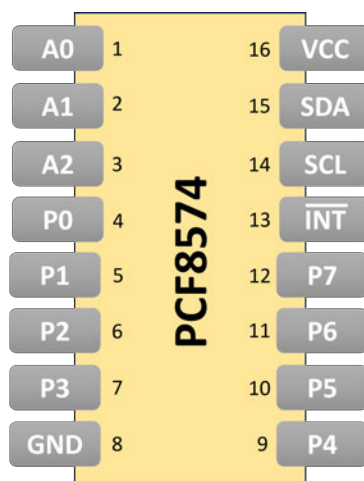
3.1 Zadania ekspanderów w systemach mikroprocesorowych

Projektując układ, można napotkać sytuację, gdy wybrany mikrokontroler ma zbyt małą liczbę portów wejść/wyjść. Pierwszym nasuwającym się rozwiązaniem jest zakup urządzenia z bogatszym asortymentem pinów. Istnieje jednak możliwość rozszerzenia możliwych połączeń bez konieczności zmiany jednostki głównej.

Ekspander wejść/wyjść to dosłownie port rozszerzeń pinów GPIO (ang. *General Purpose Input Output*). Takie urządzenia są projektowane do pracy z magistralą szeregową. Wykorzystując interfejs szeregowy, który zajmuje relatywnie mało połączeń, można powiększyć zasób dostępnych wyprowadzeń (zazwyczaj 8 dodatkowych pinów, ale zależy to od wybranego układu scalonego). Na rynku dostępnych jest wiele układów, które pozwalają na ekspandowanie portów. Urządzenia różnią się od siebie liczbą użytecznych pinów, obsługiwanyymi interfejsami czy sygnałami sterującymi [10].

3.2 Układy PCF8574

Przy opracowywaniu modułów wykorzystano ekspandery PCF8574. Jest to 8 bitowy rozszerzacz portów I/O, który do komunikacji wykorzystuje magistralę I²C. Urządzenie może być zasilane napięciem od 2,5 do 6 woltów, dzięki czemu ułatwia proces projektowania sekcji zasilania.



RYSUNEK 3.1: Rozkład wyprowadzeń układu PCF8574 [16]

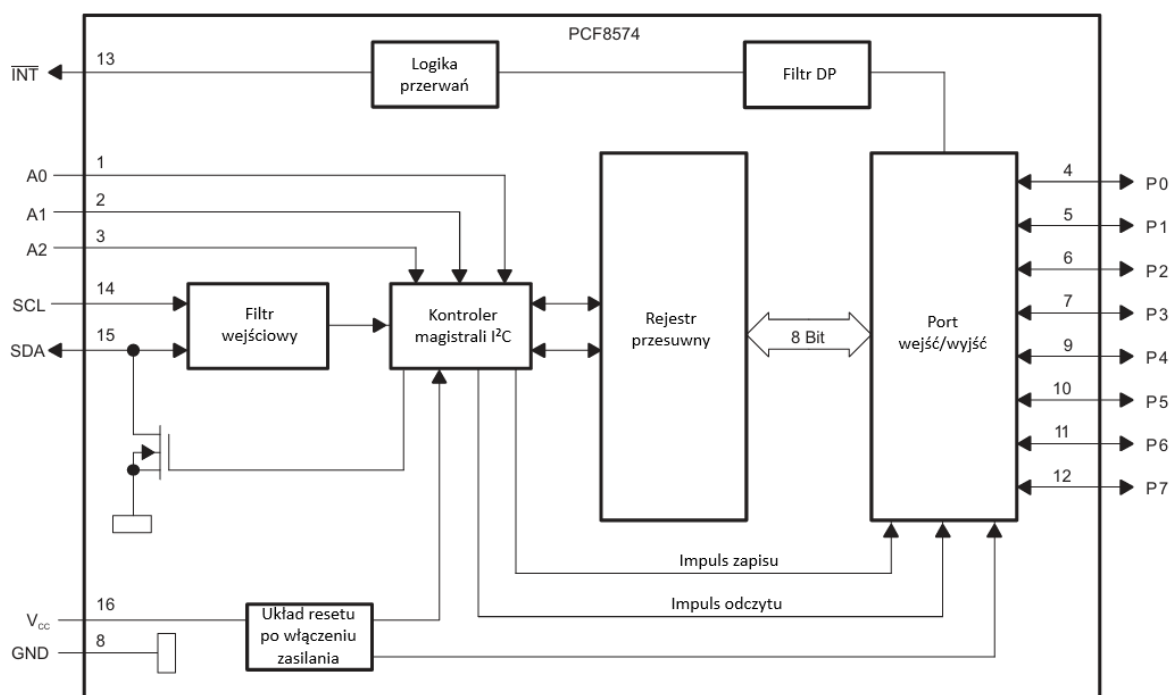
Na rysunku 3.1 przedstawiono rozkład wyjść ekspandera. Piny 1 - 3 służą do ustawienia adresu urządzenia. Kolejne 4 wyprowadzenia to połowa bajtu danych. Ósmym wyjściem jest masa układu. Kolejny półbajt danych stanowią piny od 9 do 12. Wyprowadzenie numer 13 to port przerwań. Pin 14 i 15 obsługują komunikację magistralą I²C, z czego pierwszym jest SCL, a następnym SDA. Ostatnim wyjściem jest linia zasilania.

Port wejść/wyjść został zrealizowany za pomocą quasi dwukierunkowych wejść/wyjść - quasi-bidirectional I/O. Takie rozwiązanie eliminuje potrzebę stosowania dodatkowych rejestrów konfiguracyjnych. Stan danego pinu jest definiowany przez napięcie, które się na nim znajduje. Gdy ekspander jest włączony wszystkie wyprowadzenia są w stanie wysokim, przez zastosowanie rezystora podciągającego, wymuszającego natężenie prądu o wartości $100\mu A$. Każdy pin może pracować z sygnałami o maksymalnym natężeniu prądu 25 mA.

Wymiana danych pomiędzy PCF8574, a mikrokontrolerem zaczyna się typowym sygnałem startu dla magistrali I²C. Linie SDA i SCL przed rozpoczęciem transmisji mają stan wysoki. Ściągnięcie SDA do stanu niskiego przy jednoczesnym stanie wysokim na SCL zaczyna proces komunikacji. W kolejnych taktach przesyłany jest 7 bitowy adres. Ramkę kończy bit kierunku komunikacji. Gdy jest to 1, układ przesyła sygnały podłączone do wyprowadzeń P0-P7 (tryb read). W przeciwnym przypadku na piny I/O są przekazywane dane z mikrokontrolera (tryb write).

Przykładem jak działa typowa transmisja danych będzie wymiana pomiędzy mikrokontrolerem, a ekspanderem (tryb write). Dla układu PCF8574 pierwsze 4 bity adresowe (licząc od najstarszego bitu) są stałe tworząc ciąg 0100. Następne 3 pola są definiowane przez stan napięć na wejściach A0, A1 i A2. Bajt jest zakończony zerem ($R/\overline{W}$). Po przesłaniu pierwszego bajtu ekspander o danym adresie odsyła potwierdzenie odebrania danych ang. *acknowledge*. Następnie jest przesyłany bajt surowych danych, które będą reprezentować 8 wyprowadzeń ekspandera (stan pojedynczego bitu to jeden pin). Poprawność odebrania danych jest sygnalizowana potwierdzeniem od układu PCF8574 (ponieważ on odbiera dane). Jeżeli chcemy zakończyć transmisję należy to zrobić wysyłając charakterystyczny dla protokołu I²C sygnał STOP. Jest to podciągnięcie linii SDA do stanu wysokiego, przy jednoczesnym stanie wysokim szyny SCL. W przypadku gdy chcemy odczytywać stany wejść ekspandera (tryb read) to mikrokontroler musi potwierdzać tylko odebrane dane (adres dalej jest potwierdzany przez PCF8574).

Układ omawianego ekspandera posiada linię przerwań $\overline{INT}$. Jest to sprzętowe rozwiązanie problemu sprawdzania przez urządzenie nadrzędne wszelkich zmian zachodzących w trakcie sesji komunikacyjnej. Pozwala to na rzadsze ingerowanie mikrokontrolera, co oszczędza czas. PCF8574 sam informuje o zmianach zachodzących na porcie wejść/wyjść. Wspomniana linia $\overline{INT}$ pracuje w konwencji otwartego drenu ang. *open-drain*. Uznajemy, że linia jest aktywna, gdy zaobserwujemy stan niski. Działanie linii przerwań najlepiej wyjaśni wspomniany wcześniej przykład transmisji. W momencie startu linia $\overline{INT}$ jest aktywna. Potwierdzenie ze strony PCF8574 po pierwszym bajcie może powodować podciągnięcie linii przerwań do stanu wysokiego (wyłączenie jej). Następnie są przesyłane dane. Przerwania są bardzo ważne w trybie odczytu (read). Wykorzystanie tego rozwiązania pozwala na informowanie mikrokontrolera o zmianach występujących na wejściach ekspandera. Przykładem wykorzystania tego pomysłu jest reakcja układu na kliknięcie przycisku. Głównym zastosowaniem przerwań jest alarmowanie o zmianach i kasowanie swojej aktywności po potwierdzeniach (informujemy mikrokontroler, że nie musi się skupiać na tym co się dzieje w danej sesji).



RYSUNEK 3.2: Schemat blokowy układu PCF8574 [16]

Na rysunku 3.2 przedstawiono schemat blokowy ekspandera PCF8574. Układy filtrów eliminują szumy i zwiększają poprawność odebranych/nadawanych danych. Kontroler magistrali I²C realizuje tłumaczone poprzednio schematy protokołu komunikacyjnego. PCF8574 posiada rejestr przesuwany, do którego są przekazywane dane zależnie od kierunku transmisji. Wspomniany rejestr współpracuje z portem wejść/wyjść, gdzie podłączamy peryferia. Blok logiki przerwań spełnia zadania związane z tłumaczonymi wcześniej reakcjami wyjścia $\overline{INT}$.

Układ PCF8574 to bardzo popularne urządzenie, które znajduje wiele zastosowań. Ekspander jest wykorzystywany między innymi w konwerterach do wyświetlaczy LCD, obsłudze klawiatur, sterowaniu modułami przekaźników czy agregowaniu sygnałów czujników. Pomimo tego PCF8574 ma kilka wad, które stawiają znak zapytania w kontekście doboru odpowiedniego układu uwzględniając potrzeby projektowe. Urządzenie nie posiada rejestrów sterujących, przez co ma ograniczone możliwości konfiguracji. Prostota PCF8574 jest jednocześnie jego ogromną zaletą, jak i wadą. Z myślą o bardziej wymagających projektach powstały ekspandery, których przykładowy reprezentant zostanie przedstawiony w kolejnym rozdziale [16].

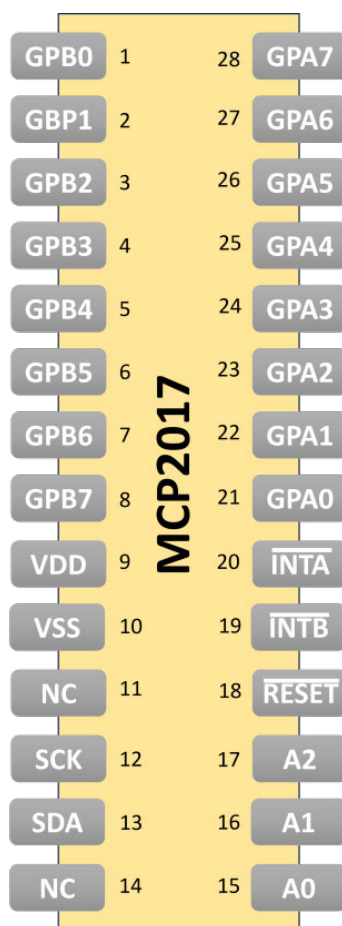
3.3 Układy MCP23x17

Układy z serii MCP23x17 to 16-bitowe ekspandery wejść/wyjść. W miejscu oznaczenia x kryje się obsługiwany interfejs komunikacji z układem. Gdy x wynosi 0, to I²C, natomiast literka S to magistrala SPI. W porównaniu do układu PCF8574, MCP23x17 daje ogromne możliwości konfiguracji. W tabeli 3.1 przedstawiono rejestry sterujące układu MCP23x17. Najważniejsze to IODIRA oraz IODIRB, umożliwiające ustawienie kierunku pinów portów. GPIOA oraz GPIOB pozwalają na odczyt stanów logicznych pinów, natomiast OLATA, OLATB pozwalają na zapis do pinów. W omawianym układzie to nie tylko 8-bitów danych oraz bit trybu decyduje o stanie pinów. Tutaj

TABELA 3.1: Tabela przedstawiająca rejestry sterujące w układach MCP23x17 [14]

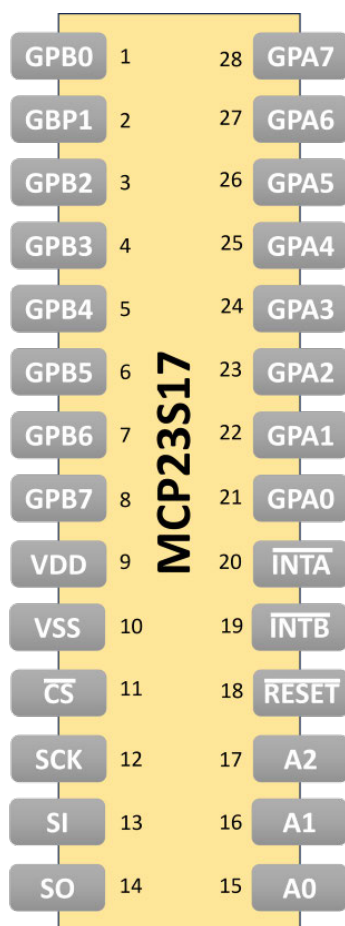
Adres (Hex)	Nazwa Rejestru	Opis
0x00	IODIRA	Kierunek pinów Portu A (1 = Wejście, 0 = Wyjście)
0x01	IODIRB	Kierunek pinów Portu B
0x02	IPOLA	Polaryzacja wejść Portu A (odwracanie stanów logicznych)
0x03	IPOLB	Polaryzacja wejść Portu B
0x04	GPINTENA	Włączenie przerwań dla pinów Portu A
0x05	GPINTENB	Włączenie przerwań dla pinów Portu B
0x06	DEFVALA	Domyślna wartość do porównań przerwań (Port A)
0x07	DEFVALB	Domyślna wartość do porównań przerwań (Port B)
0x08	INTCONA	Konfiguracja sposobu generowania przerwań (Port A)
0x09	INTCONB	Konfiguracja sposobu generowania przerwań (Port B)
0x0A	IOCON	Rejestr konfiguracji globalnej (wspólny) m.in. flaga SEQOP i BANK
0x0B	IOCON	Kopia rejestru konfiguracji globalnej
0x0C	GPPUA	Włączenie wewnętrznych rezystorów Pull-Up (Port A)
0x0D	GPPUB	Włączenie wewnętrznych rezystorów Pull-Up (Port B)
0x0E	INTFA	Flaga przerwania - wskazuje, który pin wywołał przerwanie (Port A)
0x0F	INTFB	Flaga przerwania (Port B)
0x10	INTCAPA	Przechwycony stan portu A w momencie wystąpienia przerwania
0x11	INTCAPB	Przechwycony stan portu B w momencie wystąpienia przerwania
0x12	GPIOA	Odczyt stanu logicznego pinów Portu A
0x13	GPIOB	Odczyt stanu logicznego pinów Portu B
0x14	OLATA	Zatrząsk wyjściowy Portu A (Zapis do pinów)
0x15	OLATB	Zatrząsk wyjściowy Portu B (Zapis do pinów)

programista musi pamiętać o wpisaniu danych do rejestrów, a nie bezpośrednio na piny. Powoduje to dodatkowy nakład pracy od programisty oraz konieczność podstawowej znajomości układu. Warto wspomnieć, że 16 portów układu jest podzielone na dwa bloki 8-bitowe GPA oraz GPB. Dodatkowo, każdy z tych portów może być całkowicie indywidualnie skonfigurowany jako wejście lub wyjście. W opisywanych układach odeszło się od quasi-dwukierunkowych portów, na rzecz prawdziwych w pełni konfigurowalnych portów wejścia/wyjścia. Każdy z tych pinów potrafi dostarczyć lub przyjąć prąd o natężeniu do 25 mA . Należy pamiętać jednak, że maksymalny prąd wpływający przez pin zasilania VDD to 125 mA , a wypływający przez VSS to 150 mA , trzeba zatem być ostrożnym, aby nie przekroczyć sumarycznych prądów. Dla każdego pinu skonfigurowanego jako wejście, można programowo włączyć rezystor podciągający *Pull-up* o wartości około $100\text{ k}\Omega$. Układy z serii MCP23x17 posiadają zaawansowany system przerwań - piny $\overline{INTA}$ dla banku GPA oraz $\overline{INTB}$ dla banku GPB. Mogą one sygnalizować zmianę stanu, na pinach wejściowych. Podobnie jest w układzie PCF8574, ale tutaj również mamy możliwość konfiguracji, jak zachowują się te piny. Możemy też odczytać przechwycone stany portów A oraz B w rejestrach odpowiednio INTCAPA oraz INTCAPB. Kolejną funkcjonalnością wartą omówienia jest *odwracanie polaryzacji*. Układy te posiadają specjalne rejestry (IPOLA, IPOLB) pozwalające odwrócić logikę wejść. Jeżeli do wejścia podłączony jest przycisk zwierający do masy, ekspander można tak skonfigurować, aby programistycznie odczytywany był stan logiczny jako wysoki. MCP23x17 są zaprojektowane tak, aby pracować z wieloma systemami. Obsługują szeroki zakres napięć od 1.8 V do 5.5 V . Niski pobór prądu w trybie *Standby* rzędu $1\text{ }\mu\text{A}$ sprawia, że to rozwiązanie nadaje się idealnie do układów zasilanych bateryjnie lub z akumulatorów.



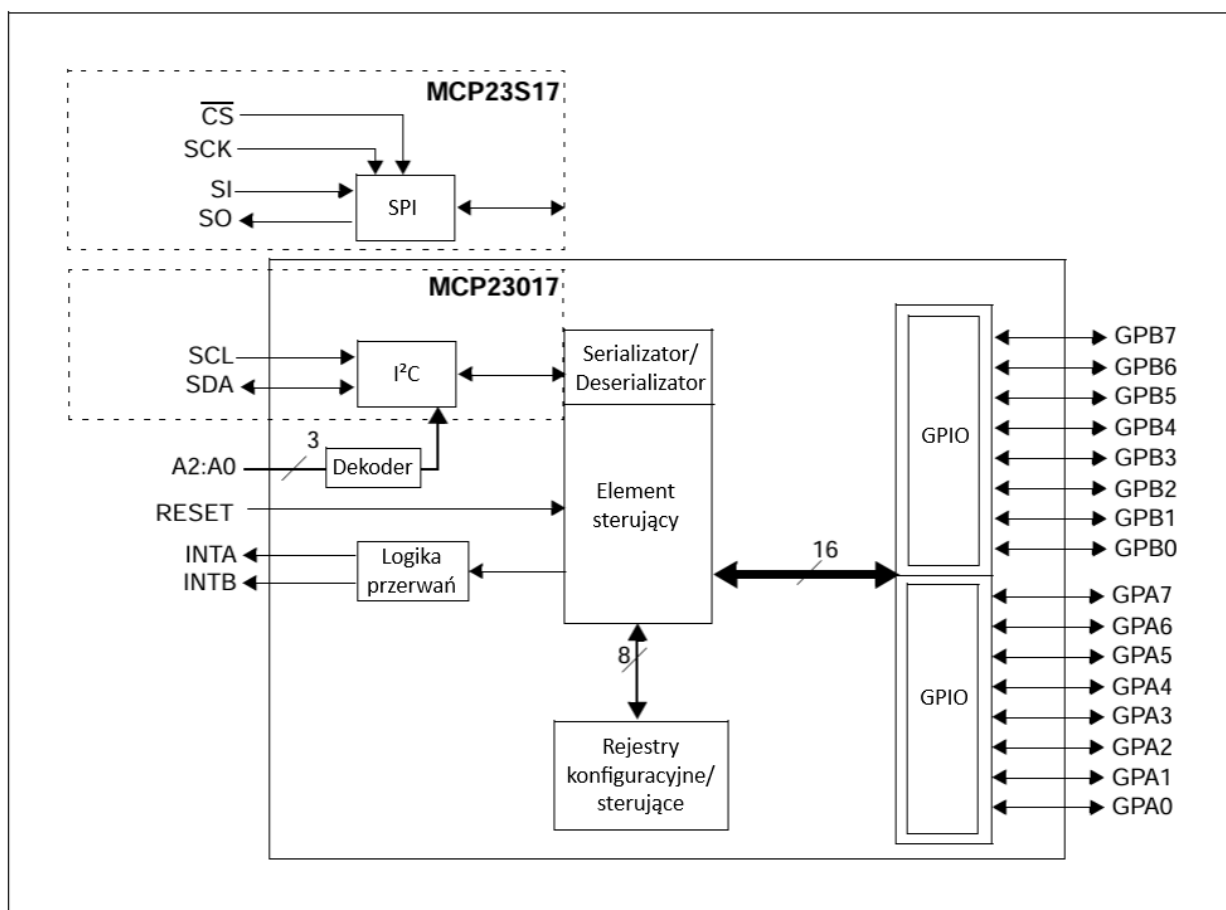
RYSUNEK 3.3: Rozkład wyprowadzeń układu MCP23017 [14]

Na rysunku 3.3 został przedstawiony rozkład wyprowadzeń dla układu MCP23017 w obudowach SOIC, SPDIP, SSOP. Piny 1-8 to wyprowadzenia wejść/wyjść dla portu B. Zasilanie jest dostarczane przez VDD oznaczone na rysunku jako pin numer 9. Masa układu znajduje się tuż obok zasilania i jest oznaczona jako VSS - pin 10. W przypadku ekspandera komunikującego się po I²C pin 12 to zegar magistrali (SCK), a pin 13 to dane (SDA). Pin 11 oraz 14 nie są podłączone. Ustawianie adresu odbywa się na pinach 15-17. Pin $\overline{RESET}$ znajduje się na pozycji 18. Piny sygnalizujące przerwanie, znajdują się odpowiednio dla $\overline{INTA}$ - 20, $\overline{INTB}$ - 19. Piny 28-21 to wyprowadzenia wejść/wyjść dla portu A.



RYSUNEK 3.4: Rozkład wyprowadzeń układu MCP23S17 [14]

W przypadku układu MCP23S17 (rys. 3.4) rozkład wyprowadzeń różni się jedynie na pinach 11-14, gdzie wyprowadzone zostały połączenia interfejsu SPI. $\overline{CS}$ (ang. *Chip Select* - wybór układu) znajduje się na pinie 11. SCK, czyli zegar magistrali jest na pozycji 12. SI (ang. *Slave Input* - Wejście danych) jest oznaczone numerem 13. SO (ang. *Slave Output* - Wyjście danych) znajduje się na pinie 14. Należy zaznaczyć, że piny adresowe (17-15) są nadal aktywne. Mimo, że jest to układ zaprojektowany do komunikacji interfejsem SPI to nadal na jednym pinie $\overline{CS}$ można połączyć wiele układów MCP23S17, co redukuje ilość koniecznych pinów do podłączenia wielu układów do mikrokontrolera. W takim przypadku, nadal jako pierwszy bajt danych należy wysłać *Optcode*, który zawiera adres i wywołuje konkretny układ MCP23S17.



RYSUNEK 3.5: Schemat blokowy układu MCP23x17 [14]

Na rysunku 3.5 został przedstawiony schemat blokowy układów MCP23017 oraz MCP23S17. Można zauważyć, że cała logika dla obydwu układów jest taka sama, różnią się jedynie modułami obsługi interfejsów (odpowiednio I²C oraz SPI). Dane komunikacyjne wchodzi na blok serializatora/deserializatora, który konwertuje przesyłane sygnały jako szeregowo na postać równoległą. W takiej formie dane przyjmuje element sterujący. Jest to główny blok logiczny. To on koordynuje pracę układu, komunikuje się przy pomocy 8-bitowej szyny danych z rejestrami konfiguracyjnymi i sterującymi, w których przechowywane są ustawienia poszczególnych wyprowadzeń. Właściwe stany logiczne są przekazywane przez 16-bitową szynę danych bezpośrednio do bloków wejścia/wyjścia (GPIO). Dzielą one dostępne linie na dwa 8-bitowe porty: Port A (wyprowadzenia GPA0-GPA7) oraz Port B (wyprowadzenia GPB0-GPB7). Architektura układu posiada również układ logiki przerwań, która pozwala na sprzętowe powiadomienia podłączonego mikrokontrolera, o zmianach stanów na wejściach za pomocą linii $\overline{INTA}$ i $\overline{INTB}$ [14].

Rozdział 4

Projekt modułu LCD z 8-bitowymi ekspanderami PCF8574

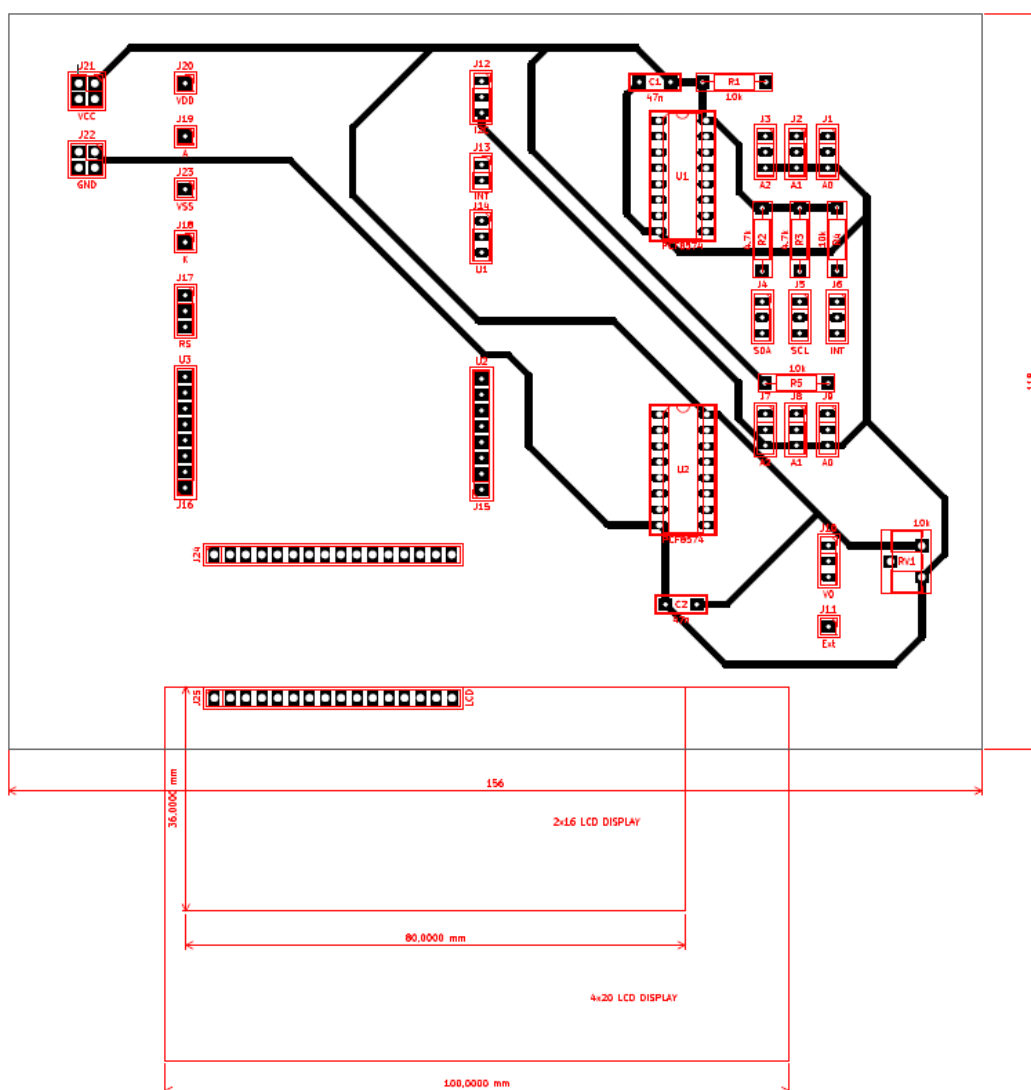
4.1 Założenia techniczne

Projektowany moduł wspierający uruchomienie wyświetlaczy LCD powinien spełniać następujące warunki:

- obsługa wyświetlaczy LCD 2x16/4x20,
- wykorzystanie 8-bitowych ekspanderów I/O,
- możliwość korzystania z interfejsu równoległego (8-bit/4-bit transmisja danych),
- możliwość korzystania z interfejsu szeregowego I²C,
- możliwość konfigurowania interfejsu na płycie modułu,
- zapewnienie intuicyjnej konfiguracji połączeń,
- zapewnienie szeregowej transmisji danych i sygnałów sterujących,
- zasilanie zewnętrzne +5 V z systemu mikroprocesorowego,
- współpraca modułu z mikroprocesorami różnych rodzin,
- możliwość współpracy z modułami platformy Arduino,
- możliwość współpracy z modułami platformy ESP32,
- możliwość dostosowania kontrastu wyświetlacza LCD,
- łatwe serwisowanie modułu.

4.2 Koncepcja modułu

W procesie tworzenia płytki drukowanej, powstało wiele koncepcji dotyczących funkcjonalności oraz rozłożenia elementów na płytce. Dzięki optymalnemu rozłożeniu elementów oraz dobremu prowadzeniu ścieżek, udało się zredukować rozmiar płytki z 156 mm szerokości, oraz 118 mm wysokości do 98 mm szerokości oraz 92 mm wysokości. W pierwszych wersjach płytek (rys. 4.1) złącza zasilania były po zewnętrznej stronie układu, w nowszych (rys. 4.3) wersjach zostały one osadzone między złączami modułu ekspandera a wyświetlaczem. Połączenia między modulem komunikacji równoległej, a układem komunikacyjnym z dwoma PCF8574 są konfigurowalne, można je połączyć wprost, aby sterować wyświetlaczem lub wykorzystać je jako piny wejściowe, dla dodatkowych układów. Połączenie jest intuicyjne, tak aby osoby początkujące miały ułatwioną możliwość konfiguracji.



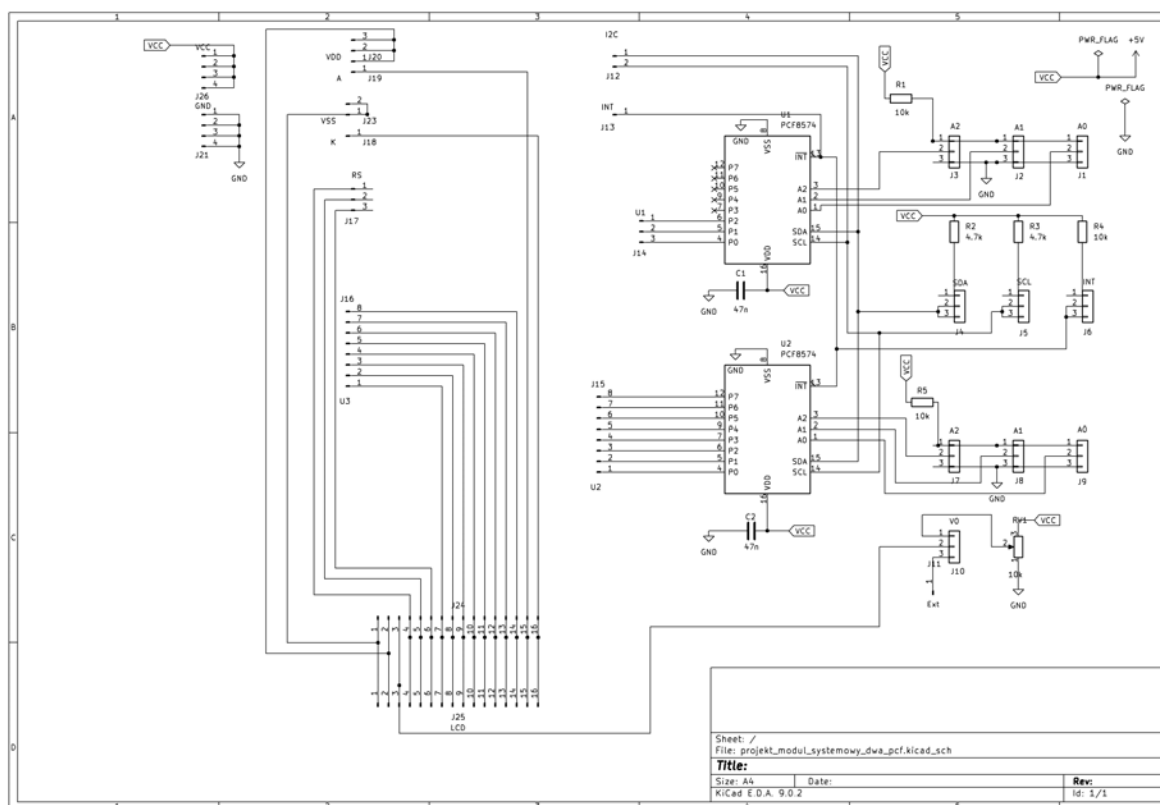
RYSUNEK 4.1: Szkic płytki z dwoma układami PCF8574, z układem elementów w wersji V1

4.3 Opis modułu

W celu ułatwienia zrozumienia zagadnień związanych ze sterowaniem standardowym wyświetlaczem, protokołów komunikacji I²C, oraz możliwościami konfiguracji systemu, została stworzona płytką drukowaną z dwoma układami PCF8574. Płytką PCB jest dedykowana dla rozmiarów wyświetlaczy 80 x 36 mm, jest to wyświetlacz z 2 wierszami oraz 16 kolumnami, oraz 98 x 60 mm - 4 wiersze, 20 kolumn. Przedstawione rodzaje wyświetlaczy mają wspólne 16-pinowe wyjście, więc moduł jest z nimi w pełni kompatybilny. Wspierana jest zarówno komunikacja równoległa 4-bitowa, 8-bitowa, ale również ze względu na dwa ekspandery - szeregową 8-bitową i 4-bitową. W konfiguracji 8-bitowej, jeden układ PCF8574 obsługuje 8 linii danych równoległych, natomiast drugi obsługuje trzy linie sterujące. W przypadku chęci wykorzystania drugiego ekspandera jako rozszerzenia wyjść mikrokontrolera, można zrezygnować z transmisji 8-bitowej na rzecz 4-bitowej transmisji. Wtedy proces komunikacji przejmuje jeden expander - 4 linie danych, oraz 3 sterujące. Drugi zostaje do dyspozycji programisty. Moduł doskonale radzi sobie z komunikacją, z płytkami ewaluacyjnymi, opartymi na mikrokontrolerach AVR, szczególnie z platformą Arduino UNO. Ze względu na możliwość konfiguracji płytki oraz jej elastyczność programista ma dużą dowolność, jeżeli chodzi o sposób podłączenia peryferiów oraz ich obsługi w kodzie. Płytką drukowaną staje się naturalnym rozszerzeniem możliwości platformy Arduino. Układ posiada możliwość konfiguracji adresów dla każdego z ekspanderów, dodatkowo można włączyć do obwodu rezystory 4,7 k Ω podciągające dla szyny I²C, oraz rezystor 10 k Ω dla złącza przerwań *INT*. Zapewniona została możliwość wyboru podciagu, ze względu na ograniczenia prądowe dla różnych układów. W przypadku wielu mikrokontrolerów istnieje też możliwość załączenia rezystorów wewnętrznych. W razie konieczności, można też skorzystać z innych rezystorów ograniczających, wyłączając wbudowane rezystory. Płytką jest zasilana napięciem +5 V. W przypadku korzystania z platformy Arduino nie jest konieczne stosowanie dodatkowych zasilaczy, ze względu na mały pobór prądu przez moduł, można go zasiląć bezpośrednio z np. Arduino UNO. Zostały opracowane wersje modułów zarówno z soldermaską pod wytrawianie przez firmy zewnętrzne oraz z frezowaniem ścieżek, które mogą zostać wykonane bezpośrednio na uczelni, ograniczając potrzebę wykorzystania skomplikowanego procesu technologicznego.

Na rysunku 4.2 został przedstawiony schemat ideowy płytki drukowanej z dwoma układami PCF8574. Jest to układ znacznie bardziej zaawansowany, niż popularne tanie moduły "I²C Backpack", wykorzystujące jeden moduł PCF8574 i pozwalające jedynie na komunikację 4-bitową. Rozwiązanie opracowane w ramach tej pracy obejmuje dwa układy PCF8574, co pozwala na realizację pełnej transmisji 8-bitowej. Jest to świetne rozwiązanie do nauki oraz eksperymentowania z wyświetlaczami opartymi na sterowniku HD44780U. Komunikacja 8-bitowa jest znacznie prostsza w implementacji programowej po stronie mikrokontrolera. Na schemacie widoczny jest również podział ról. Expander U2 obsługuje linie danych. Jego porty P0-P7 są podłączone poprzez złącze J15 oraz szynę U3 do pinów D0-D7 wyświetlacza LCD (piny 7-14 na złączu J25). Expander U1 obsługuje linie sterujące. Jego porty P0-P2 są doprowadzone na piny 4-6 złącza J25 (LCD).

Wyprowadzenia magistrali I²C znajdują się na pinach oznaczonych J12 (SDA-1, SCL-2). Złącza te są podłączone do zworek J4 (SDA) oraz J5 (SCL), które umożliwiają włączenie lub wyłączenie rezystorów 4,7k Ω , przydatne gdy już inne urządzenia posiadają swój pull-up. Obok wyprowadzenia magistrali szeregową znajduje się pin J13 obsługujący przerwania ($\overline{INT}$) oraz odpowiadająca mu zworka J6 pozwalająca sterować obwodem pull-up dla przerwań (rezystor 10k Ω).



RYSUNEK 4.2: Schemat modułu z dwoma układami PCF8574

Każdy z układów posiada zworki konfigurujące sprzętowy adres magistrali (J1, J2, J3 dla U1 oraz J7, J8, J9 dla U2). Rozwiązanie to pozwala na uniknięcie konfliktów adresów na magistrali i precyzyjne przypisanie różnych adresów bazowych dla bloku sterowania i bloku danych.

Zastosowanie układów scalonych wykonanych w technologii CMOS, wymaga eliminacji zakłóceń wysokoczęstotliwościowych w liniach zasilających, które powstają podczas szybkiego przełączania stanów logicznych. Każdy z układów PCF8574 posiada kondensator odsprężający (C1 oraz C2) [27] o pojemności 47 nF . Za poprawną czytelność znaków na wyświetlaczu odpowiada potencjometr montażowy RV1 o rezystancji $10\text{ k}\Omega$ dołączony do pinu V0 wyświetlacza. Działa on jako dzielnik napięcia, między linią VCC a masą układu. Jego włączenie do układu jest również konfigurowalne na poziomie sprzętowym za pomocą zworki J10. Umożliwia ona wybór między wewnętrznym potencjometrem, a zewnętrznym źródłem odniesienia oznaczonym jako Ext. Złącza J18 oraz J19 pozwalają na elastyczne zarządzanie zasilaniem anody i katody podświetlenia. Dzięki zastosowaniu zworek możliwe jest stałe podłączenie podświetlenia do zasilania lub wyprowadzenie sterowania na zewnątrz.

4.4 Płytką PCB z soldermaską

Zaprojektowany moduł stanowi uniwersalną płytkę bazową pełniącą rolę zaawansowanego konwertera transmisji zarówno szeregowej na równoległą, jak i równoległej na szeregową. Płytką została zaprojektowana do celów ewaluacyjnych i rozwojowych. Została przewidziana duża liczba wyprowadzeń testowych oraz zworek konfiguracyjnych.

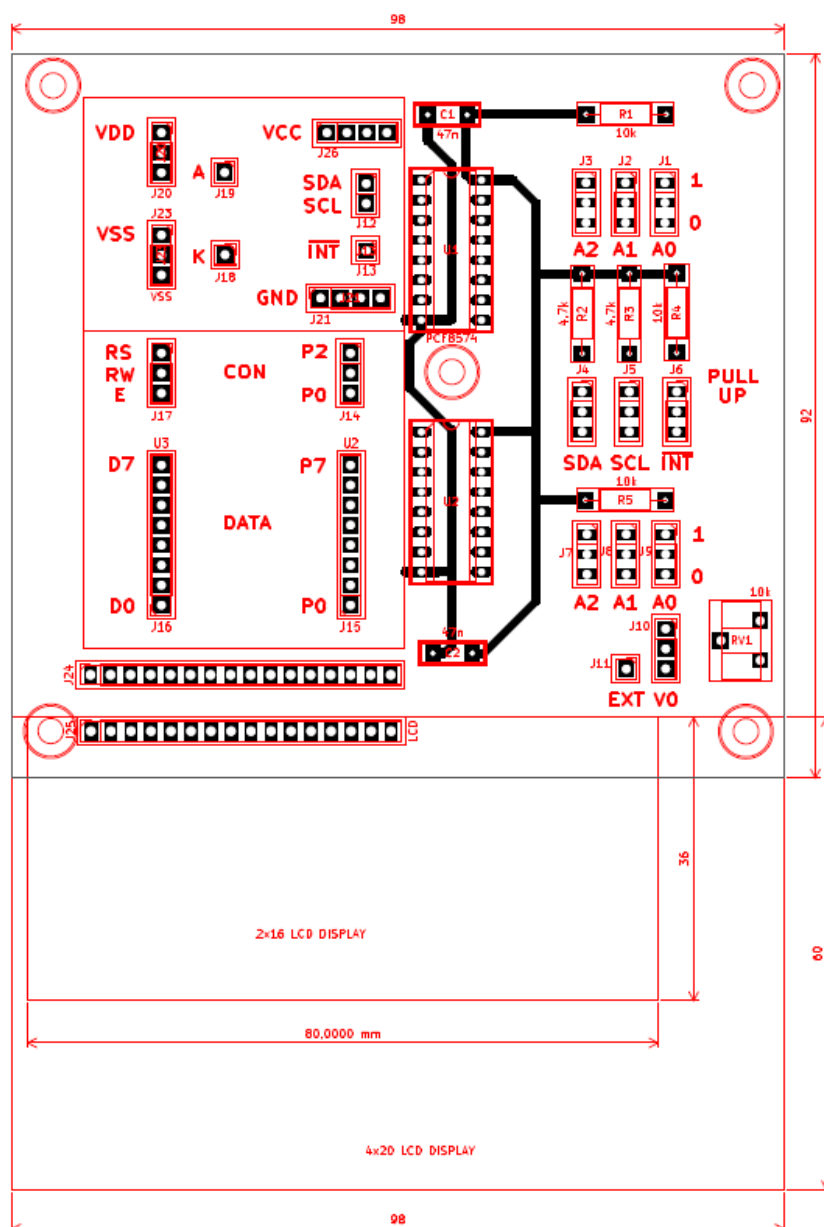
Główny obrys płytki drukowanej (rys. 4.2) wynosi 98 na 92 mm. Posiada ona cztery otwory montażowe po rogach płytki oraz jeden w środku. Pozwala to na stabilne zamontowanie płytki do obudowy. Na projekcie naniesiono obrysy referencyjne dwóch popularnych typów wyświetlaczy 2x16 oraz 4x20. Wyświetlacz podłączony do złącza J25 wystaje poza płytkę. Daje to oszczędność miejsca w obudowie, oraz pełen wgląd w strukturę płytki, w razie koniecznych modyfikacji połączeń lub konfiguracji zwerek.

Warstwa górna przeznaczona jest na montaż elementów przewlekanych (THT). Znajdują się tu korpusy układów scalonych, złącza, zworki, rezystory, kondensatory oraz potencjometr. Na tej warstwie znajdują się również ścieżki zasilające układy oraz połączenia masy.

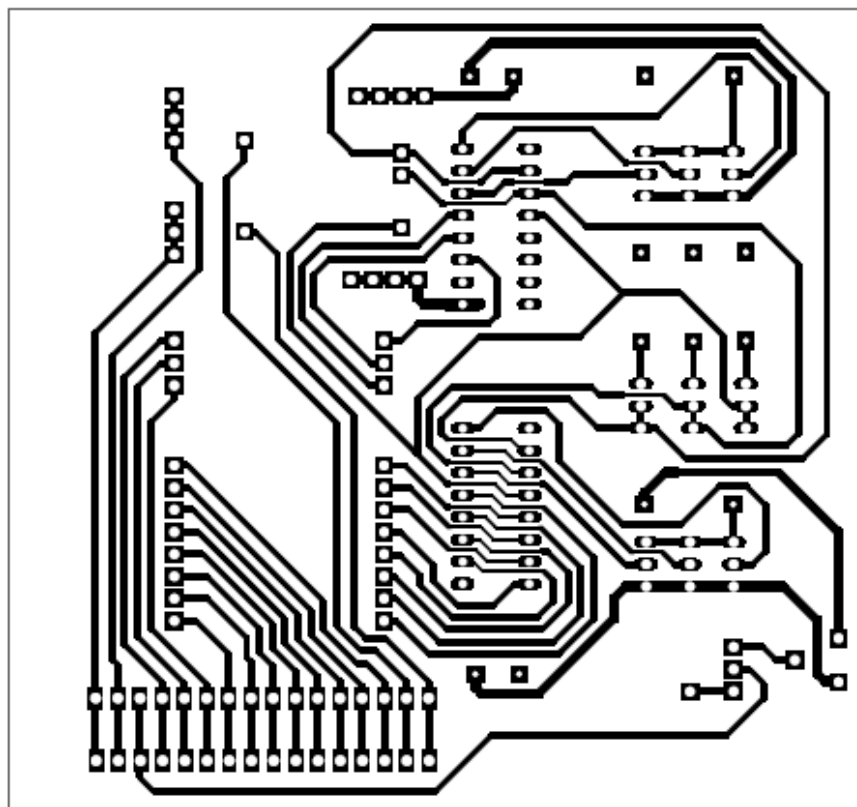
Elementy na płytce zostały umieszczone uwzględniając podział na bloki funkcjonalne. Układy scalone U1, U2 oraz ich kondensatory odprężające zostały umieszczone w centrum płytki, co minimalizuje długość ścieżek sygnałowych do złącz konfiguracyjnych. Złącza te zostały zgrupowane w lewej części płytki. Podłączenie ich jest bardzo intuicyjne, aby płytka zadziałała w trybie szeregowej transmisji danych wystarczy połączyć złącze z lewej strony do złącza odpowiadającego z prawej strony. W przypadku chęci skorzystania z transmisji równoległej, wystarczy wpiąć się w złącza po prawej stronie bloku konfiguracyjnego. Sekcja komunikacji z wyświetlaczem została wyraźnie rozdzielona od sekcji zasilania oraz komunikacji z układami PCF8574. Po prawej stronie płytki znajduje się blok konfiguracyjny. To właśnie na tej stronie znajdują się zworki konfiguracyjne oraz rezystory pull-up. W tym bloku można skonfigurować adres układów PCF8574 oraz włączyć lub wyłączyć sprzętowy pull-up dla szyny I²C lub linii przerwań $\overline{INT}$. Poniżej znajduje się potencjometr regulujący kontrast wraz z jego zworkami konfiguracyjnymi.

Warstwa opisowa (ang. *Silkscreen*) została bardzo szczegółowo opisana, co pozwoli na uniknięcie wielu błędów podczas konfiguracji układu. Zastosowano czytelne etykiety dla poszczególnych bloków "PULL UP", "DATA", "CON" (ang. *Control*), co ułatwia diagnostykę i uruchamianie układu bez konieczności ciągłego spoglądania na schemat ideowy. Każda ze zworki również jest oznaczona, a zworki mają dodatkowo opisane stany logiczne "1" i "0", co minimalizuje błędy złej adresacji układów. Piny zasilania oraz magistrali komunikacyjnej (SDA, SCL) są jednoznacznie zidentyfikowane.

Na spodniej stronie płytki, przedstawionej na rysunku 4.4 zostały poprowadzone ścieżki sygnałowe. Zgodnie z dobrymi praktykami projektowymi, odstęp izolacyjny ang. *clearance* między ścieżkami oraz między padami a ścieżkami są duże. Zabieg ten z jednej strony znacząco ułatwia ręczny montaż elementów w technologii przewlekanej (THT) i ogranicza ryzyko zwarc (tzw. mostków lutowniczych). Po drugie, zwiększony dystans między równoległymi liniami sygnałowymi zmniejsza pojemności pasożytnicze, co korzystnie wpływa na integralność przesyłanych sygnałów, zwłaszcza w obszarze 8-bitowej magistrali danych [7, 19].

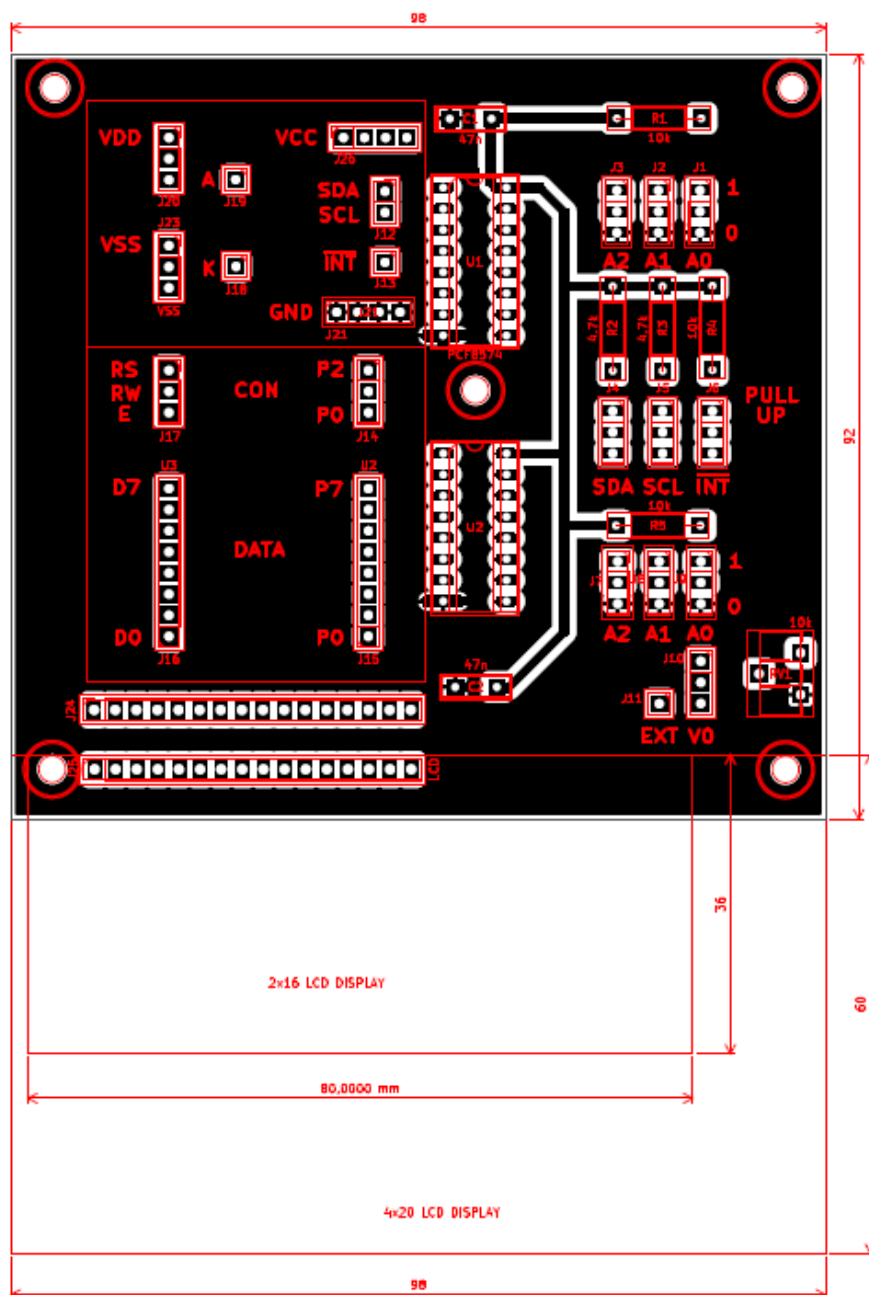


RYSUNEK 4.3: Szkic płytki z dwoma układami PCF8574



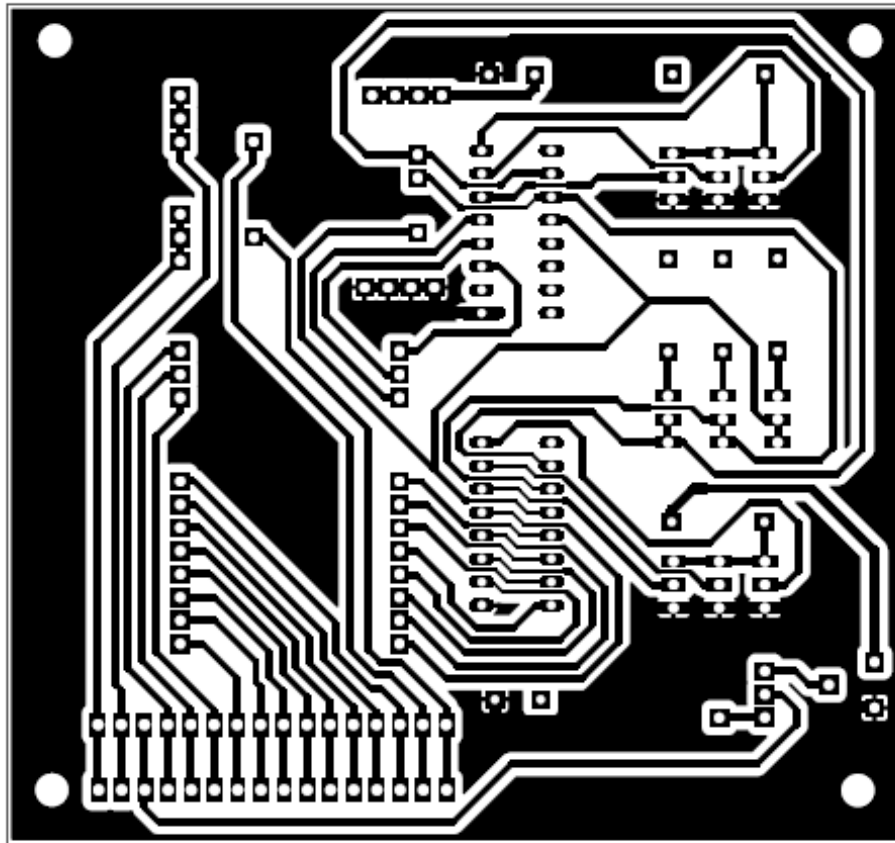
RYSUNEK 4.4: Szkic spodniej strony płytki z dwoma układami PCF8574

4.5 Płytką PCB z frezowaniem ścieżek



RYSUNEK 4.5: Szkic płytki z dwoma układami PCF8574 wersja pod frezowanie

W tej sekcji przedstawiono wersję płytki, która może zostać wykonana lokalnie na uczelni. Jest to wersja zoptymalizowana pod kątem wytwarzania metodą obróbki ubytkowej - *frezowania izolacyjnego CNC*. Pod względem elektrycznym i topologicznym układ zachowuje funkcjonalność pierwowzoru. Strona wierzchnia płytki (rys. 4.5) to strona montażowa. Ponieważ jest to płytką wykonywaną lokalnie i nie zawiera pokrycia izolacyjnego (ang. *solder mask*) oraz nadruków (ang. *silk-screen*), napisy zostaną umieszczone poprzez naklejenie odpowiednich etykiet. Nie zdecydowano się na frezowanie napisów w miedzi. To stwarzałoby ryzyko zwarcie (odprysk miedzi), ale też obniżało estetykę a nawet czytelność napisów. Aby zapobiec utlenianiu miedzi, zalecane jest polakierowanie powierzchni płytki.



RYSUNEK 4.6: Szkic spodniej strony płytki z dwoma układami PCF8574 wersja pod frezowanie

Dolna warstwa płytki (rys. 4.6) zawiera ścieżki oraz pola lutownicze. Został zastosowany większy odstęp między polem masy a, padami montażowymi, co ułatwi montaż i lutowanie. Cyna nie będzie się rozlewać i powodować zwarcia do masy.

Rozmieszczenie elementów jest podyktowane łatwością prowadzenia ścieżek przy użyciu frezarki, gdzie przelotki są problematyczne do wykonania w warunkach domowych. Pady mas w układach PCF8574 (U1 oraz U2) zostały wydłużone, gdyż pin ten pełni również rolę przelotki, więc rozgrzanie go po obu stronach płytki zapewni dobry rozplływ cyny, a w konsekwencji stabilne połączenie elektryczne.

Zestaw złącz, układów, zworek, kondensatorów oraz rezystorów nie został zmieniony. Nie zmienił się również ich rozkład.

Rozdział 5

Projekt modułu LCD z 16-bitowym ekspanderem MCP23017

5.1 Założenia techniczne

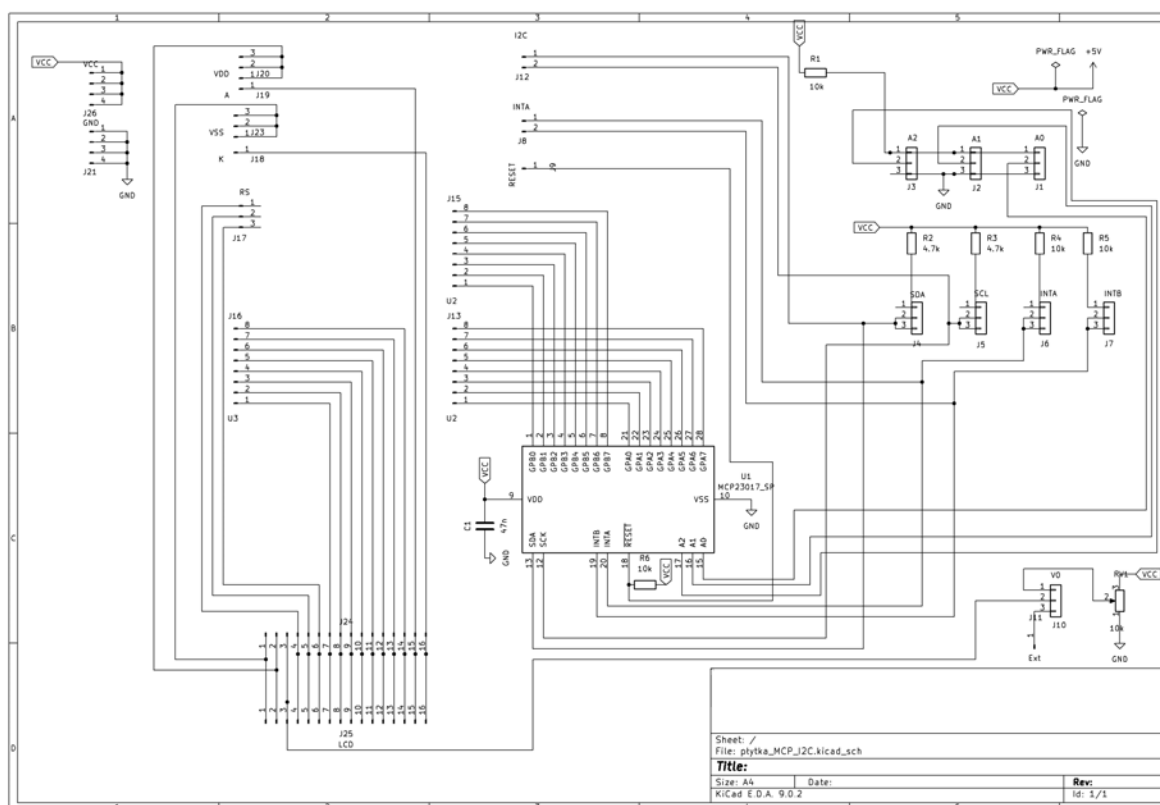
Projektowany moduł wspierający uruchomienie wyświetlaczy LCD powinien spełniać następujące warunki:

- obsługa wyświetlaczy LCD 2x16/4x20,
- wykorzystanie 16-bitowych ekspanderów I/O,
- możliwość korzystania z interfejsu równoległego (8-bit/4-bit transmisja danych),
- możliwość korzystania z interfejsu szeregowego I²C,
- możliwość konfigurowania interfejsu na płycie modułu,
- zapewnienie intuicyjnej konfiguracji połączeń,
- zapewnienie szeregowej transmisji danych i sygnałów sterujących,
- zasilanie zewnętrzne +5 V z systemu mikroprocesorowego,
- współpraca modułu z mikroprocesorami różnych rodzin,
- możliwość współpracy z modułami platformy Arduino,
- możliwość współpracy z modułami platformy ESP32,
- możliwość wykorzystania 5 linii jako I/O ogólnego przeznaczenia,
- możliwość dostosowania kontrastu wyświetlacza LCD,
- łatwe serwisowanie modułu.

5.2 Koncepcja modułu

W trakcie tworzenia modułu wykonano kilka wersji projektowych. Aby osiągnąć mniejszy rozmiar płytki drukowanej konieczna była zmiana położenia większości elementów, które znajdowały się w module z układami PCF8574. W finalnej wersji obrócono ekspander o 90 stopni tzn. równolegle do ekranu. Pozwoliło to na przemieszczenie układu scalonego w część centralną PCB, co zwolniło dużo miejsca po bokach płytki. Głównym założeniem przy tym projekcie było maksymalne zbliżenie elementów do siebie, o ile była taka możliwość. Znaczącą zmianą względem modułu z rozdziału nr. 4 jest przeniesienie sekcji zasilania (VSS, VCC, VDD i GND). Przekształcono również umiejscowienie złączy adresowych, sterowniczych i komunikacyjnych. Oba porty wejść/wyjść zostały rozdzielone na płytce w taki sposób, aby jedno złącze było intuicyjnie naprzeciwko złącz wyświetlacza, natomiast drugie wstawiono na lewym brzegu płytki. Rezystory podciągające przemieszczono na prawą stronę PCB. Zmianom położenia nie podlegał potencjometr do ustawiania kontrastu wyświetlacza. Finalne rozmiary płytki drukowanej to 80x70 mm, więc względem płytki z dwoma PCF8574 skrócono szerokość o 18 mm i wysokość o 15 mm.

5.3 Opis modułu



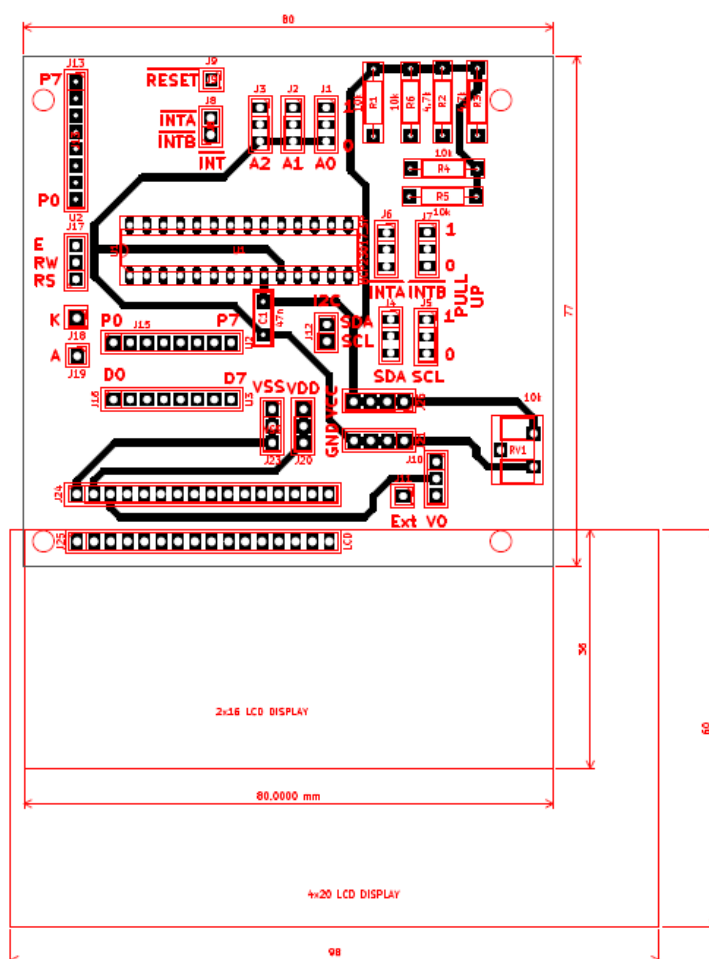
RYSUNEK 5.1: Schemat ideowy modułu z MCP23017

Projekt modułu oparty na układzie PCF8574 pozwolił na zdobycie potrzebnego doświadczenia i oswojenie się z zadaniami, układem na płytce drukowanej i ograniczeniami w procesie projektowania. Zdobyte doświadczenie stworzyło idealny fundament pod projekt modułu z wykorzystaniem

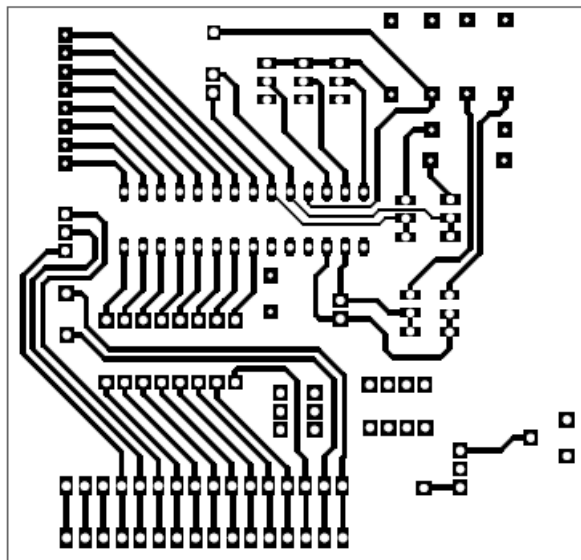
rozbudowanego ekspandera MCP23017. Płytki drukowane miała przejść ewolucję, w której zawrze się implementację pod nowy układ scalony. Dzięki omówionym wcześniej przewagom MCP23017 nad PCF8574 możliwe było ograniczenie liczby układów z 2 na 1. Projekt w swoich założeniach miał obsługiwać dwa rodzaje wyświetlaczy LCD o tych samych parametrach, co w koncepcji z ekspanderem PCF8574. Układ MCP23017 korzysta z interfejsu I²C. Jednym z wyzwań przy tworzeniu płytki drukowanej była minimalizacja jej rozmiarów. Moduł został skonstruowany z myślą o intuicyjnym zastosowaniu przez użytkownika, dlatego zostały naniesione czytelne opisy złączy. Układ zaprojektowano do działania z różnymi typami mikrokontrolerów. Wyprodukowano dwie wersje modułu, dedykowane na zamówienie (z soldermaską) i do wyfrezowania na uczelni.

Na rysunku 5.1 znajduje się schemat ideowy omawianego modułu. Zawiera on wszystkie potrzebne połączenia do komunikacji z wyświetlaczem LCD, którego złącze jest na dole schematu. Na środku znajduje się układ scalony MCP23017 z wszystkimi niezbędnymi peryferiami. Złącza J15 i J13 są połączone z portami wejść/wyjść ekspandera. Zaobserwować można złącza do podciągania linii interfejsu I²C, pola adresowe czy wyjścia przerwań.

5.4 Płytki PCB z soldermaską



RYСУNEK 5.2: Szkic płytki drukowanej opartej na układzie MCP23017



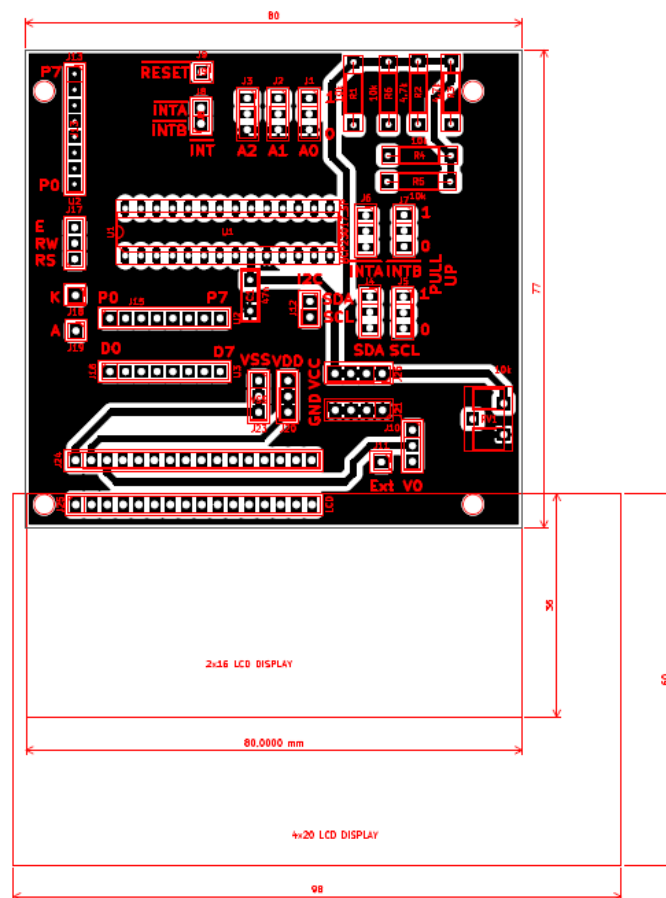
RYSUNEK 5.3: Szkic spodniej strony płytki opartej na układzie MCP23017

Rysunek przedstawia płytkę PCB przeznaczoną do produkcji dla podmiotów zewnętrznych. Układ zawiera warstwę ochronną tzn. soldermaskę, która pozwala na bezpieczne korzystanie z przedstawionego rozwiązania. Na rysunkach 5.2 i 5.3 przedstawiono wygląd warstwy wierzchniej i spodniej płytki drukowanej [7, 19].

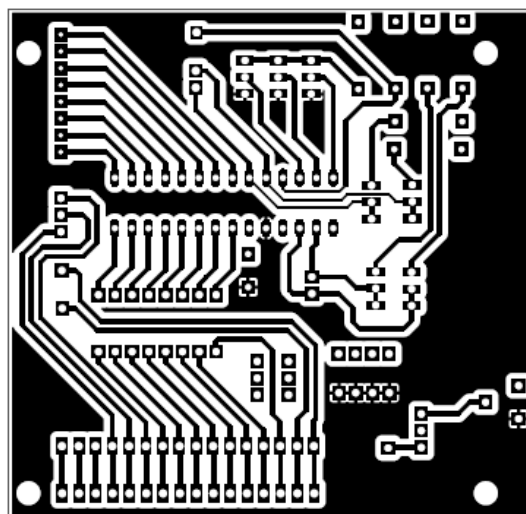
5.5 Płytką PCB z frezowaniem ścieżek

Perspektywa samodzielnego wytwarzania obwodów drukowanych (PCB) przy użyciu zaplecza sprzętowego instytutu wymusiła konieczność gruntownej rewizji przyjętych wcześniej założeń projektowych. Technologia frezowania izolacyjnego (CNC) znacząco różni się od klasycznych metod trawienia chemicznego, narzucając rygorystyczne obostrzenia w zakresie topologii obwodu. Fizyczne wymiary narzędzia skrawającego (frezu typu grawerskiego) zdefiniowały minimalny, krytyczny odstęp izolacyjny (ang. *clearance*) pomiędzy sąsiadującymi polami lutowniczymi oraz ścieżkami. Zbyt gęste upakowanie elementów mogłoby uniemożliwić przejście frezu, skutkując powstawaniem zwarców.

Szczególną uwagę zwrócono na kąty prowadzenia połączeń – konsekwentnie stosowano załamania o kącie 45 stopni, co ułatwia płynną, ciągłą pracę wrzeciona maszyny i zapobiega niepotrzebnemu zużyciu narzędzi. Ponadto, w celu skrócenia czasu pracy frezarki, nieużywane przestrzenie na płycie wypełniono płaszczyzną miedzi, pozostawiając jedynie wąski kontur izolacyjny wokół właściwych ścieżek. Wdrożenie tych restrykcyjnych reguł projektowych (DRC) zaowocowało stworzeniem finalnej wersji płytki, w pełni zoptymalizowanej pod kątem bezproblemowej produkcji maszynowej na terenie instytutu.



RYSUNEK 5.4: Szkic płytki drukowanej opartej na układzie MCP23017, wersja pod frezowanie



RYSUNEK 5.5: Szkic spodniej strony płytki opartej na układzie MCP23017, wersja pod frezowanie

Rozdział 6

Projekt modułu LCD z 16-bitowym ekspanderem MCP23S17

6.1 Założenia techniczne

Ograniczenia magistrali I²C, ale też bardzo podobna struktura układu MCP23S17, skłoniły do stworzenia modułu obsługującego magistralę SPI. Wspomniany układ ma te same rejestry co jego bliźniaczy odpowiednik MCP23017, co od programisty, w przypadku chęci osiągnięcia większej przepustowości, wymaga jedynie zmiany protokołu komunikacyjnego. Jest to modyfikacja poprzedniego modułu. Różni się zastosowanym układem U1, który współpracuje z magistralą SPI oraz wyprowadzeniem pinów magistrali. W przypadku I²C, były to dwa piny, a w przypadku SPI to są aż cztery piny. Reszta parametrów pozostała bez zmian.

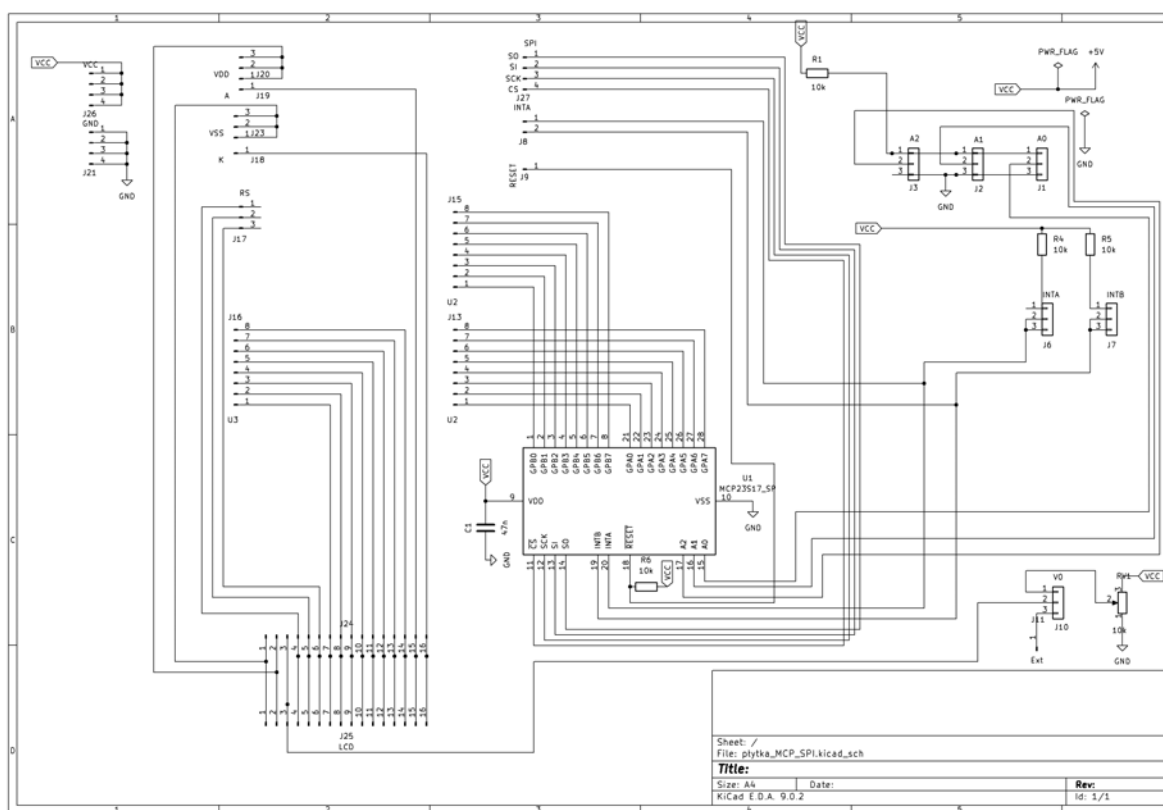
W związku z tym założenia projektowe można ująć podobnie jak dla modułu MCP23017:

- obsługa wyświetlaczy LCD 2x16/4x20,
- wykorzystanie 16-bitowych ekspanderów I/O,
- możliwość korzystania z interfejsu równoległego (8-bit/4-bit transmisja danych),
- możliwość korzystania z interfejsu szeregowego SPI,
- możliwość konfigurowania interfejsu na płytce modułu,
- zapewnienie intuicyjnej konfiguracji połączeń,
- zapewnienie szeregowej transmisji danych i sygnałów sterujących,
- zasilanie zewnętrzne +5 V z systemu mikroprocesorowego,
- współpraca modułu z mikroprocesorami różnych rodzin,
- możliwość współpracy z modułami platformy Arduino,
- możliwość współpracy z modułami platformy ESP32,
- możliwość wykorzystania 5 linii jako I/O ogólnego przeznaczenia,
- możliwość dostosowania kontrastu wyświetlacza LCD,
- łatwe serwisowanie modułu.

6.2 Koncepcja modułu

Zmiana układu scalonego nie spowodowała dużych zmian w kontekście rozmieszczenia złączy wewnątrz płytki drukowanej. Jediną różnicą względem poprzedniego modułu było dostosowanie się pod komunikację magistralą SPI. Skutkowało to usunięciem rezystorów podciągających do linii SDA i SCL. Wraz z rezystorami zniknęły złącza do podciągania szyn interfejsu I²C. Wolne miejsce pozwoliło na dodanie portu 4-pinowego do obsługi nowej magistrali.

6.3 Opis modułu



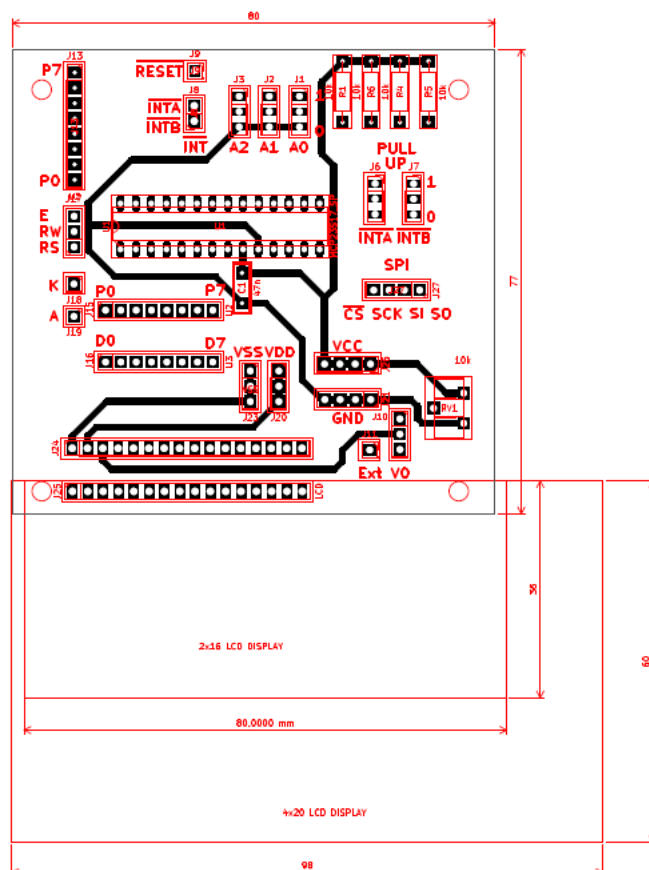
RYSUNEK 6.1: Schemat ideowy modułu z MCP23S17

Przedstawiony moduł sprzętowy realizuje komunikację z alfanumerycznym wyświetlaczem LCD za pomocą synchronicznej magistrali szeregowej SPI. W tym przypadku również jest dostępna pełna komunikacja równoległa. Jeden 16-bitowy układ obsługuje w pełni wyświetlacz. Upraszcza to znacząco prowadzenie ścieżek na płytce drukowanej i zmniejsza koszty produkcji. Komunikacja z mikrokontrolerem odbywa się poprzez złącze J27 (SPI). Zastosowanie magistrali SPI (linie MISO/SO, MOSI/SI, SCK, CS) pozwala na osiągnięcie znacznie większych prędkości transferu danych (do 10 MHz) w porównaniu do standardowego I²C. Zazwyczaj urządzenia SPI identyfikowane są za pomocą linii wyboru ang. *Chip Select* (CS) dla każdego układu na magistrali. Układ MCP23S17 wspiera jednak sprzętowe adresowanie w ramach tej samej ramki SPI. Dzięki temu w przypadku rozbudowy systemu, kilka takich ekspanderów może współdzielić tę samą fizyczną linię CS, co stanowi ogromną oszczędność pinów w mikrokontrolerze.

16-bitowa przestrzeń wejść/wyjść układu MCP23S17 jest podzielona na dwa niezależne 8-bitowe banki (PORT A i PORT B). Na schemacie (rys. 6.1) widać wyraźny, funkcjonalny podział tych portów. Wszystkie piny od GPB0 do GPB7 zostały doprowadzone przez złącze J15 do pinów D0-D7 na głównym złączu wyświetlacza J25. Szyna sterująca¹, wykorzystuje pierwsze trzy piny portu A (GPA0-GPA2). Pozostałe piny pozostają niewykorzystane, co pozostawia rezerwę na ewentualną rozbudowę.

Projekt zachowuje ciągłość technologiczną z poprzednimi modułami, powielając sprawdzone bloki funkcjonalne. Wykorzystywany jest reset sprzętowy - w przeciwieństwie do układów PCF, MCP23S17 posiada sprzętowe wejście resetujące układ. Zostało ono podciągnięte do zasilania poprzez rezystor R6 o wartości $10\text{ k}\Omega$ i dalej wyprowadzone do złącza J9. W tym przypadku również zadbano o wyprowadzenie sygnałów przerwań dla INTA oraz INTB, do złącza J8, a za pomocą zworek J6 i J7 można włączyć PULL-UP poprzez rezystory R4 oraz R5, każdy o wartości $10\text{ k}\Omega$. Dalej jest wykorzystywany kondensator filtrujący C1 o wartości 47 nF dla zasilania ekspandera [27]. Potencjometr RV1 do regulacji napięcia polaryzacji matrycy oraz złącza konfiguracyjne do zasilania układów oraz podświetlenia (anoda i katoda) pozostały bez zmian.

6.4 Płytkę PCB z soldermaską



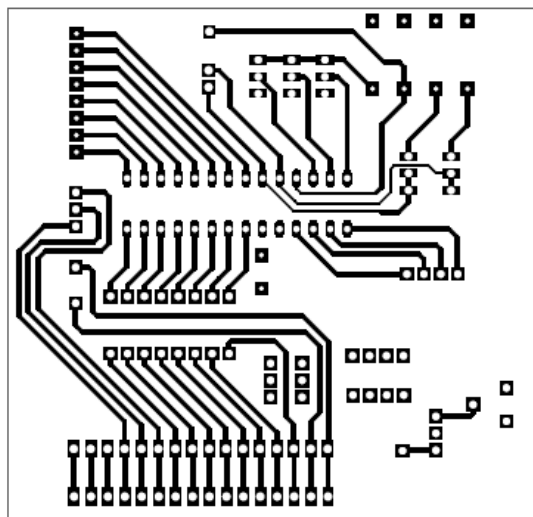
RYSunek 6.2: Szkic płytki drukowanej opartej na układzie MCP23S17

¹linie RS, RW, EN

Na rysunku 6.2 przedstawiono górną warstwę płytki drukowanej, opracowaną dla potrzeb produkcji przez podmiot zewnętrzny. Moduł posiada czytelne opisy złączy, tak aby używanie układu było intuicyjne. Płytkę drukowaną charakteryzuje się bardziej zwartą konstrukcją niż poprzednie wersje, a jej wymiary wynoszą 80 x 77 mm. Projekt został zrealizowany jako obwód dwuwarstwowy.

Na warstwie górnej znajdują się korpusy elementów THT oraz stosowne opisy ułatwiające konfigurację oraz zrozumienie połączeń na płytce. Najważniejszym elementem w centralnej części płytki jest układ U1 - 16-bitowy ekspander. Przy jego nóżce zasilania (VCC) umieszczony został kondensator filtrujący C1 gwarantujący stabilność zasilania [27]. Po prawej stronie występują zworki konfigurujące PULL-UP dla przerwań INTA oraz INTB, a tuż pod nimi złącze do komunikacji SPI. W prawej dolnej sekcji znajduje się potencjometr RV1 oraz jego zworki konfigurujące. W górnej części płytki znajdują się zworki konfigurujące adres urządzenia oraz wyprowadzenia INTA, INTB i RESET. Po prawej stronie zostały wyprowadzone pinu portu A, tuż pod nimi złącze sygnałów sterujących oraz zasilanie podświetlenia (anoda oraz katoda).

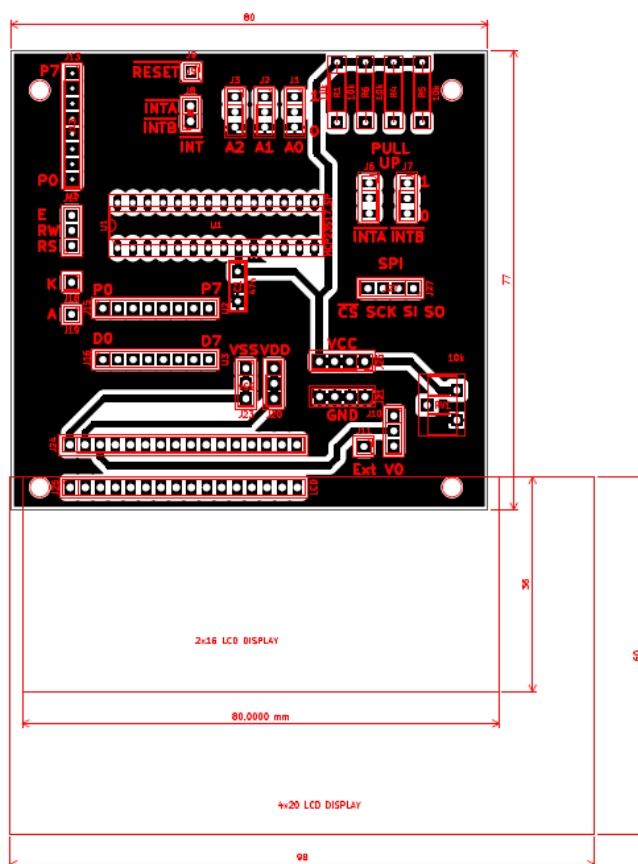
Warstwa opisowa (ang. *Silkscreen*) została szczegółowo opracowana. Złącze J27 jednoznacznie opisuje każde połączenie dla magistrali SPI. Przerwania oraz reset zostały wyraźnie zaznaczone, a zworki adresowe dla układu MCP23S17 są wyraźnie oznaczone stanami logicznymi 0 i 1, tak jak w poprzednich projektach. W tej płytce również występuje 16-pinowe złącze J25 dla ekranu LCD.



RYSUNEK 6.3: Szkic spodniej strony płytki opartej na układzie MCP23S17

Warstwa dolna płytki, przedstawiona na rysunku 6.3 zawiera w głównej mierze ścieżki sygnałowe oraz pady, do których lutowane są elementy. Ze względu na zastosowanie montażu przewlekane THT to właśnie na tej stronie realizowane są fizyczne połączenia poprzez lutowanie. Należy zaznaczyć, że zadbano o odpowiednie grubości ścieżek oraz odstęp między nimi, aby ograniczyć zjawisko przesłuchów, co przy prędkościach transmisji występujących w protokole SPI jest ważne. To rozwiązanie zapewnia wysoką integralność sygnałów cyfrowych wymienianych z układem MCP23S17 [19, 7].

6.5 Płytki PCB z frezowaniem ścieżek



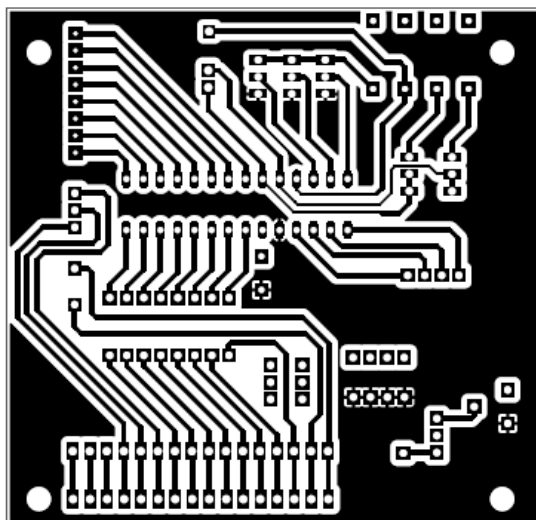
RYСУNEK 6.4: Szkic płytki drukowanej opartej na układzie MCP23S17 wersja pod frez

Wersja modułu zoptymalizowana pod kątem szybkiego wytworzenia z wykorzystaniem obróbki ubytkowej, tj. frezowania izolacyjnego CNC, zaprezentowana na rysunku 6.4 strona top, w porównaniu do poprzedniej wersji charakteryzuje się wylaną płaszczyzną masy na całej powierzchni płytki. Wybór tej technologii poniósł za sobą jeszcze kilka innych konsekwencji, podobnych jak w przypadku poprzedniej płytki. Pierwszą są napisy, w tym przypadku również zdecydowano o naklejanych napisach. Ze względu, na to, że ta płytki jest jeszcze mniejsza to miejsca na wygrawerowane napisy w miedzi przez maszynę CNC, niosłyby za sobą jeszcze większe ryzyko na powstanie zwarć, lub zerwanie połączeń. Gdyby zdecydowano się na małe napisy w miedzi byłyby one zdecydowanie mało czytelne, dlatego wybór naklejanych napisów wydaje się ścieżką optymalną.

Układ komunikuje się za pomocą magistrali SPI, która w przeciwieństwie do powolnego I²C jest interfejsem znacznie szybszym, a dodatkowo synchronicznym (częstotliwość SCK może wynosić 10 MHz). Przy takich prędkościach zbocza sygnałów są bardzo strome, co generuje zakłócenia. Płaszczyzna masy drastycznie zmniejsza impedancję pętli powrotnej i działa jak ekran elektromagnetyczny. Dodatkową korzyścią z wykorzystaniem płaszczyzny masy jest zwiększona odporność na zakłócenia zewnętrzne oraz mniejszą emisję zakłóceń elektromagnetycznych (EMI).

Białe pola widoczne na rysunku 6.4 to przestrzeń bez miedzi i to właśnie w tych miejscach ostrze maszyny numerycznej CNC będzie się poruszało, usuwając miedź i tworząc żądane połączenia elektryczne. Pozostałe pole miedzi minimalizuje czas pracy frezarki oraz znacząco wydłuża

żywołność narzędzi skrawających. Na warstwie górnej zdecydowano prowadzić ścieżki zasilania oraz jedną ścieżkę sygnałową od potencjometru, aż do pinu V0 złącza wyświetlacza.



RYSUNEK 6.5: Szkic spodniej strony płytki opartej na układzie MCP23S17 wersja pod frez

Widoczna na rysunku 6.5 warstwa dolna płytki zawiera ścieżki sygnałowe, mają one duży odstęp (ang. *clearance*), co ma zminimalizować ryzyko zwarcia między sygnałami. Na tej warstwie elementy będą lutowane do padów, większy odstęp pozwoli na łatwiejsze lutowanie elementów oraz zminimalizuje postania mostków lutowniczych między padami. Linie magistrali szybkiej SPI (SCK, SI, SO) są stosunkowo krótkie i otoczone masą, co zapobiega przesłuchom na wolniejsze, asynchroniczne linie sterujące wyświetlaczem.

Rozdział 7

Projekt modułu LCD z 16-bitowymi ekspanderami MCP23x17

7.1 Założenia techniczne

Podczas fazy koncepcyjnej i projektowej dedykowanych modułów edukacyjnych opartych na 16-bitowych ekspanderach wejść/wyjść, wyłoniono innowacyjną koncepcję zintegrowania dwóch niezależnych interfejsów komunikacyjnych na wspólnym obwodzie drukowanym (PCB). Zdecydowano się na fizyczne połączenie w obrębie jednego laminatu układu MCP23017 (wykorzystującego magistralę I²C) oraz jego odpowiednika MCP23S17 (sterowanego za pomocą szybkiej magistrali SPI).

Dzięki takiej hybrydowej topologii, docelowy użytkownik modułu – na przykład student w laboratorium dydaktycznym – zyskuje możliwość płynnego i natychmiastowego przełączania się pomiędzy różnymi standardami transmisji szeregowej. Nie wymaga to przepinania samej matrycy LCD, modyfikacji głównych przewodów sygnałowych czy wymiany całych płytek testowych. Stawia to doskonałe, zintegrowane narzędzie do bezpośredniej analizy porównawczej obu protokołów, pozwalające na empiryczne badanie różnic w opóźnieniach programowych, przepływności danych oraz złożoności obsługi warstwy sprzętowej. Mimo iż zintegrowany układ charakteryzuje się relatywnie większą powierzchnią laminatu w porównaniu do pojedynczych rozwiązań, w ogólnym rozrachunku znacząco optymalizuje przestrzeń roboczą i ułatwia przechowywanie sprzętu w zapleczu laboratoryjnym, eliminując potrzebę dublowania tych samych złączy i peryferiów.

W konsekwencji założenia techniczne dla projektowanej płytki są następujące:

- obsługa wyświetlaczy LCD 2x16 i LCD 4x20 z typowym złączem 16-pin,
- wykorzystanie 16-bitowego ekspandera I/O z interfejsem I²C,
- wykorzystanie 16-bitowego ekspandera I/O z interfejsem SPI,
- możliwość korzystania z interfejsu równoległego (8-bit/4-bit transmisja danych),
- konfigurowanie interfejsów na płytce modułu,
- czytelność i intuicyjność interfejsów konfiguracyjnych,
- dołączanie/odłączanie rezystorów podciągających dla linii SDA, SCL,
- dołączanie/odłączanie rezystorów podciągających dla linii OD (ang. *Open Drain*),

- możliwość konfigurowania wolnych linii I/O,
- konfigurowanie 3 bitowych adresów dla układów ekspanderów,
- zasilanie zewnętrzne systemu mikroprocesorowego,
- współpraca modułu z mikroprocesorami różnych rodzin
- możliwość współpracy z modułami platformy Arduino,
- możliwość współpracy z modułami platformy ESP32.

7.2 Koncepcja modułu

Znaczącym wyzwaniem z zakresu inżynierii komputerowego wspomagania projektowania (CAD) okazał się optymalny rozkład przestrzenny (ang. component placement) głównych układów scalonych. Zastosowanie standardowych, przewlekanych obudów typu DIP-28 dla wieloportowych ekspanderów wymaga zagospodarowania dużej powierzchni laminatu. W początkowej fazie projektowania rozważano horyzontalną orientację układów względem złącza wyświetlacza. Wstępne analizy trasowania ścieżek wykazały jednak, że ułożenie poziome drastycznie komplikuje wyprowadzenie magistral sygnałowych i wymusza niepożądane, głębokie modyfikacje w wypracowanym już, ergonomicznym układzie złącz peryferyjnych. Zmiana ta zaburzyłaby również czytelność warstwy opisowej (ang. silkscreen), co w przypadku narzędzia dydaktycznego jest zjawiskiem wysoce niepożądanym. Ostatecznie zdecydowano się na orientację pionową, która umożliwiła zachowanie symetrii i logicznego podziału płyty na niezależną sekcję I²C (układ U1) oraz sekcję SPI (układ U2).

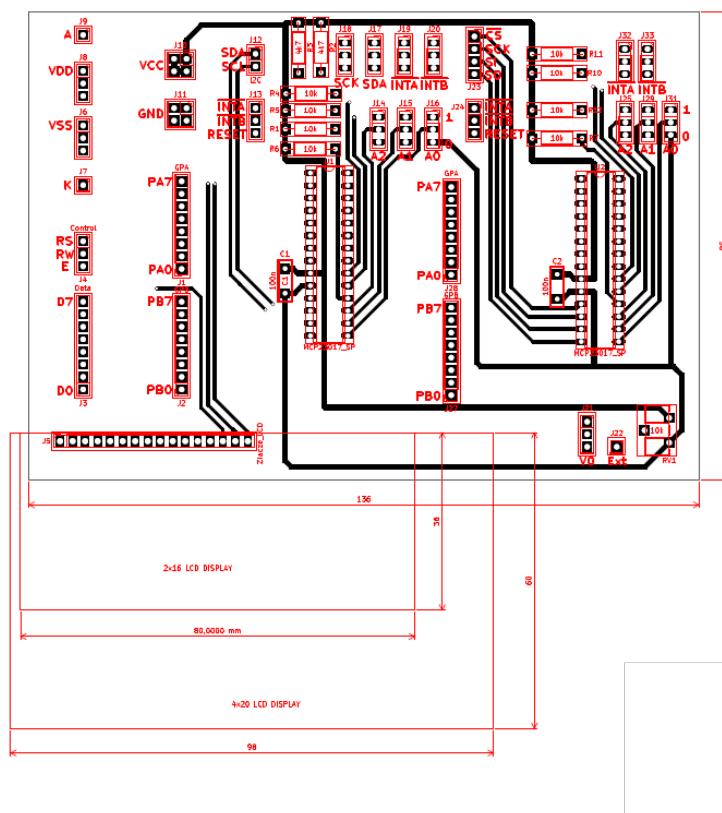
Z uwagi na konieczność implementacji znacznej liczby złącz konfiguracyjnych i elementów pasywnych, początkowa wersja modułu osiągnęła wymiary 136 mm szerokości oraz 95 mm wysokości. Na ten stosunkowo duży rozmiar bezpośrednio wpłynęła potrzeba umieszczenia obwodów kondycjonujących i sterujących. Jak wynika z analizy schematu ideowego, na płycie zaimplementowano między innymi:

Dedykowane rezystory podciągające (ang. *pull-up*) o wartości 4.7 k Ω i 10 k Ω dla linii sygnałowych magistrali I²C (SDA, SCL) oraz linii resetujących.

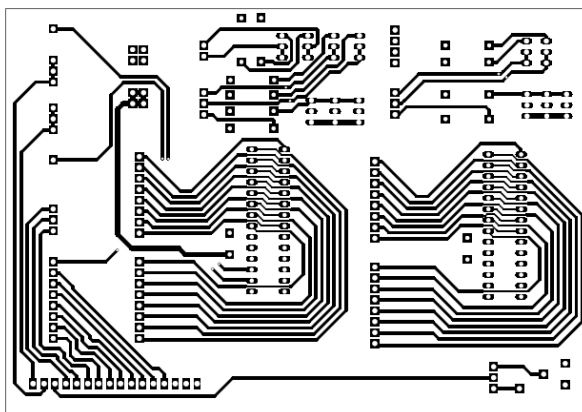
Rozbudowane bloki wyboru adresów sprzętowych. Zastosowanie zwerek dla linii A0, A1 i A2 pozwala na fizyczną modyfikację adresu każdego z ekspanderów, co sprzętowo umożliwia podłączenie do ośmiu takich samych układów na jednej magistrali bez ryzyka wystąpienia konfliktu adresowego.

Pełny dostęp do sprzętowych linii przerwań (INTA, INTB) dla obu układów, pozwalający na implementację zaawansowanych algorytmów asynchronicznych w programie mikrokontrolera.

Dodatkowo, moduł wyposażono we wspólny, 16-pinowy interfejs dla wyświetlacza alfanumerycznego, zintegrowany z precyzyjnym potencjometrem do regulacji kontrastu matrycy (RV1) oraz dedykowanymi wyprowadzeniami dla zasilania podświetlenia (piny A i K). Tak duża gęstość komponentów THT na obwodzie drukowanym wymagała rygorystycznego zaplanowania tras sygnałowych, aby uniknąć konieczności stosowania wielowarstwowych struktur PCB, co z powodzeniem zrealizowano zachowując klasyczną topologię dwuwarstwową.



RYСУNEK 7.1: Pierwsza wersja szkicu wierzchniej strony płytki z modułami MCP23x17



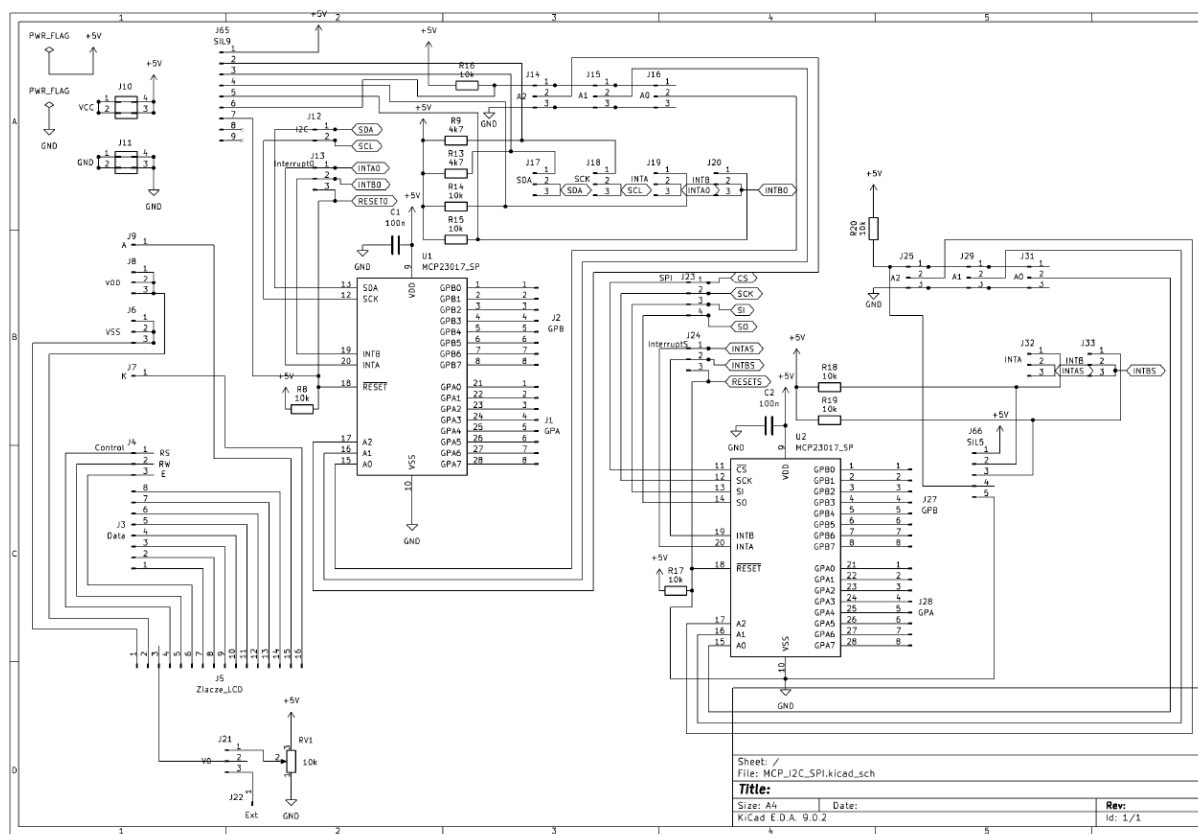
RYСУNEK 7.2: Pierwsza wersja szkicu spodniej strony płytki z modułami MCP23x17

Każda następna wersja płytki drukowanej konsekwentnie zmniejszała jej fizyczne wymiary. Ogromny postęp w pracy osiągnięto poprzez umiejętne poprowadzenie ścieżek z przelotkami, zmianą orientacji na poziomą złącz adresowych, złącz przerwań i podciągów interfejsu I²C. W projekcie wykorzystano również drabinki rezystorowe typu SIL9 oraz SIL5, dzięki czemu zmniejszono obszar przeznaczony dla rezystorów. Zadbano również o czytelność modułu dla użytkownika. Obszary danych magistral szeregowych zostały przegrodzone wyraźnymi prostokątami, wraz z odpowiednimi nazwami interfejsu. Finalna wersja płytki z dwoma ekspanderami miała wymiary 97,7 mm szerokości i 98,8 mm wysokości. Regularna i sumienna praca pozwoliły na stworzenie modułu o podobnych

rozmiarach do płytki z działu czwartego, natomiast upakowanie komponentów, jak i samo zaawansowanie modułu były bez porównania większe.

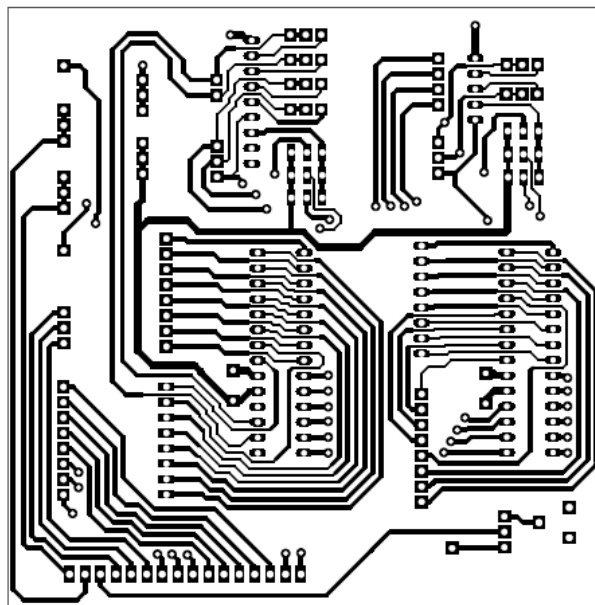
7.3 Opis modułu

Zaprojektowany układ pełni funkcję uniwersalnego modułu rozszerzeń portów wejścia/wyjścia. Głównym zadaniem modułu było zwiększenie liczby linii sterujących i obsługa standardowego wyświetlacza LCD. Układ wykorzystywał dwa najpopularniejsze interfejsy szeregowy tzn. SPI oraz I²C. Płytkę PCB podzielono na dwie sekcje, zależne od obsługiwanej magistrali. Dużą zaletą zaprojektowanego modułu była możliwość korzystania z dwóch interfejsów jednocześnie. Głównym zamysłem takiego podejścia była współpraca układu zarówno z wyświetlaczem, jak i różnego rodzaju czujnikami, które obsługiwały jedną z dostępnych magistral. Takie rozwiązanie pozwalało na wykorzystanie pojedynczego projektu do wielu liczby pomysłów np. pomiar temperatury lub transmisja bezprzewodowa.



RYSUNEK 7.3: Schemat ideowy modułu z MCP23x17

oraz ustawienie zworki V0 w odpowiedni tryb pracy. W dolnej części płytki znajduje się złącze wyświetlacza LCD.

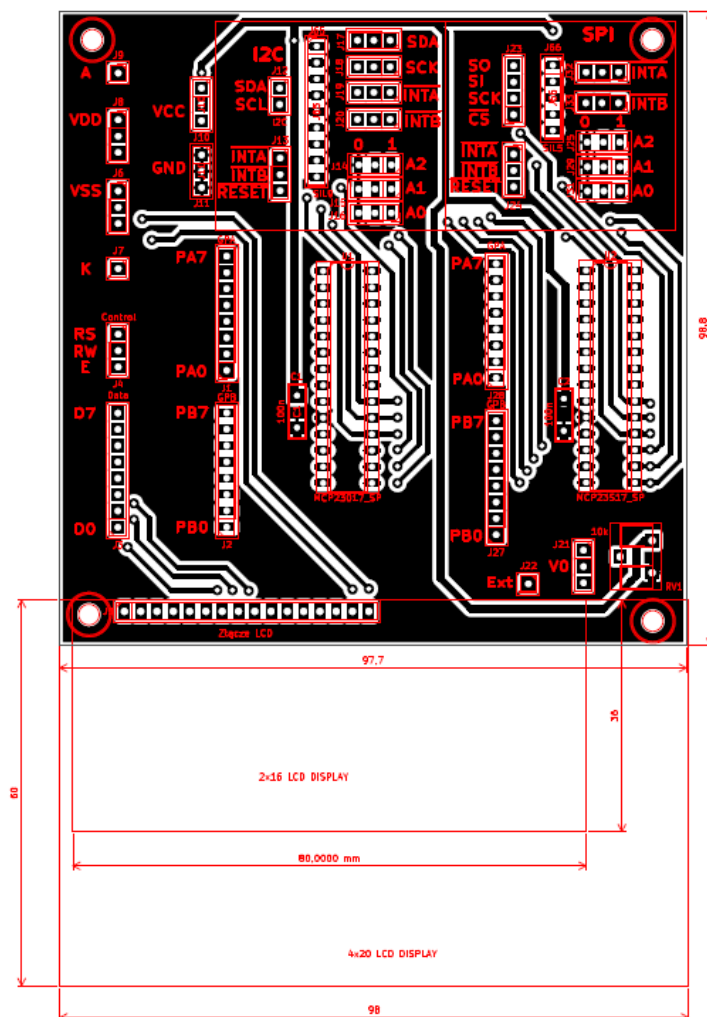


RYSUNEK 7.5: Spodnia strona płytki PCB z dwoma układami MCP23x17

Warstwa dolna obwodu drukowanego, przedstawiona na rysunku 7.5, stanowi główną płaszczyznę trasowania sygnałów oraz część dystrybucji zasilania. Zgodnie z dobrymi praktykami, zróżnicowano szerokości prowadzonych ścieżek w zależności od ich funkcji. Główne linie zasilające poprowadzono ścieżkami o szerokości 40 mils, co zapewnia odpowiednią obciążalność prądową i minimalizuje rezystancję pasożytniczą. Z kolei standardowe szyny sygnałowe wytrasowano z domyślną szerokością 25 mils. Aby zachować wymogi dotyczące odstępów izolacyjnych, w obszarach o dużym zagęszczeniu otworów zastosowano zwężenie ścieżek do 15 mils. Zabieg ten skutecznie zapobiega powstawaniu zwarców oraz mostków lutowniczych podczas montażu. Dostosowano również wymiary wyprowadzeń stosowanych komponentów przewlekanych (THT). Dla złącza matrycy LCD zastosowano wiertła o średnicy 0,8 mm, natomiast dla standardowych wyprowadzeń wybrano otwory o średnicy 0,9 mm. W przypadku padów układów scalonych oraz kondensatorów odsprężających średnicę wiercenia zredukowano do 0,7 mm, co sprzyja lepszemu formowaniu się spoiny lutowniczej. Zmiany warstw sygnałowych zrealizowano za pomocą przelotek o średnicy zewnętrznej padu równej 55 mils i średnicy otworu 0,6 mm.

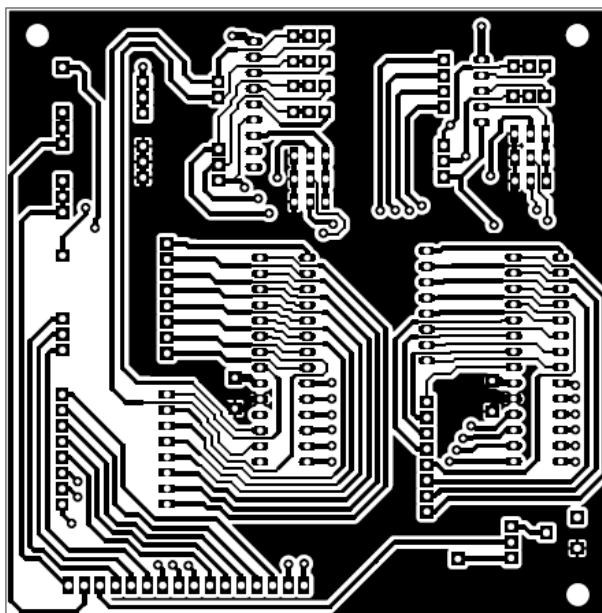
7.5 Płytki PCB z frezowaniem ścieżek

Alternatywna wersja docelowej płyty głównej zachowuje pełną funkcjonalność układu bazowego, jednak jej topologia została zmodyfikowana pod kątem wytwarzania metodą frezowania izolacyjnego (CNC) oraz poprawy parametrów elektrycznych.



RYСУNEK 7.6: Wierzchnia strona płytki PCB wypełniona polem masy z dwoma układami MCP23x17

Warstwa górna płytki PCB, widoczna na rysunku 7.6 pełni funkcję warstwy montażowej dla elementów THT. Poprowadzono na niej tylko te ścieżki sygnałowe, które wymagały bezkolizyjnego skrzyżowania z głównymi magistralami. Znajdują się tu również wszystkie złącza i zworki konfiguracyjne. W tej technologii warstwa opisowa (oznaczenia elementów), musi być wykonana ręcznie, jako naklejki z oznaczeniami, bądź wygrawerowana w technologii CNC, co obarczone jest ryzykiem małej czytelności oznaczeń.



RYSUNEK 7.7: Spodnia strona płytki PCB wypełniona polem masy z dwoma układami MCP23x17

Warstwa dolna płytki PCB, widoczna na rysunku 7.7 skupia główną mozaikę połączeń sygnałowych i zasilających. Trasowanie ścieżek zrealizowano z zachowaniem powiększonych odstępów izolacyjnych (ang. *clearance*), które dostosowano do fizycznej średnicy narzędzia skrawającego (frezu).

Najważniejszą różnicą względem wersji trawionej chemicznie jest płaszczyzna masy. Puste przestrzenie na warstwie dolnej wypełniono wylewką miedzi podłączoną do potencjału masy (GND). Skraca to znacząco czas pracy maszyny CNC i minimalizuje zużycie frezów, ponieważ usuwa się tylko wąski margines wokół ścieżek. Rozlana masa tworzy doskonały ekran elektromagnetyczny (EMC). Zapewnia to niskoimpedancyjną ścieżkę powrotu dla szybkich sygnałów cyfrowych, drastycznie redukując podatność układu na zakłócenia i przesłuchy.

Rozdział 8

Projekt systemu rozproszonego z łącznością radiową

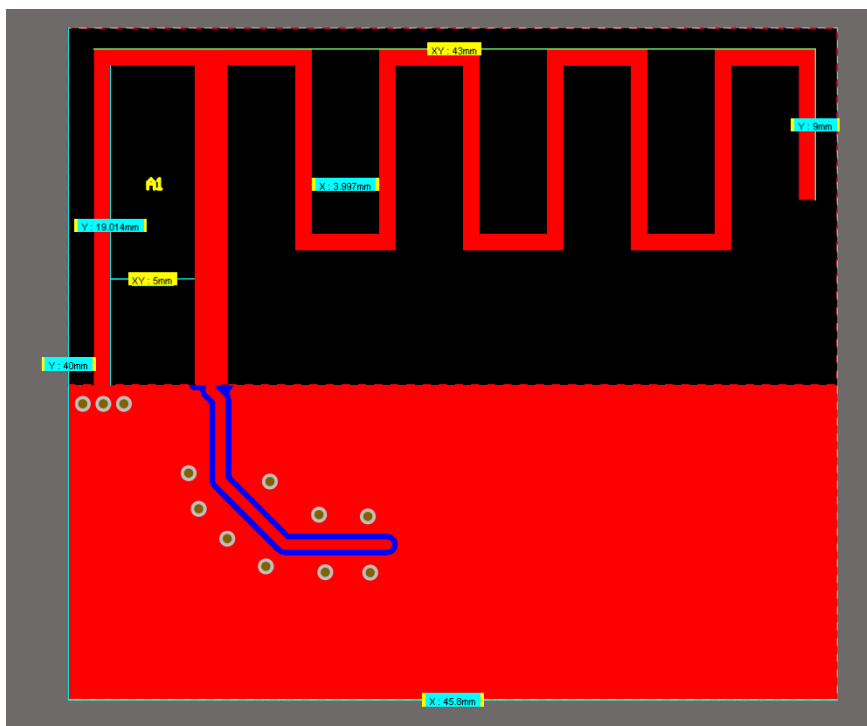
8.1 Założenia techniczne

W czasach intensywnego rozwoju technologii komunikacji radiowej np. 5G, WiFi i Bluetooth wiele projektów rezygnuje z transmisji przewodowej, na rzecz propagowania danych w otwartą przestrzeń. Wraz z popytem na łączność bezprzewodową powstał cały szereg programów symulacyjnych na potrzeby badań prototypowanych rozwiązań. Projekt modułów z ekspanderami portów I/O znakomicie wpasowywał się w swoich założeniach w rozszerzenie koncepcji o łączność RF. Mikrokontrolery takie jak ESP32 czy niektóre urządzenia z rodziny Arduino pozwalają na taką wymianę danych, zazwyczaj operując na częstotliwościach gigahercowych. Transmisja w takim paśmie charakteryzuje się dużą przepływnością, lecz znikomym zasięgiem. Założenia zakładają, że projekt powinna cechować stabilna komunikacja w promieniu kilkunastu, a nawet kilkudziesięciu metrów. Z tego względu postanowiono, że transmisja ma odbywać się na paśmie nielicencjonowanym o częstotliwości środkowej 868 MHz. Kolejnym kryterium był kształt samego promiennika wysyłającego sygnał w świat. Zainspirowanie prezencją i prostotą wykonania anteny w mikrokontrolerze ESP32 skłoniło do wyboru anteny typu IFA. Na potrzeby rozszerzenia o łączność bezprzewodową postanowiono zaprojektować własne moduły antenowe, zintegrowane z układem scalonym, który obsługiwał taki typ komunikacji. Wybór padł na urządzenie firmy Texas Instruments, transceiver CC1101. Układ zapewniał konwersję danych z SPI na standard RF, dzięki czemu mógł współpracować z ekspanderem MCP23S17, który znajdował się na jednym z zaprojektowanych wcześniej modułów. Poza tym CC1101 pracował na napięciu zasilania 3,3 V i zapewniał wiele funkcji związanych z technologią radiową, np. kilka rodzajów modulacji, pracę na różnych pasmach i wzmacnienie sygnału nawet o 12 dBm. Kolejną ważną cechą układu była możliwość zabezpieczenia transmisji przez bitowe słowa synchronizacyjne lub rozszerzenie pakietu o nadmiarowy kod cykliczny CRC.

8.2 Koncepcja modułu

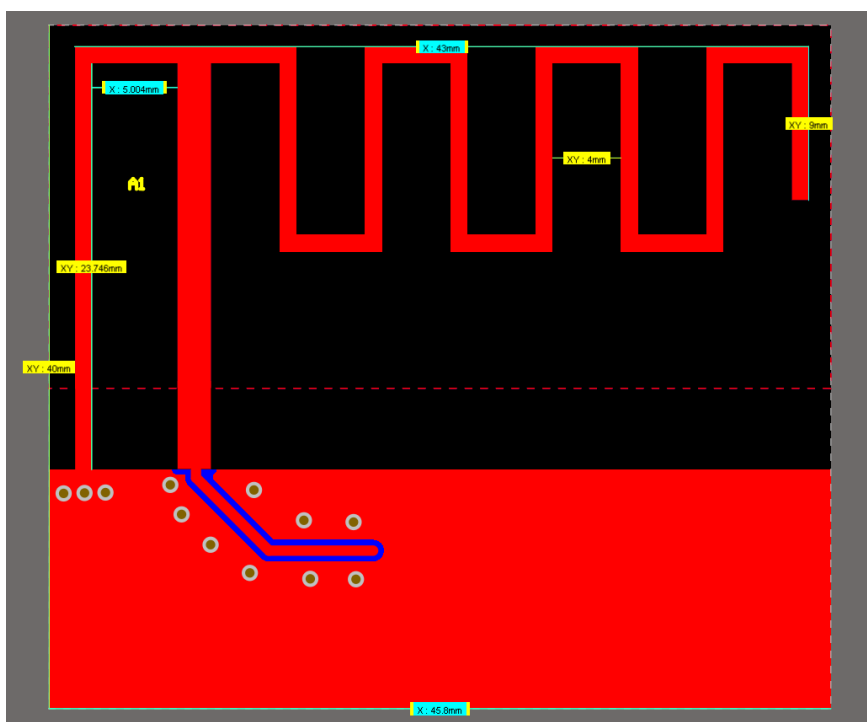
Na potrzeby symulacyjne przygotowano 3 warianty płytek PCB: podstawową, ze skróconą płaszczyzną masy oraz wydłużoną. Każda z płytek została przebadana, a wyniki pomiarów zostały opisane w rozdziale "Badania parametrów propagacyjnych". Płytką podstawową była inspiracją modułu instruktażowego z dokumentacji udostępnionej przez Texas Instruments. Na rysunku 8.1

przedstawiono układ na płytce. Wymagania programu CST Studio wymagały usunięcia z symulacji niektórych elementów z wyjątkiem anteny, ścieżki sygnałowej i płaszczyzny masy.

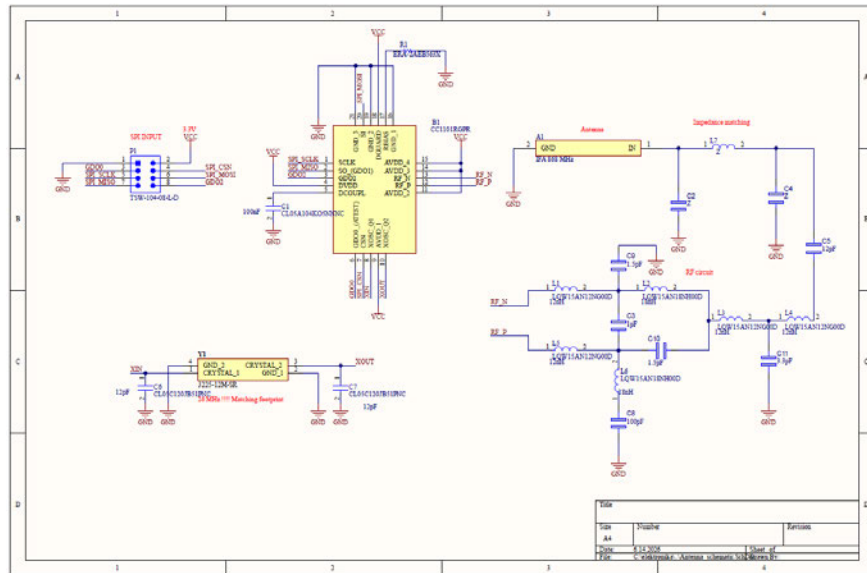


RYSUNEK 8.1: Podstawowa płytka PCB

W celu zbadania wpływu zmniejszenia płaszczyzny masy stworzono płytke widoczną na rysunku 8.2. Masa została skrócona o 5 mm w stosunku do wersji podstawowej. Skróceniu uległa ścieżka sygnałowa, natomiast wydłużyły się ramiona anteny.

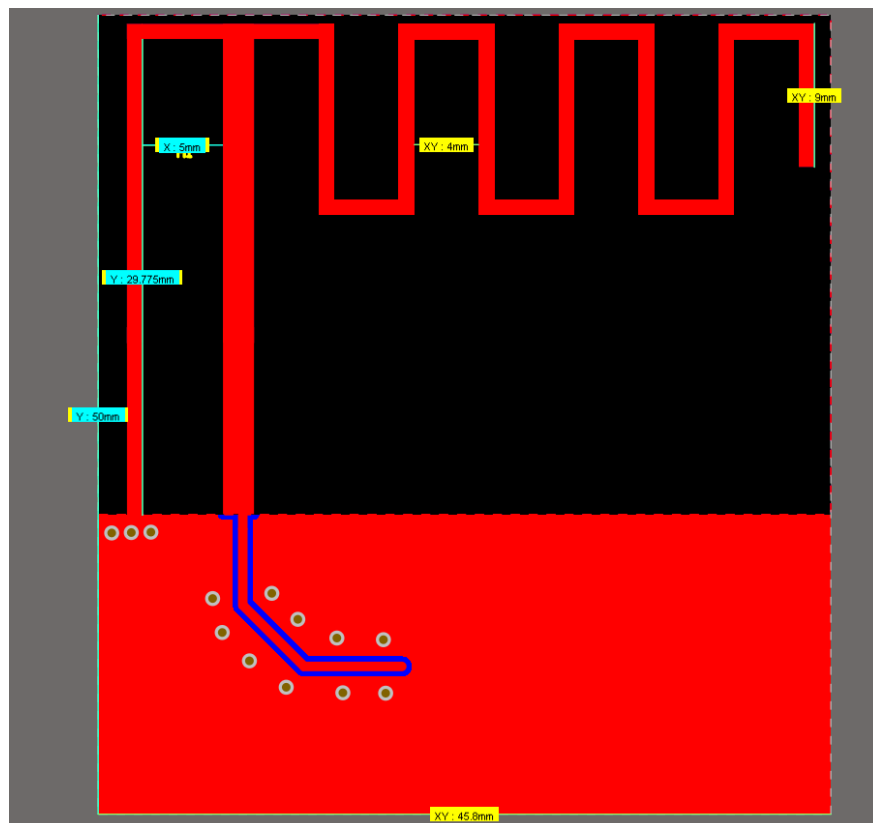


RYSUNEK 8.2: Moduł antenowy z skróconą płaszczyzną masy



RYSUNEK 8.4: Schemat ideowy modułu bezprzewodowego

Po sprawdzeniu płytki z skróconą masą, naturalnym krokiem było wydłużenie układu i płaszczyny odniesienia w celach badawczych. Na rysunku 8.3 przedstawiono wydłużony płytkę modułu antenowego.



RYSUNEK 8.3: Moduł antenowy z powiększonym rozmiarem Y

8.3 Opis modułu

Przedstawiony na rysunku 8.4 schemat ideowy ukazuje implementację układu nadawczo-odbiorczego CC1101 firmy Texas Instrument, do obsługi komunikacji w paśmie sub-gigahercowym. Moduł wraz z anteną, został zaprojektowany do pracy w paśmie ISM 868 MHz. Wyprowadzenie radiowe układu CC1101, oznaczone jako RF_N (pin 13) oraz RF_P (pin 12), są portami różnicowymi (zbalansowanymi). Oznacza to, że sygnał użyteczny jest różnicą potencjałów między tymi dwoma wyprowadzeniami. Z kolei zastosowana w projekcie antena drukowana (A1-IFA 868 MHz) jest elementem niesymetrycznym (niezbalansowanym), odniesionym do masy, o nominalnej impedancji wynoszącej $50\ \Omega$. Bezpośrednie podłączenie portów różnicowych z niesymetryczną anteną skutkowałoby drastycznym niedopasowaniem i stratą większości mocy wypromieniowywanej. Aby temu zapobiec, zastosowano dyskretny obwód symetryzatora, tzw. *balun*.

Na omawianym schemacie układ symetryzatora oraz wstępnego dopasowania impedancji został zrealizowany przy pomocy elementów dyskretnych LC. Topologia jest zgodna z notą katalogową producenta układu. Symetryzator pełni dwie fundamentalne funkcje. Transformacja zakresu trybu pracy zamienia sygnał różnicowy z pinów RF_P i RF_N na sygnał niesymetryczny (single-ended), który może być poprowadzony jedną ścieżką w kierunku anteny. Drugą właściwością symetryzatora jest dopasowanie impedancyjne. Wzmacniacz mocy (PA) oraz tor odbiorczy (LNA) wbudowane w CC1101 wymagają do optymalnej pracy specyficznej impedancji obciążenia. Zgodnie ze specyfikacją dla pasma 868 MHz wynosi ona optymalnie $86,5 + j43\ \Omega$. Zadaniem elementów składowych baluna jest przetransformowanie tej zespolonej wartości do standardowej rzeczywistej impedancji systemowej wynoszącej $50\ \Omega$.

Kondensator C8 pełni dodatkową funkcję separacji galwanicznej (blokowania składowej stałej), zapewniając zwarcie dla sygnałów wysokiej częstotliwości (w.cz.) do masy, jednocześnie zapobiegając przepływowi prądu stałego z wewnętrznych obwodów polaryzacji układu CC1101.

Za węzłem sumującym baluna sygnał trafia na układ filtra dolnoprzepustowego, zbudowanego z cewek L3, L4 oraz kondensatora C11. Jest to krytyczny element z punktu widzenia kompatybilności elektromagnetycznej. Modulacje cyfrowe generują niepożądane wyższe harmoniczne (zwłaszcza drugą i trzecią harmoniczną częstotliwości bazowej), które ten filtr ma za zadanie skutecznie wytłumić przed wypromieniowaniem ich w „eter”.

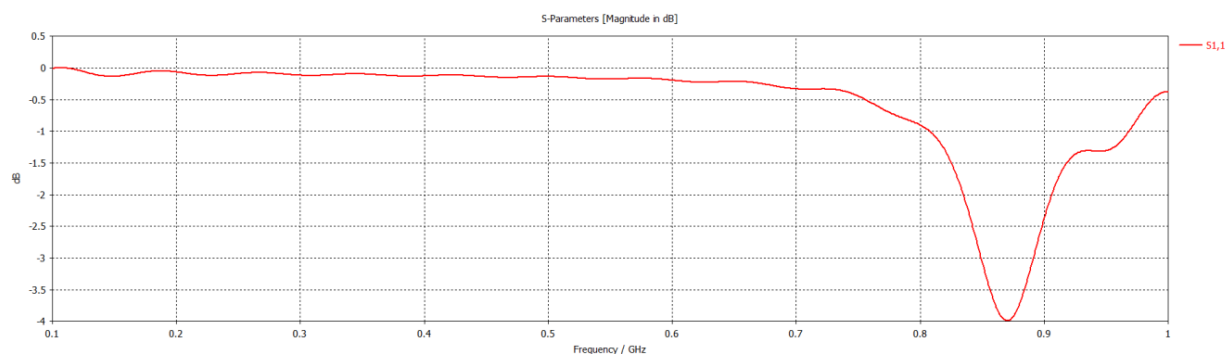
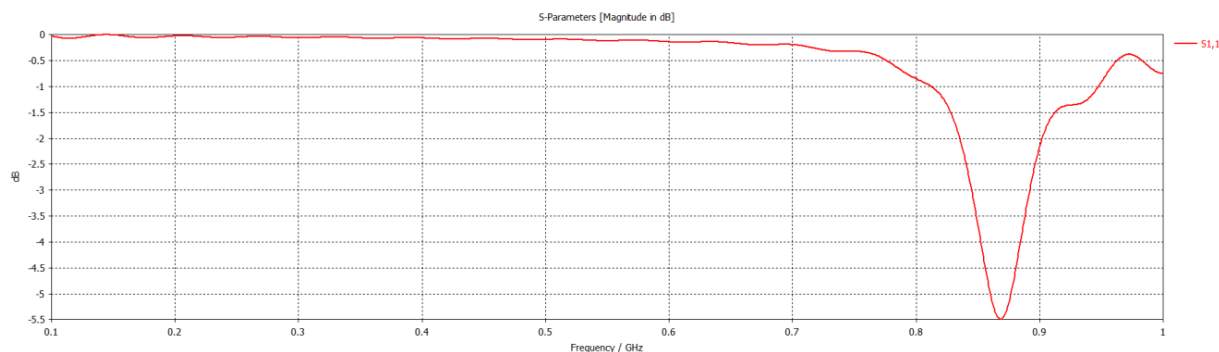
Ostatnim blokiem przed samą anteną jest obwód dopasowujący, składający się z elementu L7 oraz kondensatorów C2, C4, C5. Moduł ten pozwala na precyzyjne dostrojenie anteny w warunkach docelowej pracy urządzenia. Obecność elementów dielektrycznych przestraja rezonans anteny, co kompensuje się poprzez dobór wartości elementów w tej sekcji.

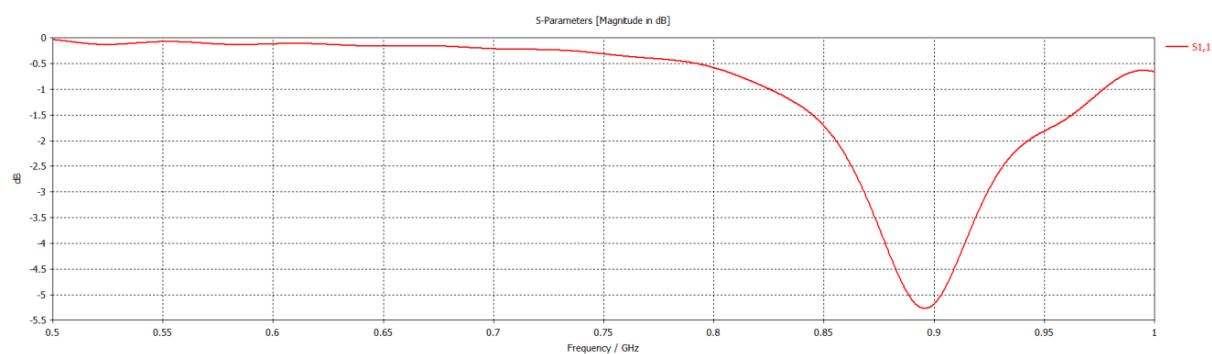
Do precyzyjnego taktowania wewnętrznego syntezer PLL wykorzystano zewnętrzny rezonator kwarcowy Y1 o częstotliwości 26 MHz, obciążony kondensatorami C6 i C7, co zapewnia odpowiednią stabilną częstotliwość wymaganą dla wąskopasmowej transmisji RF [15].

8.4 Badania parametrów propagacyjnych

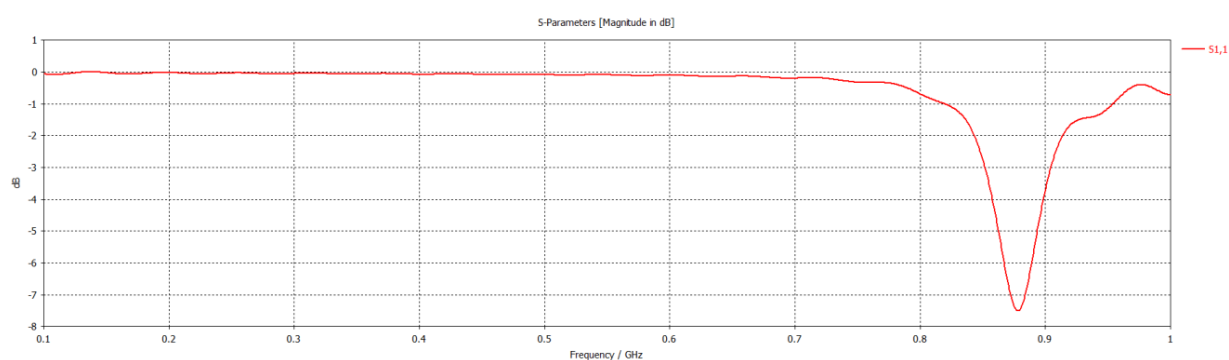
W celu wstępnej weryfikacji projektowanego modułu poddano go badaniom w programie symulacyjnym CST Studio Suite 2019. Oprogramowanie to profesjonalne środowisko wyspecjalizowane w testowaniu układów radiowych różnego rodzaju. Z pomocą programu badano 3 najważniejsze parametry sygnału tj. współczynnik S1.1, wykres Smitha oraz wykres promieniowania w polu dalekim. Dzięki tym parametrom można było oszacować, czy dany wariant miał predyspozycje

do dalszej realizacji. Współczynnik $S_{1.1}$ to parametr odpowiadający wprost za dopasowanie anteny. Była to skala odbicia napięcia od portu wejściowego, w której im mniejszy wynik w dB, tym lepiej. Za pomocą wykresu Smitha określano charakter impedancji, gdzie wartością oczekiwaną było równe 50Ω . Dzięki wykresowi propagacji w polu dalekim sprawdzono, czy antena była dookólna. CST Studio posiadał możliwość optymalizacji parametrów anteny w zależności od wyznaczonego celu. Proces optymalizacji polegał na zmianie parametrów fizycznych wybranych elementów całego modułu. Dla prostoty zmieniano tylko 2 wielkości: odległość pomiędzy nogą sygnałową i nogą odniesienia anteny oraz długością ostatniego meandra. Celem optymalizacji było osiągnięcie wartości współczynnika $S_{1.1}$ na poziomie mniejszym niż -10 dB. Każdy wariant płytki był poddany badaniu bez optymalizacji i po procesie optymalizacji.

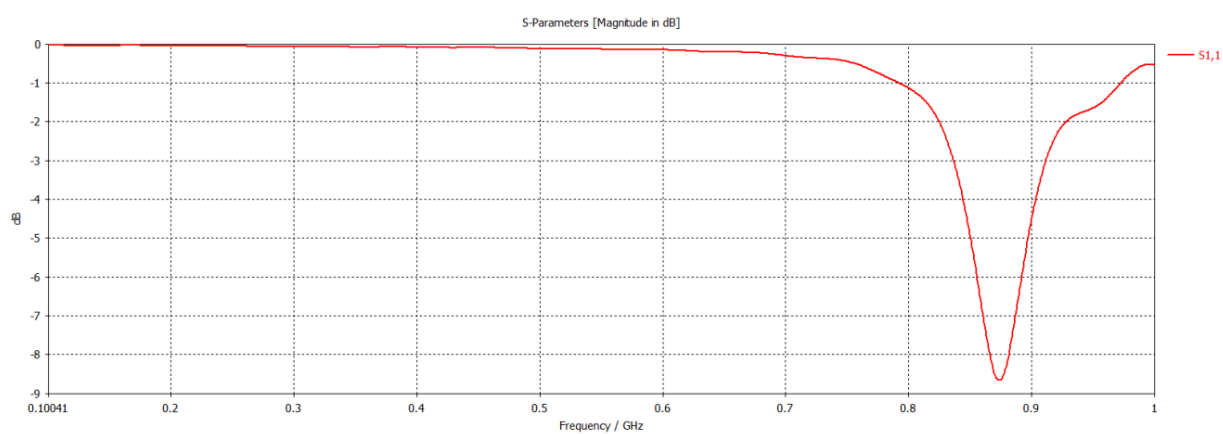
RYSUNEK 8.5: Współczynnik S_{11} - płytka bazowa bez optymalizacjiRYSUNEK 8.6: Współczynnik S_{11} - płytka bazowa zoptymalizowana



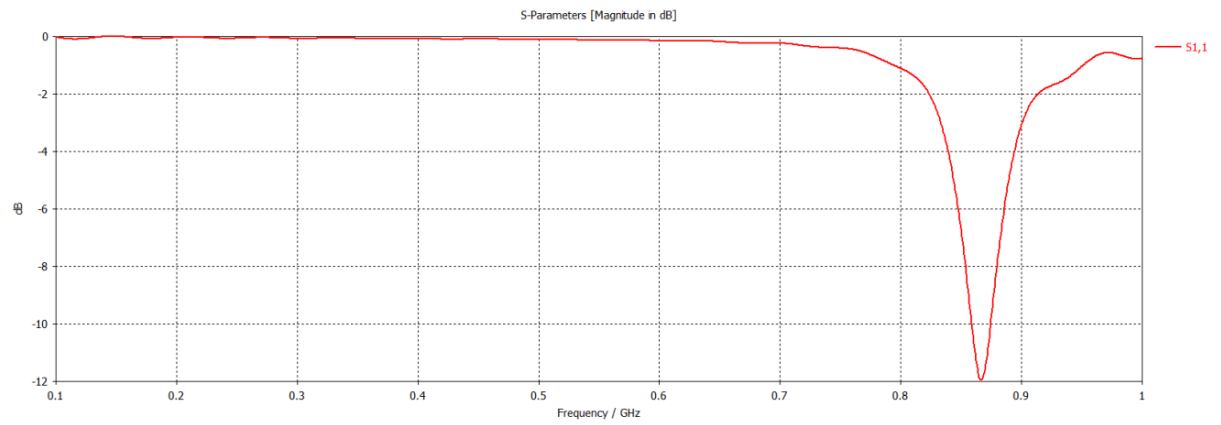
RYSUNEK 8.7: Współczynnik S11 - płytka z podciętą płaszczyzną masy o 5 mm bez optymalizacji



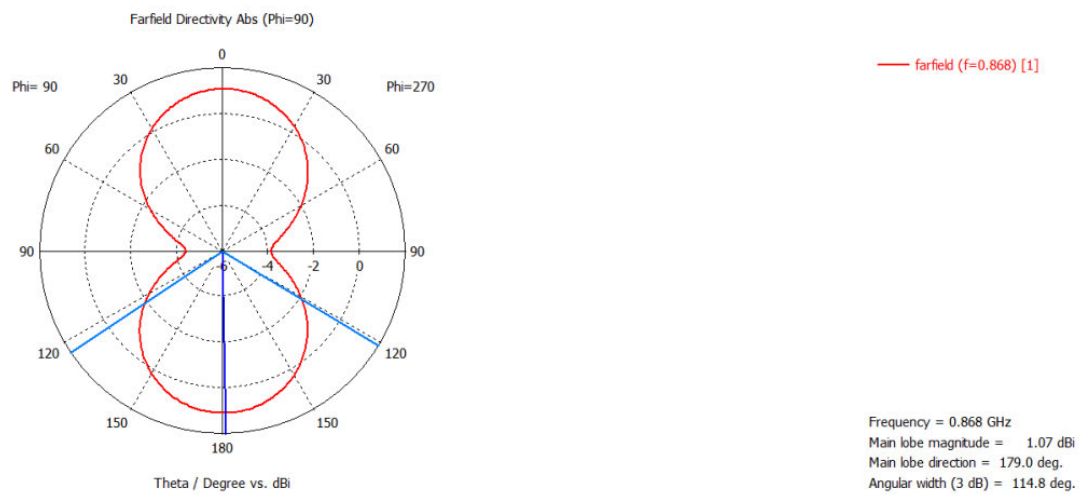
RYSUNEK 8.8: Współczynnik S11 - płytka z podciętą płaszczyzną masy o 5 mm zoptymalizowana



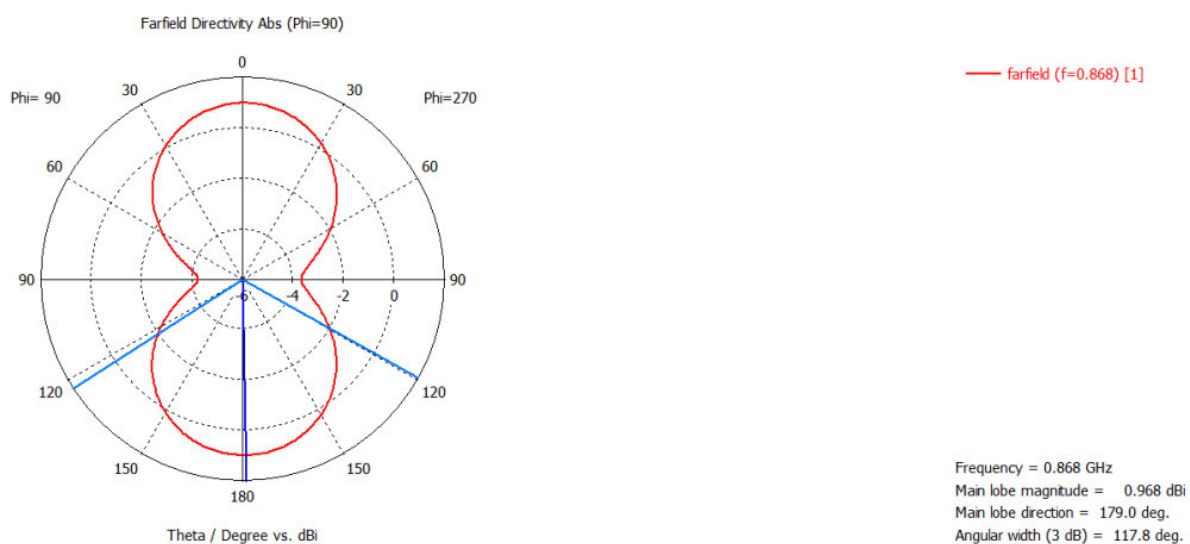
RYSUNEK 8.9: Współczynnik S11 - powiększona płytka bez optymalizacji



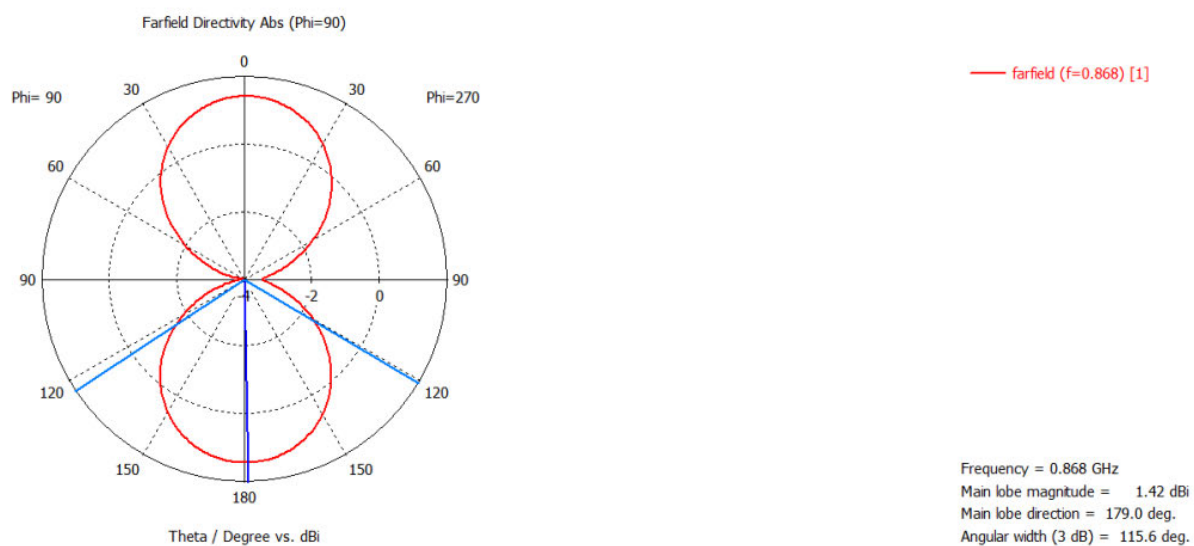
RYSUNEK 8.10: Współczynnik S11 - powiększona płytka zoptymalizowana



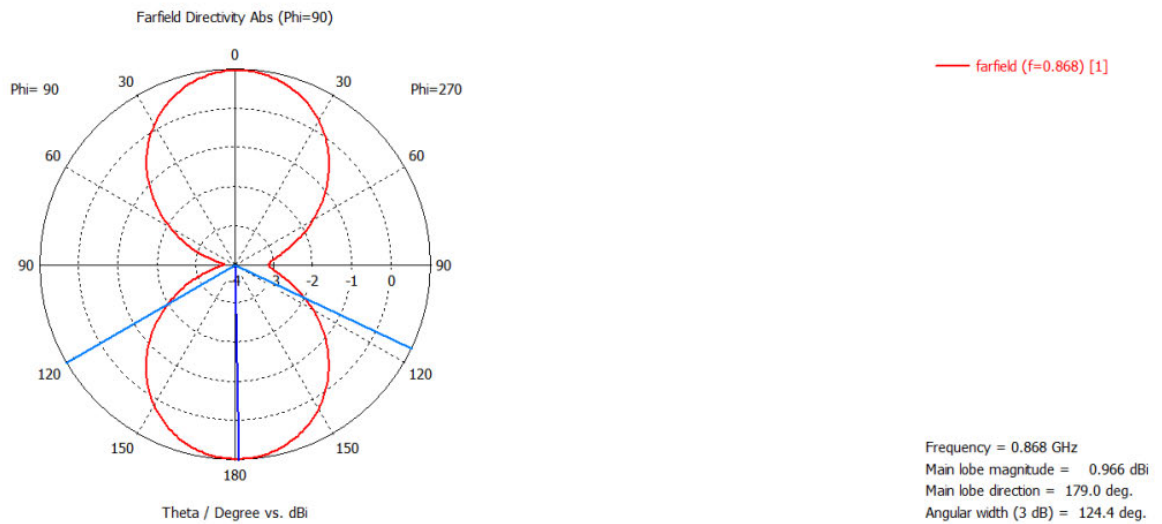
RYSUNEK 8.11: Wykres pola dalekiego - płytka bazowa bez optymalizacji



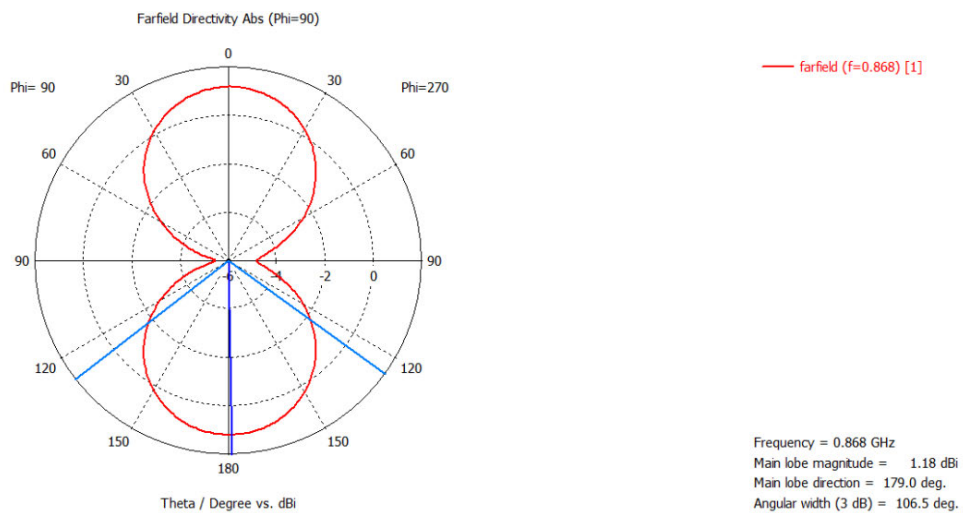
RYSUNEK 8.12: Wykres pola dalekiego - płytka bazowa zoptymalizowana



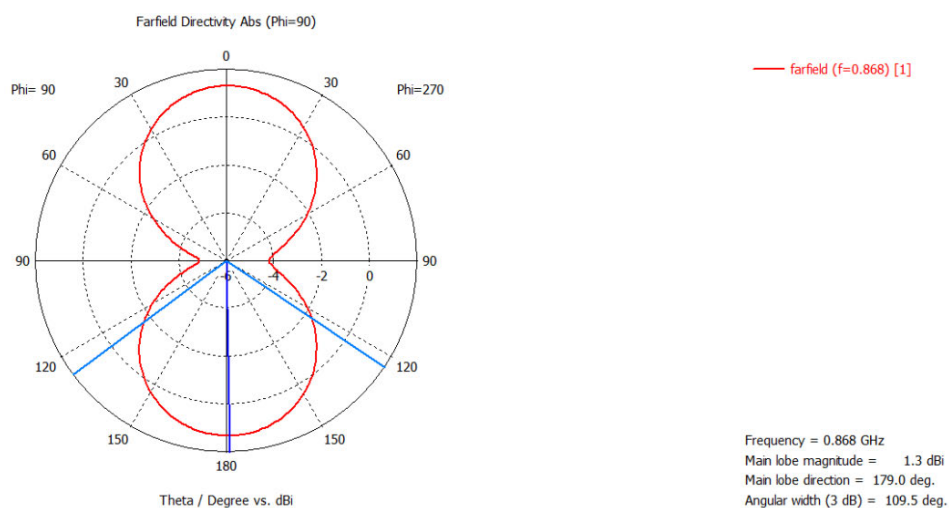
RYSUNEK 8.13: Wykres pola dalekiego - płytka z obniżoną płaszczyzną masy bez optymalizacji



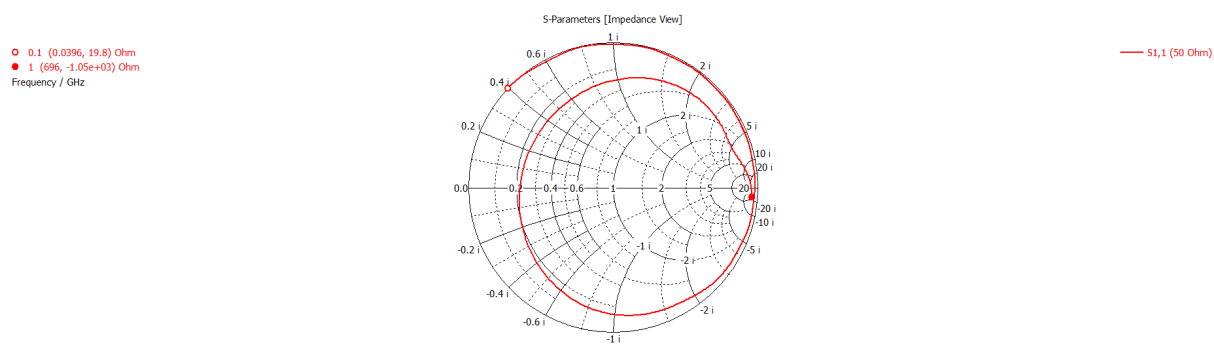
RYSUNEK 8.14: Wykres pola dalekiego - płytka z obniżoną płaszczyzną masy zoptymalizowana



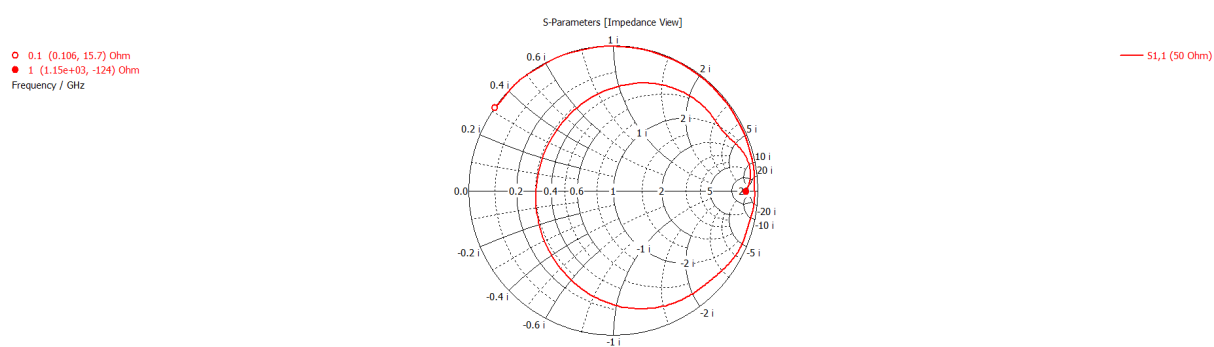
RYSUNEK 8.15: Wykres pola dalekiego - powiększona płytka bez optymalizacji



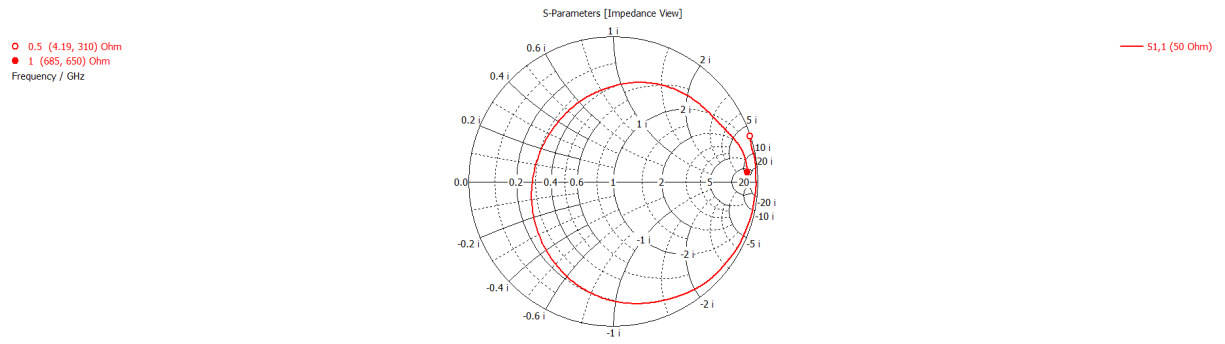
RYSUNEK 8.16: Wykres pola dalekiego - powiększona płytka zoptymalizowana



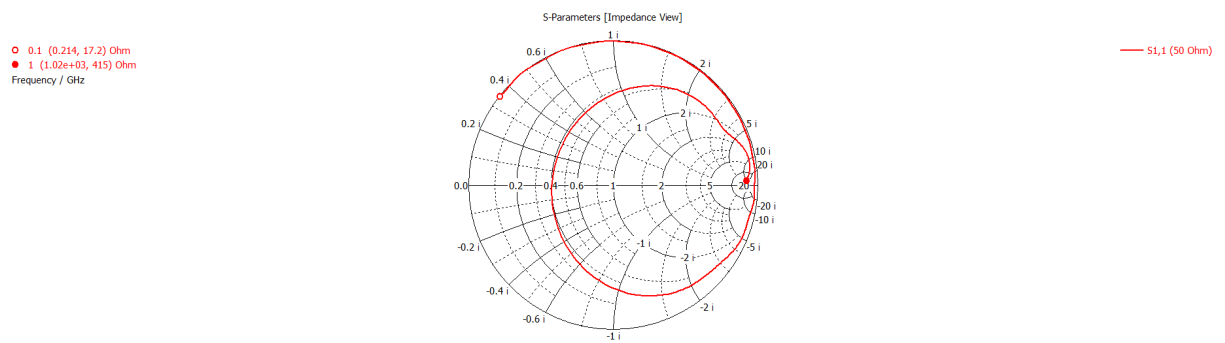
RYSUNEK 8.17: Wykres Smitha - płytka bazowa



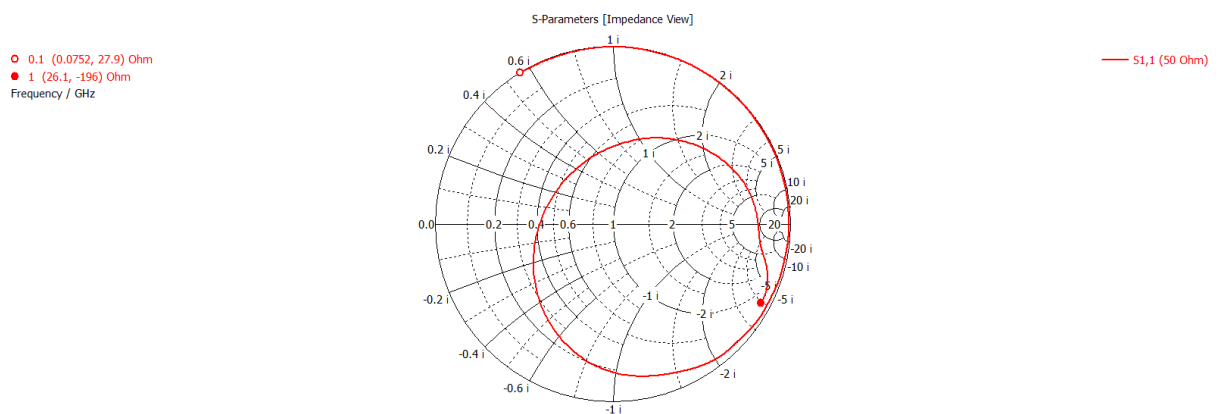
RYSUNEK 8.18: Wykres Smitha - płytka bazowa zoptymalizowana



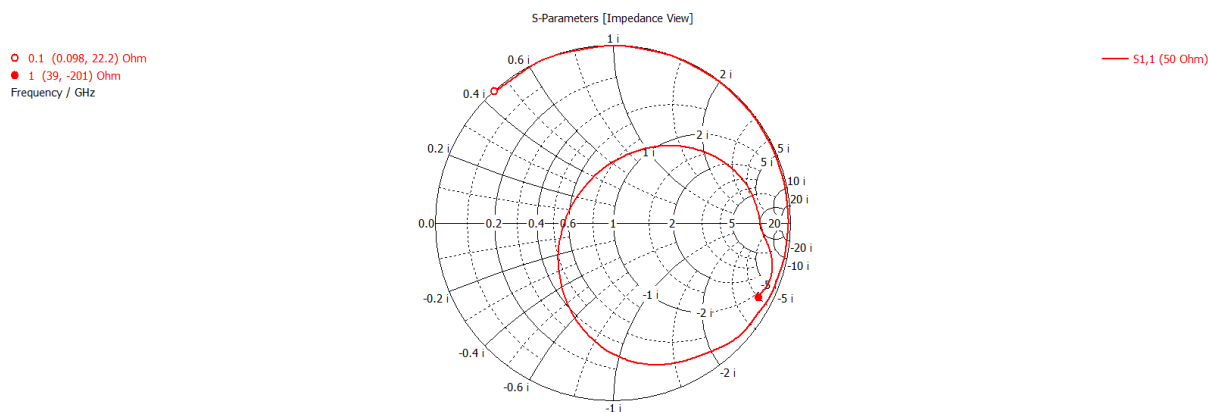
RYSUNEK 8.19: Wykres Smitha - płytka z obniżoną płaszczyzną masy bez optymalizacji



RYSUNEK 8.20: Wykres Smitha - płytka z obniżoną płaszczyzną masy optymalizowana



RYSUNEK 8.21: Wykres Smitha - powiększona płytka bez optymalizacji



RYSUNEK 8.22: Wykres Smitha - powiększona płytka optymalizowana

Wyniki symulacji dla modeli bazowych wykazały niedopasowanie toru radiowego. Wersja bazowa, nawet po optymalizacji, osiągnęła współczynnik S_{11} na poziomie zaledwie -4,5 dB dla częstotliwości 868 MHz. Wynikało to ze zbyt bliskiego sąsiedztwa litej wylewki masy, która generowała silne pasożytnicze sprzężenia pojemnościowe. Próba fizycznego podcięcia masy o 5 mm zmniejszyła pojemność brzegową, jednak drastycznie przesunęła częstotliwość rezonansową w górę (do 880 MHz po optymalizacji) oraz uniemożliwiła poprawne zestrojenie układu do wymaganej impedancji 50 Ω .

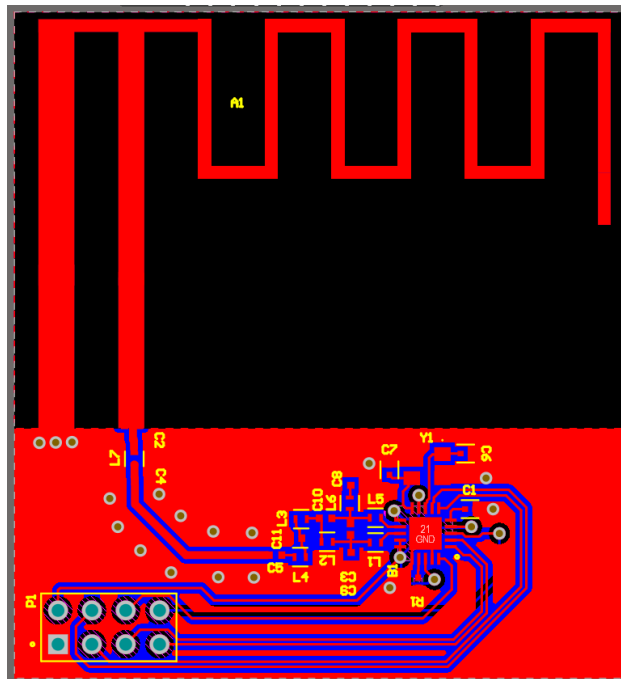
Przełom w badaniach nastąpił w wariancie płytki z wydłużonym laminatem. Już przed procesem optymalizacji struktura wykazywała poprawny kierunek zmian, zbliżając się do pożądanego rezonansu ze współczynnikiem $S_{11} = -8,6$ dB. Po zastosowaniu algorytmu optymalizacyjnego ang. *Trust Region Framework* model docelowy osiągnął wynik $S_{11} = -12,6$ dB precyzyjnie na częstotliwości środkowej 868 MHz. Z analizy wykresu Smitha wynika ponadto, że powiększona geometria pozwoliła znacząco podnieść rzeczywistą rezystancję wejściową anteny z poziomu 11,5 Ω do optymalnej wartości 32,32 Ω . Zmierzona na wykresie impedancja $Z = 32,32 + j11,65 \Omega$ stwarza dogodne warunki do finalnego dostrojenia toru za pomocą wtrąconego szeregowo kondensatora kompensacyjnego.

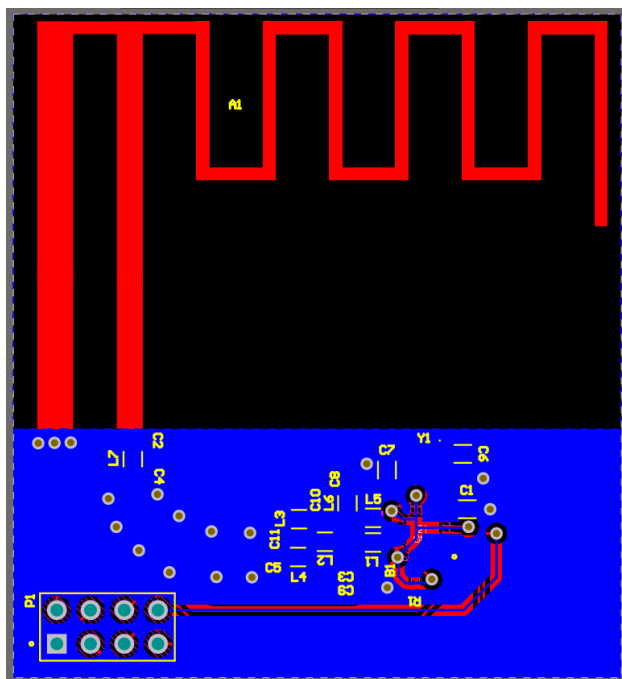
Równie istotne różnice odnotowano w analizie charakterystyk kierunkowych. W modelach bazowych zbyt mała powierzchnia przeciwwagi powodowała powstanie głębokich minimów polaryzacyjnych i silnych przewężeń wiązki. Zwiększenie pola powierzchni miedzi w wydłużonej płytce ustabilizowało rozkład prądów powierzchniowych. Uzyskano w ten sposób pożądaną, quasi-dookólną charakterystykę przestrzenną. Docelowy wariant płytki osiągnął zysk na poziomie 1,30 dBi oraz bardzo szeroki kąt połowy mocy wynoszący 109,5°. Przekłada się to bezpośrednio na eliminację martwych stref zasięgu oraz stabilną komunikację w paśmie ISM¹, niezależnie od fizycznej orientacji urządzenia w przestrzeni.

¹ang. *Industrial, Scientific, Medical* - wydzielony zakres fal radiowych przeznaczony do zastosowań przemysłowych, naukowych, medycznych oraz domowych.

8.5 Płytki PCB

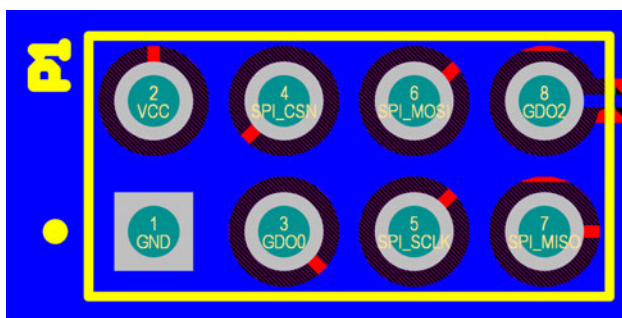
Projekt urządzenia został zrealizowany na dwuwarstwowej płytce PCB o grubości laminatu FR4 0.8 mm. Układ płytki został zaprojektowany ze szczególnym uwzględnieniem wymagań dla układów wysokiej częstotliwości (w.cz.), dążąc do minimalizacji strat sygnału użytecznego oraz ograniczenia emisji niepożądanych zakłóceń elektromagnetycznych.





RYSUNEK 8.24: Szcik płytki drukowanej opartej na układzie CC1101 strona spodnia

Warstwa dolna (rys. 8.24) pełni w dużej mierze funkcję litej płaszczyzny masy. Jest to kluczowy element dla stabilnej pracy toru RF. Wzdłuż ścieżki przenoszącej częstotliwość radiową rozmieszczono przelotki zszywające. Tworzą one barierę ekranującą i zapewniają, że płaszczyzna masy na warstwie wierzchniej ma ten sam potencjał co główna masa na warstwie spodniej, zapobiegając powstawaniu pętli masy. Na omawianej stronie płytki znajduje się również ścieżka sygnałowa dla pinu funkcyjnego GDO2 oraz część ścieżek zasilających tuż pod układem CC1101.



RYSUNEK 8.25: Złącze z wyprowadzeniami modułu

Na rysunku 8.25 przedstawiono wyprowadzenia, służące do połączenia modułu z systemem mikroprocesorowym:

- pin 1 (GND) - masa odniesienia układu. Dla zapewnienia prawidłowego działania toru radiowego oraz minimalizacji szumów cyfrowych, wyprowadzenie to powinno zostać połączone bezpośrednio z główną płaszczyzną masy (ang. *Ground Plane*) systemu mikrokontrolera nadrzędnego za pomocą połączenia o jak najniższej impedancji. Pozwala to uniknąć powstawania pętli masy, które mogłyby negatywnie wpłynąć na czułość odbiornika RF,
- pin 2 (VCC) - główna linia zasilająca transiwer CC1101. Nominalne napięcie pracy dla tego układu wynosi 3,3 V. Z uwagi na wysoką czułość struktur w.cz. na tętnienia napięcia, linia

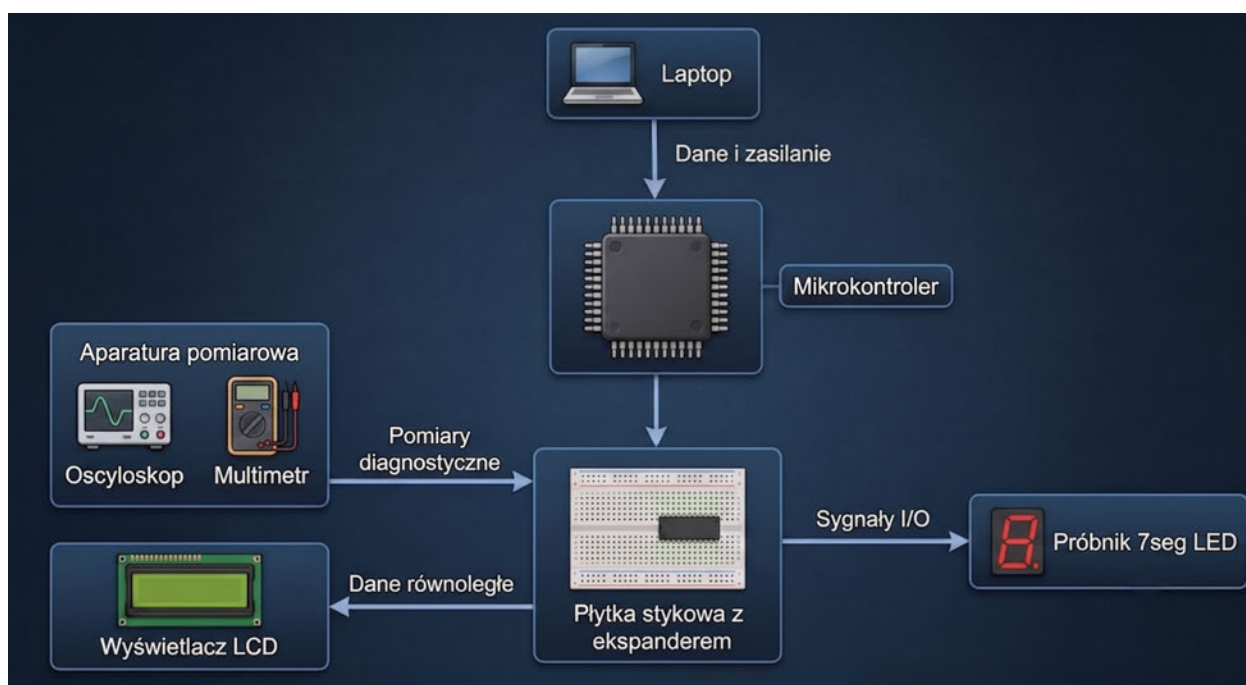
ta wymaga stabilnego źródła zasilania oraz ścisłego odsprzęgnięcia zasilania za pomocą dedykowanych kondensatorów ceramicznych o niskiej zastępczej rezystancji szeregowej (ESR), umieszczonych jak najbliżej struktury układu,

- pin 3 (GDO0) - główne, wielofunkcyjne wyjście cyfrowe ogólnego przeznaczenia (ang. *General Purpose Digital Output Pin 0*). Przy domyślnej konfiguracji lub odpowiedniej modyfikacji rejestru sterującego IOCFG0, pin ten pełni rolę sprzętowego wyjścia przerwania. Sygnalizuje on zdarzenia takie jak poprawne zdekodowanie słowa synchronizującego (ang. *Sync Word*) czy odebranie kompletnego pakietu danych, automatycznie zmieniając stan na wysoki i informując mikrokontroler Master o konieczności opróżnienia bufora RX FIFO,
- pin 4 (SPLCSN) - sygnał wyboru urządzenia na magistrali szeregowej (ang. *Chip Select Not*). Jest to wejście cyfrowe aktywne w stanie niskim (logiczne zero). Przejście linii w stan niski wybudza wewnętrzny kontroler SPI układu CC1101 i inicjuje transakcję, natomiast powrót do stanu wysokiego zatrzymuje przesłane rejestry konfiguracyjne i zwalnia magistralę, umożliwiając komunikację z innymi urządzeniami Slave,
- pin 5 (SPLSCLK) - wejściowa linia sygnału zegarowego magistrali SPI (ang. *Serial Clock*). Sygnał ten jest generowany wyłącznie przez urządzenie nadrzędne i służy do pełnej synchronizacji bitowej transferu danych. Układ CC1101 próbuje i zatrzymuje stany logiczne na liniach danych zgodnie z konfiguracją trybu SPI Mode 0,0 (próbkowanie na zboczu narastającym),
- pin 6 (SPLMOSI) - linia wejściowa danych szeregowych. Za jej pośrednictwem mikrokontroler nadrzędny przesyła do układu CC1101 bajty nagłówkowe (adresowe), komendy strobowe (ang. *Command Strokes*) oraz dane konfiguracyjne przeznaczone do zapisania w rejestrach wewnętrznych lub w buforze TX FIFO,
- pin 7 (SPLMISO) - linia wyjściowa danych szeregowych. Służy do synchronicznego wyprowadzania danych z rejestrów statusowych transiwera oraz zawartości bufora RX FIFO do pamięci mikrokontrolera. Linia ta przechodzi w stan wysokiej impedancji, gdy pin CSN znajduje się w stanie wysokim, co pozwala na współdzielenie magistrali z innymi układami peryferyjnymi,
- pin 8 (GDO2) - dodatkowe, w pełni programowalne wyjście cyfrowe ogólnego przeznaczenia (ang. *General Purpose Digital Output Pin 2*). Za pośrednictwem rejestru konfiguracyjnego IOCFG2 może zostać zaprogramowane do sygnalizowania różnorodnych stanów wewnętrznych transiwera. W zaawansowanych aplikacjach radiowych wykorzystuje się je jako wskaźnik zajętości kanału (ang. *Clear Channel Assessment – CCA*), sygnał osiągnięcia progu wypełnienia bufora FIFO lub jako sprzętowe wybudzenie w trybie energooszczędnym (ang. *Wake-on-Radio*).

Rozdział 9

Testowanie ekspanderów wejść/wyjść

9.1 Środowisko pomiarowe



RYSUNEK 9.1: Środowisko pomiarowe

Przed zaprojektowaniem płytki drukowanej z układami MCP23x17, zostało zbudowane stanowisko pomiarowe, zilustrowane na schemacie blokowym (rys. 9.1), które pozwala na kompleksową diagnozę badanego modułu [23]. Głównymi elementami zestawu były:

- komputer klasy PC (laptop): Pełni rolę nadrzędnej stacji roboczej. Służy do kompilacji kodu, programowania mikrokontrolera oraz zapewnia zasilanie dla całego układu poprzez port USB,
- mikrokontroler nadrzędny (ang. *Master*): W tej roli został wykorzystany mikrokontroler z rodziny AVR, wbudowany w płytkę rozwojową Arduino UNO. Odpowiada on za generowanie sygnałów sterujących na magistrali SPI (MOSI, SCK, SS) oraz realizację algorytmu testowego,

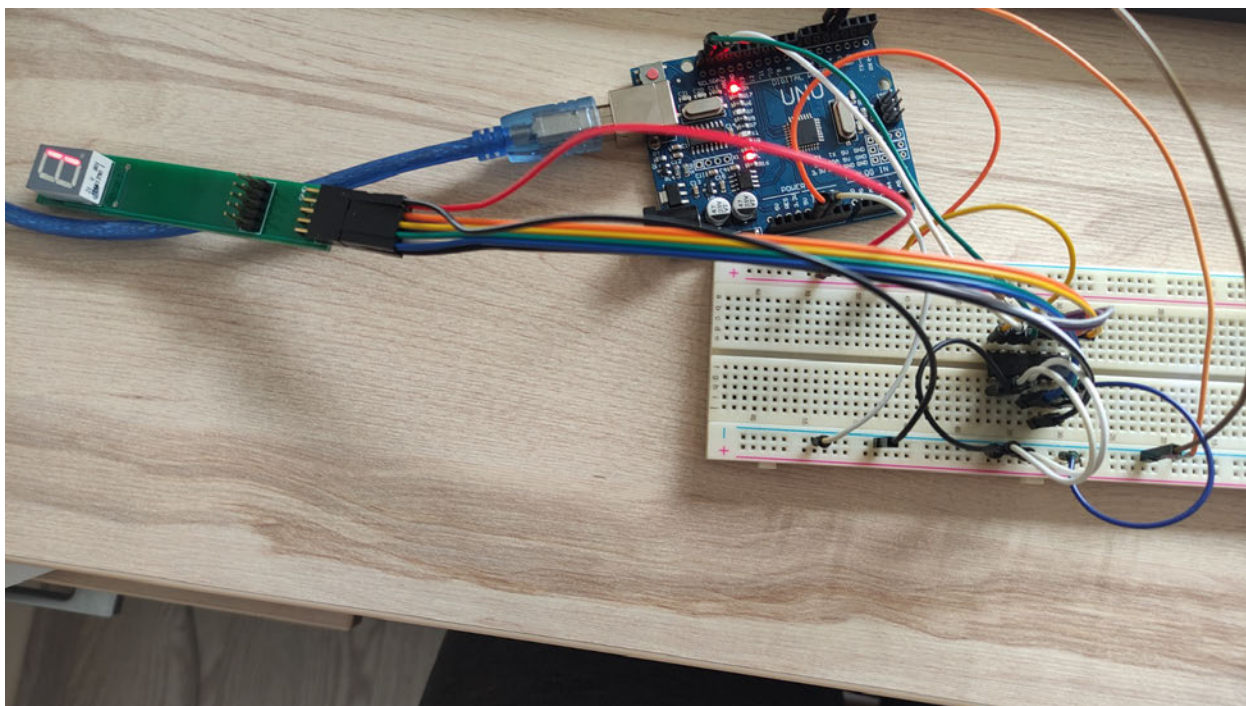
- płytka stykowa z ekspanderem: Główny obiekt badań. Zastosowanie płytki prototypowej zapewnia szybką rekonfigurację połączeń podczas testów bez konieczności lutowania,
- moduły wykonawcze i wskaźnikowe: Do weryfikacji stanów na portach ekspanderów służył próbnik 7-segmentowy LED - pełnił on funkcję prostego szybkiego wskaźnika wizualnego. Pozwala on na natychmiastową, doświadczalną weryfikację stanów logicznych na pojedynczych pinach wyjściowych ekspandera. Kolejnym elementem wykonawczym był wyświetlacz LCD oparty na sterowniku HD44780U. Służy on do weryfikacji poprawności przesyłu złożonych równoległych danych 8-bitowych oraz trzech sygnałów sterujących,
- aparatura pomiarowa: Zostały wykorzystane: Oscyloskop, który okazał się niezbędny do analizy czasowej sygnałów cyfrowych. Pozwala on na podgląd przebiegów magistrali SPI oraz badanie zjawisk takich jak drgania styków (ang. *bouncing*). Multimetr za pomocą którego zostały zweryfikowane napięcia zasilania, sprawdzenie ciągłości obwodu.

9.2 Testy funkcjonalne układów PCF8574

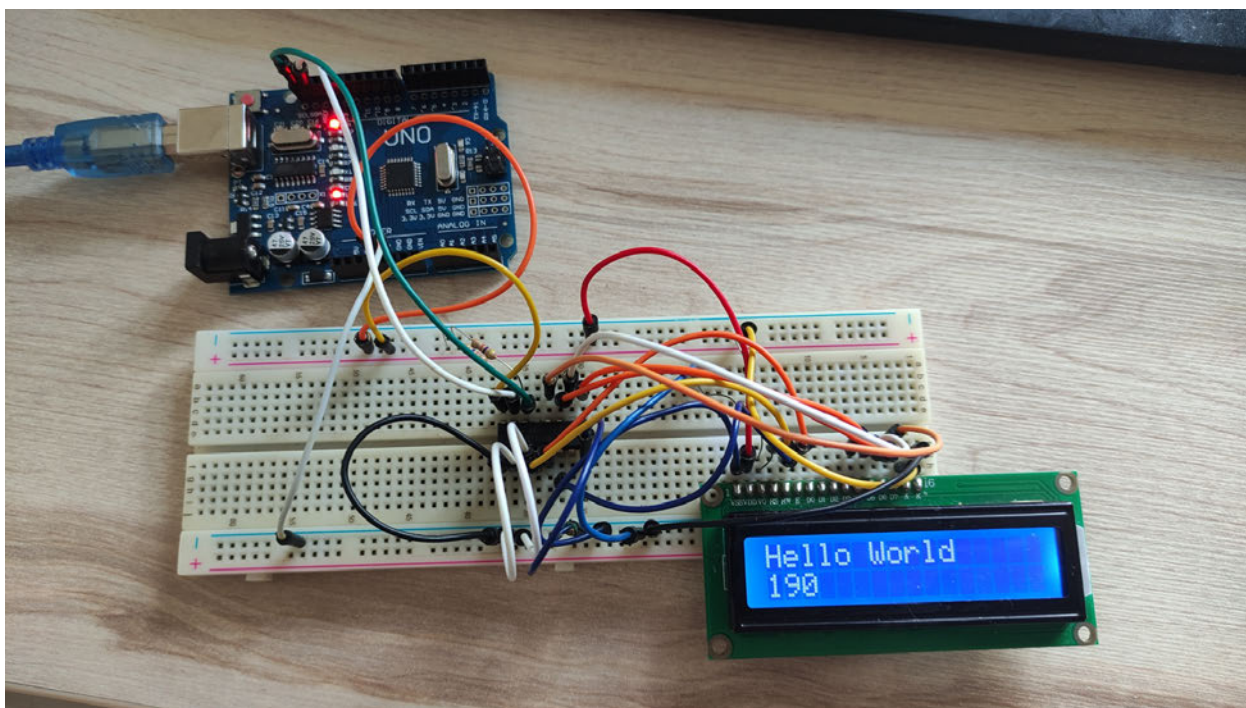
Zanim przystąpiono do tworzenia płytek drukowanych, wykonano kilka testów funkcjonalnych. Pierwszym testem było sprawdzenie działania portów GPIO. Jako wskaźnik stanów logicznych został wykorzystany moduł wyświetlacza 7-segment. Sterowany był poprzez mikrokontroler ATmega AVR 328p, za pomocą programu podobnego do przedstawionego na rysunku 9.8, różni się jedynie implementacją interfejsu, oraz przekazywaniem danych wprost, bez rejestrów sterujących. Program ten wysyła wartość liczbową na ekran z różnymi odstępami czasu. Miało to na celu sprawdzenie, jak układ zachowuje się podczas transmisji wielu danych sekwencyjnie, czy jest stabilny oraz poprawnie wystawia dane na swoich wyprowadzeniach. Jest to kluczowe do sterowania wyświetlaczem, gdzie liczy się poprawność każdej wysłanej komendy. Wynik testu został przedstawiony na rysunku 9.2.

Kolejnym krokiem były testy z docelowym wyświetlaczem, który przyjmował dane równoległe wprost zdeserializowane przy pomocy układu PCF8574. Przy pierwszej próbie układ zachował się nieoczekiwanie, a na wyświetlaczu nie ukazał się pełen napis "Hello, World", tylko "Hdlln, Vnrld". Wskazywało to na przesunięcie liter o jeden wpis w tabeli ASCII¹. Sugeruje to zamianę najmniej znaczącego bitu danych. Dodatkową oznaką nieprawidłowego działania wyświetlacza była niepełna inicjalizacja wyświetlacza. Pojawiał się kursor, który w sekwencji inicjalizacyjnej został wyłączony. Usterka, wyglądająca na groźną w rzeczywistości okazała się złym podłączeniem przewodu danych D7. Po wyeliminowaniu błędu układ działał w sposób niezakłócony, również w trybie dwustronnej komunikacji z wykorzystaniem flagi Busy. Na rysunku 9.3 został przedstawiony wynik eksperymentu. Kod sterujący znajduje się na rysunkach 9.4, 9.5, 9.6. To był jasny sygnał do przystąpienia do prac związanych z projektowaniem płytki PCB.

¹standard kodowania znaków operujący na 7-bitach



RYSUNEK 9.2: Wynik testu układu PCF8574 na próbniku 7-segmentowym



RYSUNEK 9.3: Wynik testu układu PCF8574 na wyświetlaczu HD44780 w trybie odczytu flagi BS

```

1  #include <avr/io.h>
2  #include <util/delay.h>
3  #include <avr/interrupt.h>
4  #include <stdio.h>
5
6  //Adres I2C ekspandera PCF8574 (w zależności od wersji układu i podłączenia pinów A0-A2, 0x20 to domyślny adres)
7  #define PCF8574_ADDRESS 0x20
8
9  //Mapowanie pinów ekspandera PCF8574 na piny wyświetlacza LCD
10 #define RS 0 //Register Select: 0 = komenda, 1 = dane
11 #define EN 1 //Enable: pin zezwalający na zapis/odczyt
12 #define RW 2 //Read/Write: 0 = zapis, 1 = odczyt
13
14 //Piny danych dla trybu 4-bitowego
15 #define D4 4
16 #define D5 5
17 #define D6 6
18 #define D7 7
19
20 //Zmienna przechowująca aktualny stan pinów ekspandera (tzw. shadow register)
21 uint8_t to_send = 0;
22
23 //Licznik sekund aktualizowany w przerwaniu
24 volatile uint16_t seconds = 0; //Dodano 'volatile', ponieważ zmienna jest modyfikowana w przerwaniu
25
26 //Inicjalizacja magistrali I2C (w AVR nazywanej TWI)
27 void I2C_Init() {
28     TWSR = 0x00; //Brak preskalera dla I2C
29     TWBR = 0x48; //Ustawienie prędkości transmisji (SCL). Dla zegara 16MHz daje to ok. 100 kHz
30     TWCR = (1 << TWEN); //Włączenie modułu TWI
31 }
32
33 //Generowanie warunku START na magistrali I2C
34 void I2C_Start() {
35     TWCR = (1 << TWSTA) | (1 << TWEN) | (1 << TWINT);
36     while (!(TWCR & (1 << TWINT))); //Czekaj na zakończenie operacji (ustawienie flagi TWINT)
37 }
38
39 //Generowanie warunku STOP na magistrali I2C
40 void I2C_Stop() {
41     TWCR = (1 << TWSTO) | (1 << TWEN) | (1 << TWINT);
42 }
43
44 //Wysłanie pojedynczego bajtu przez I2C
45 void I2C_Write(uint8_t data) {
46     TWDR = data; //Załadowanie danych do rejestru danych TWI
47     TWCR = (1 << TWEN) | (1 << TWINT); //Rozpoczęcie transmisji
48     while (!(TWCR & (1 << TWINT))); //Czekaj na zakończenie wysyłania
49 }
50
51 //Wysłanie bajtu danych do ekspandera PCF8574
52 void PCF8574_Write(uint8_t data) {
53     I2C_Start();
54     I2C_Write(PCF8574_ADDRESS << 1); //Wysłanie adresu z bitem R/W ustawionym na 0 (zapis)
55     I2C_Write(data); //Wysłanie właściwych danych
56     I2C_Stop();
57 }
58
59 //Odczyt bajtu danych z ekspandera PCF8574
60 uint8_t PCF8574_Read() {
61     uint8_t data;
62     I2C_Start();
63     I2C_Write((PCF8574_ADDRESS << 1) | 1); //Wysłanie adresu z bitem R/W ustawionym na 1 (odczyt)
64     TWCR = (1 << TWEN) | (1 << TWINT); //Odbiór danych (bez potwierdzenia ACK - NACK)
65     while (!(TWCR & (1 << TWINT))); //Czekaj na zakończenie odbioru
66     data = TWDR; //Zapisz odebrane dane
67     I2C_Stop();
68     return data;
69 }
70
71 //Pulsowanie pinu EN (Enable), które zatwierdza dane na pinach D4-D7 wyświetlacza
72 void lcd_enable(){
73     to_send |= (1<<EN); //Ustawienie EN na stan wysoki
74     PCF8574_Write(to_send);
75     _delay_us(1); //Krótkie opóźnienie dla stabilizacji
76     to_send &= ~(1<<EN); //Ustawienie EN na stan niski (opadające zbocze zatwierdza dane)
77     PCF8574_Write(to_send);
78     _delay_us(50); //Czas na przetworzenie przez LCD
79 }
80

```

RYSUNEK 9.4: Kod programu testowego układu PCF8574. Część I

```

81 //Sprawdzenie flagi zajętości (Busy Flag) wyświetlacza LCD
82 bool checkBS(){
83     to_send &= ~(1 << RS); //RS = 0 (tryb instrukcji)
84     to_send |= (1 << RW); //RW = 1 (tryb odczytu)
85     PCF8574_Write(to_send);
86
87     to_send |= (1<<EN); //EN = 1 (rozpoczęcie odczytu)
88     PCF8574_Write(to_send);
89     _delay_us(1);
90
91     //Odczyt stanu pinów, flaga zajętości znajduje się na pinie D7
92     bool busy = (PCF8574_Read() & (1 << D7));
93
94     to_send &= ~(1<<EN); //EN = 0 (zakończenie odczytu pierwszej połowy bajtu)
95     PCF8574_Write(to_send);
96
97     //Wyświetlacz w trybie 4-bitowym wymaga drugiego taktu EN dla młodszych 4 bitów,
98     //nawet jeśli nas interesuje tylko starsza część (z flagą D7)
99     to_send |= (1<<EN);
100    PCF8574_Write(to_send);
101    _delay_us(1);
102    to_send &= ~(1<<EN);
103    PCF8574_Write(to_send);
104
105    to_send &= ~(1 << RW); //Powrót do trybu zapisu (RW = 0)
106    PCF8574_Write(to_send);
107
108    return busy; //Zwraca 1 jeśli LCD jest zajęty, 0 jeśli gotowy
109 }
110
111 //Pętla oczekująca na zwolnienie flagi zajętości LCD
112 void wait_until_BS(){
113     while(checkBS()==1){
114         _delay_us(1);
115     }
116 }
117
118 //Funkcja wysyłająca bajt do LCD w trybie 4-bitowym (najpierw starsze, potem młodsze 4 bity)
119 void lcd_send(uint8_t data, uint8_t is_data){
120     // Ustawienie pinu RS w zależności od tego, czy wysyłamy komendę czy znak
121     if(is_data) to_send |= (1<<RS);
122     else to_send &= ~(1<<RS);
123
124     //Wysłanie starszej połówki bajtu (bity 4-7)
125     //Czyścimy starsze 4 bity w to_send i wstawiamy starsze 4 bity z 'data'
126     to_send = (to_send & 0b0001111) | (data & 0b1111000);
127     PCF8574_Write(to_send);
128     lcd_enable(); //Zatwierdzenie starszej połówki
129
130     //Wysłanie młodszej połówki bajtu (bity 0-3) przesuniętej na pozycje 4-7
131     to_send = (to_send & 0b0001111) | ((data << 4) & 0b1111000);
132     PCF8574_Write(to_send);
133     lcd_enable(); //Zatwierdzenie młodszej połówki
134
135     wait_until_BS(); //Oczekiwanie na przetworzenie przez LCD
136 }
137
138 //Wysłanie komendy do LCD
139 void lcd_command(uint8_t cmd){
140     lcd_send(cmd, 0); // is_data = 0
141 }
142
143 //Wysłanie znaku (danych) do LCD
144 void lcd_data(uint8_t data){
145     lcd_send(data, 1); // is_data = 1
146 }
147
148 //Procedura inicjalizacji kontrolera HD44780 w trybie 4-bitowym
149 void lcd_init(){
150     _delay_ms(40); //Czekaj na ustabilizowanie napięcia zasilania LCD (>15ms)
151
152     //Sekwencja inicjalizacyjna "wymuszająca" reset i tryb 8-bitowy (roboczo)
153     to_send = 0b0000011 << 4;
154     lcd_enable();
155     _delay_ms(5);
156
157     lcd_enable();
158     _delay_us(150);
159
160     lcd_enable();

```

RYSUNEK 9.5: Kod programu testowego układu PCF8574. Część II

```

161     wait_until_BS();
162
163     //Przełączenie w tryb 4-bitowy
164     to_send = 0b00000010 << 4;
165     lcd_enable();
166     wait_until_BS();
167
168     //Konfiguracja LCD (już w trybie 4-bitowym przy użyciu pełnej funkcji lcd_command)
169     lcd_command(0b00101000); //Interfejs 4-bitowy, 2 linie, czcionka 5x8
170     lcd_command(0b00001100); //Włącz wyświetlacz, wyłącz kursor i jego miganie
171     lcd_command(0b00000001); //Wyczyść ekran (Clear Display)
172
173     lcd_command(0b00000110); //Inkrementowanie adresu po każdym znaku (Entry Mode Set)
174 }
175
176 //Ustawienie kursora na wybranej pozycji (wiersz, kolumna)
177 void lcd_set_cursor(uint8_t row, uint8_t col){
178     uint8_t pos = 0;
179     if(row == 0) pos = col;           //Wiersz 0 zaczyna się od adresu 0x00
180     else pos = 0b01000000 + col;     //Wiersz 1 zaczyna się od adresu 0x40 (64)
181     lcd_command(0b10000000 | pos);   //Komenda Set DDRAM Address (najstarszy bit = 1)
182 }
183
184 // Wyświetlenie ciągu znaków (string) na LCD
185 void lcd_print(const char *str) {
186     while (*str) {
187         lcd_data(*str++); //Wysyłaj znak po znaku, aż natrafisz na znak końca '\0'
188     }
189 }
190
191 //Konfiguracja Timera 1 do odliczania sekund
192 void timer1_init(){
193     //Tryb CTC (Clear Timer on Compare Match) i preskaler = 1024
194     TCCR1B |= (1<<WGM12) | (1<<CS12) | (1<<CS10);
195
196     //Ustawienie wartości porównania (TOP).
197     //Wzór: F_CPU / (Preskaler * Częstotliwość) - 1
198     //Dla 16 MHz: 16000000 / (1024 * 1Hz) - 1 = 15624
199     OCR1A = 15624;
200
201     //Włączenie przerwania od zrównania z OCR1A
202     TIMSK1 |= (1<<OCIE1A);
203
204     //Globalne odblokowanie przerw
205     sei();
206 }
207
208 int main() {
209     I2C_Init();           //Inicjalizacja I2C
210     lcd_init();           //Inicjalizacja LCD
211     timer1_init();        //Inicjalizacja Timera 1
212
213     lcd_set_cursor(0, 0); //Ustawienie na początku pierwszej linii
214     lcd_print("Hello World");
215
216     char buffer[16]; //Bufor tekstowy dla funkcji sprintf
217
218     while (1) {
219         lcd_set_cursor(1, 0); //Ustawienie na początku drugiej linii
220         sprintf(buffer, "%u", seconds); //Konwersja liczby całkowitej na tekst
221         lcd_print(buffer); //Wyświetlenie aktualnej liczby sekund
222     }
223
224     return 0;
225 }
226
227 //Wektor przerwania Timera 1 (wywoływany co 1 sekundę)
228 ISR(TIMER1_COMPA_vect){
229     seconds++; //Zwiększenie licznika
230 }

```

RYSUNEK 9.6: Kod programu testowego układu PCF8574. Część III

9.3 Testy funkcjonalne układów MCP23017

Pozytywny rezultat testów funkcjonalnych na układzie PCF8574 skutkował przeprowadzeniem badań z użyciem ekspanderów z serii MCP23x17. Na rysunku 9.7 przedstawiono fragment obsługi magistrali I²C. Jedną z różnic względem kodu z testów układu PCF8574 było użycie nowej funkcji `void MCP_Write`, która pozwalała wykorzystać złożoność ekspandera MCP23017 tzn. wpisywać dane do poszczególnych rejestrów urządzenia. W kodzie wykonano kilka zestawów badawczych, które miały sprawdzić zachowanie MCP23017 przy zapisie do poszczególnych rejestrów I/O (OLAT i GPIO). W tym celu wykorzystano dwa próbniki 7-segment, ponieważ expander ma dwa porty wejść/wyjść. Kod zawierał 4 testy (zmienna `test_mode`), które służyły do sprawdzania układu MCP23017. Za pomocą wcześniej wymienionej funkcji zapisywania danych, wysyłano koordynaty dla próbników (wyświetlane cyfry). Zależnie od wybranego trybu pracy informacje były zapisywane na różnych rejestrach lub na wszystkich tego samego rodzaju. Wnioskiem z badania zachowania układu było stwierdzenie, że niezależnie od wybranego rejestru (na danym porcie) wynik na wyświetlaczu był natychmiastowo aktualizowany. Rozważania na ten temat były spowodowane niejednoznacznym wyjaśnieniem pracy i zachowań rejestrów I/O (OLAT, GPIO) w nocie katalogowej ekspandera.

```

36 //Obsługa sprzętowa I2C (w AVR TWI)
37
38 void I2C_Init() {
39     TWSR = 0x00;
40     TWBR = 0x2; //Prędkość transmisji
41     TWCR = (1 << TWEN);
42 }
43
44 void I2C_Start() {
45     TWCR = (1 << TWSTA) | (1 << TWEN) | (1 << TWINT);
46     while (!(TWCR & (1 << TWINT)));
47 }
48
49 void I2C_Stop() {
50     TWCR = (1 << TWSTO) | (1 << TWEN) | (1 << TWINT);
51
52     //ta poprawka naprawiła problem komunikacji z MCP, przy większych prędkościach znak start nadpisywał STOP i układ sie gubił
53     while(TWCR & (1 << TWSTO));
54     _delay_us(10);
55 }
56
57 void I2C_Write(uint8_t data) {
58     TWDR = data;
59     TWCR = (1 << TWEN) | (1 << TWINT);
60     while (!(TWCR & (1 << TWINT)));
61 }
62
63 // --- Obsługa MCP23017 ---
64 void MCP_Write(uint8_t reg, uint8_t data) {
65     I2C_Start();
66     I2C_Write(MCP_OPCODE); //Wysłanie adresu urządzenia (z bitem zapisu)
67     I2C_Write(reg);        //Wysłanie adresu rejestru
68     I2C_Write(data);       //Wysłanie danych
69     I2C_Stop();
70 }

```

RYSUNEK 9.7: Fragment kodu obsługujący komunikację z MCP23017

9.4 Testy funkcjonalne układów MCP23S17

Biorąc pod uwagę fakt, że układy MCP23017 oraz MCP23S17 współdzielią niemal identyczną architekturę wewnętrzną - posiadają ten sam rdzeń logiczny, mapę rejestrów oraz strukturę portów wejścia/wyjścia, proces weryfikacji sprzętowej oparto na założeniach i środowisku testowym przygotowanym wcześniej dla układu na szynie I²C. Zastosowanie analogicznej metody badawczej, wykorzystującej próbniki 7-segmentowe jako deterministyczne wskaźniki stanów logicznych, pozwoliło na rzetelne i miarodajne porównanie obu rozwiązań technologicznych.

Przejście na protokół SPI udowodniło zalety modularnego pisania kodu. Najważniejsze funkcje i maszyna stanów głównego programu testowego nie uległy zmianie. Modyfikacji poddano wyłącznie warstwę fizyczną transferu danych. Transakcje oparte na adresowaniu I²C oraz bitach potwierdzeń zostały zastąpione przez szybką, synchroniczną transmisję szeregową. Wymagało to zaimplementowania programowej obsługi pinu wyboru pinu CS. Komunikacja z układem MCP23S17 wymagała każdorazowego wygenerowania stanu niskiego na linii CS, a następnie wysłaniu trzech bajtów poprzez linię MOSI: bajtu kontrolnego (tzw. *Opcode* zawierający sprzętowy adres układu), bajtu adresu rejestru oraz właściwego bajtu danych. Dopiero powrót linii CS do stanu wysokiego powodował zatrzaśnięcie przesłanych informacji w strukturze układu. Cały kod realizujący te operacje, z uwzględnieniem podziału na logiczne bloki inicjalizacyjne i wykonawcze, jest widoczny na rysunkach 9.8, 9.9, 9.10.

Kluczowym wnioskiem płynącym ze zmiany interfejsu komunikacyjnego była poprawa przepływności danych. Magistrala SPI, w przeciwieństwie do I²C, nie wykorzystuje topologii otwartego drenu i rezystorów podciągających, lecz opiera się na wyjściach typu *push-pull* [17]. Pozwala to na uzyskanie bardzo stromych zboczy sygnałów, co z kolei umożliwia taktowanie linii zegarowej SCK częstotliwościami rzędu kilku, a nawet kilkunastu megaherców. Dodatkowo, brak narzutu protokołu w postaci bitów startu, stopu, oraz konieczności oczekiwania na bity potwierdzeń od układu *slave*, zminimalizował czas propagacji instrukcji od mikrokontrolera do wyprowadzeń ekspandera.

Podczas testów szczególną uwagę zwrócono na interakcję z rejestrami sterującymi kierunkiem i stanem portów, które w modelu SPI posiadają dokładnie te same adresy (np. 0x12 dla GPIOA, 0x14 dla OLATA), co w modelu I²C. Zgodnie z założeniami teoretycznymi opisanymi w nocie katalogowej, zapis danych do rejestru portu (GPIO) oraz bezpośrednio do rejestru zatrzaśków wyjściowych (OLAT) przynosił identyczne, natychmiastowe rezultaty. Wyniki obserwacji potwierdziły wnioski płynące z poprzednich badań: stany logiczne na wyprowadzeniach układu MCP23S17 były aktualizowane bez zauważalnych opóźnień i prawidłowo, stabilnie odzwierciedlane na diodach próbników 7-segmentowych, bez efektu drgania czy migotania sygnału.

Pomyślne zakończenie testów z wykorzystaniem próbników 7-segmentowych udowodniło pełną kompatybilność programową i architektoniczną obu układów. Skrócenie czasu transmisji pojedynczego bajtu komendy dzięki magistrali SPI bezpośrednio rozwiązuje problem wąskiego gardła komunikacyjnego. Wyniki te stanowią fundament pod docelową implementację sterownika LCD zgodnego ze standardem HD44780. Pozwala to założyć, że oprogramowanie wyższego poziomu będzie funkcjonować całkowicie transparentnie, niezależnie od tego czy fizycznym mostkiem między mikrokontrolerem a matrycą LCD jest układ MCP23017, czy jego szybszy odpowiednik MCP23S17 [2].

```

1  #include <avr/io.h>
2  #include <stdio.h>
3  #include <avr/interrupt.h>
4  #include <util/delay.h>
5
6  #define F_CPU 16000000UL //częstotliwość zegara 16 MHz
7  #define MISO PB4
8  #define MOSI PB3
9  #define SCK PB5
10 #define SS PB2
11 #define MCP_RESET PB1
12
13 //Definicje Rejestrów MCP23S17 (BANK=0)
14 #define MCP_IODIRA 0x00
15 #define MCP_IODIRB 0x01
16 #define MCP_GPIOA 0x12 //Można też użyć OLATA (0x14)
17 #define MCP_GPIOB 0x13 //Można też użyć OLATB (0x15)
18 #define MCP_OLATA 0x14
19 #define MCP_OLATB 0x15
20
21 #define MCP_OPCODE 0x40 //A0,A1,A2 = GND
22
23 #define BTN1 PD0
24 #define BTN2 PD1
25 #define BTN3 PD2
26 #define BTN4 PD3
27 #define BTNSEL PD4
28
29 volatile uint16_t seconds = 0;
30
31 uint8_t portB_shadow = 0; //zmienna globalna dla stanu portu B układu MCP
32
33 volatile uint8_t test_mode = 0;
34
35 // --- Inicjalizacja Przycisków na Przerwaniach (PCINT) ---
36 void buttons_init() {
37     //Ustawienie pinów jako wejścia
38     DDRD &= ~(1<<BTN1) | (1<<BTN2) | (1<<BTN3) | (1<<BTN4) | (1<<BTNSEL));
39
40     //Włączenie Pull-Up (Wymagane, nawet przy debouncingu sprzętowym, chyba że jest zewnętrzny rezystor)
41     PORTD |= (1<<BTN1) | (1<<BTN2) | (1<<BTN3) | (1<<BTN4) | (1<<BTNSEL);
42
43     //Włączenie przerwań PCINT dla grupy PCINT2 (Port D)
44     PCICR |= (1 << PCIE2);
45
46     //Wybór konkretnych pinów, które mają wywołać przerwanie
47     PCMSK2 |= (1 << PCINT16) | (1 << PCINT17) | (1 << PCINT18) | (1 << PCINT19);
48
49     //Globalne przerwania włączamy w main przez sei()
50 }
51
52 void SPI_INIT()
53 {
54     DDRB |= (1 << MOSI) | (1 << SCK) | (1 << SS) | (1 << MCP_RESET); //ustawienie pinów jako wyjścia
55     DDRB &= ~(1 << MISO); //ustawienie pinów jako wejścia
56     PORTB |= (1 << SS) | (1 << MCP_RESET); //ustawienie SS i MCP_RESET na wysokim poziomie
57     SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR0); //włączenie SPI jako tryb master i zegar F_CPU/16
58 }
59
60 uint8_t SPI_Transfer(uint8_t data)
61 {
62     SPDR = data; //wysłanie bajtu
63     while (!(SPSR & (1 << SPIF))); //czekanie na zakończenie transmisji
64     return SPDR;
65 }
66

```

RYSUNEK 9.8: Kod programu testowego dla MCP23S17, ustawiający piny GPIO.
Część I

```

67 // --- Obsługa MCP23S17 ---
68 void MCP_Write(uint8_t reg, uint8_t data) {
69     PORTB &= ~(1 << SS); //CS Low (Start)
70     _delay_ms(1);
71     SPI_Transfer(MCP_OPCODE);
72     SPI_Transfer(reg);
73     SPI_Transfer(data);
74     _delay_ms(1);
75     PORTB |= (1 << SS); //CS High (Stop)
76 }
77
78 void MCP_Init(void) {
79     //Ustawienie IODIRA i IODIRB na 0x00 (Wszystkie piny jako WYJŚCIA)
80     //Dzięki trybowi sekwencyjnemu (domyślnemu), można wysłać to jednym ciągiem!
81
82     PORTB &= ~(1 << SS); //CS Low
83     SPI_Transfer(MCP_OPCODE);
84     SPI_Transfer(MCP_IODIRA); //Start od adresu 0x00
85     SPI_Transfer(0x00); //Wpis do IODIRA (0x00) -> wskaźnik sam skoczy na 0x01
86     SPI_Transfer(0x00); //Wpis do IODIRB (0x01)
87     PORTB |= (1 << SS); //CS High
88     _delay_ms(10);
89     MCP_Write(MCP_GPIOB, 0x00);
90     MCP_Write(MCP_GPIOA, 0x00);
91 }
92
93 void MCP_SoftReset() {
94     PORTB &= ~(1 << MCP_RESET);
95     _delay_ms(10);
96     PORTB |= (1 << MCP_RESET);
97     _delay_ms(10);
98     MCP_Init(); //Konieczna re-inicjalizacja po resecie!
99 }
100
101 int main(void)
102 {
103     SPI_INIT();
104     MCP_Init();
105     buttons_init();
106     sei();
107
108     while (1) {
109         if (test_mode == 0) {
110             //MCP_SoftReset();
111             portB_shadow = 0x60;
112             MCP_Write(MCP_GPIOA, portB_shadow);
113             MCP_Write(MCP_GPIOB, portB_shadow);
114             _delay_ms(3000);
115             if( !(PIND & (1 << BTNSSEL)) ) //jeżeli przycisk zwarty do GND
116             {
117                 portB_shadow = 0b00000010;
118                 MCP_Write(MCP_GPIOA, portB_shadow);
119                 MCP_Write(MCP_GPIOB, portB_shadow);
120                 _delay_ms(1000);
121             }
122         }
123         else if(test_mode == 1){
124             //MCP_SoftReset();
125             portB_shadow = 0x60;
126             MCP_Write(MCP_OLATA, portB_shadow);
127             MCP_Write(MCP_OLATB, portB_shadow);
128             _delay_ms(3000);
129             if( !(PIND & (1 << BTNSSEL))) //jeżeli przycisk zwarty do GND
130             {
131                 portB_shadow = 0b00000010;
132                 MCP_Write(MCP_OLATA, portB_shadow);
133                 MCP_Write(MCP_OLATB, portB_shadow);
134                 _delay_ms(1000);
135             }
136         }
137     }
138 }

```

RYSUNEK 9.9: Kod programu testowego dla MCP23S17, ustawiający piny GPIO.
Część II

```
137     else if(test_mode == 2){
138         //MCP_SoftReset();
139         portB_shadow = 0x60;
140         MCP_Write(MCP_OLATA, portB_shadow);
141         MCP_Write(MCP_OLATB, portB_shadow);
142         _delay_ms(3000);
143         portB_shadow = 0xDA;
144         MCP_Write(MCP_GPIOA, portB_shadow);
145         MCP_Write(MCP_GPIOB, portB_shadow);
146         _delay_ms(3000);
147     }
148     else if(test_mode == 3){
149         //MCP_SoftReset();
150         portB_shadow = 0x60;
151         MCP_Write(MCP_GPIOA, portB_shadow);
152         MCP_Write(MCP_GPIOB, portB_shadow);
153         _delay_ms(3000);
154         portB_shadow = 0xDA;
155         MCP_Write(MCP_OLATA, portB_shadow);
156         MCP_Write(MCP_OLATB, portB_shadow);
157         _delay_ms(3000);
158     }
159 }
160 return 0;
161 }
```

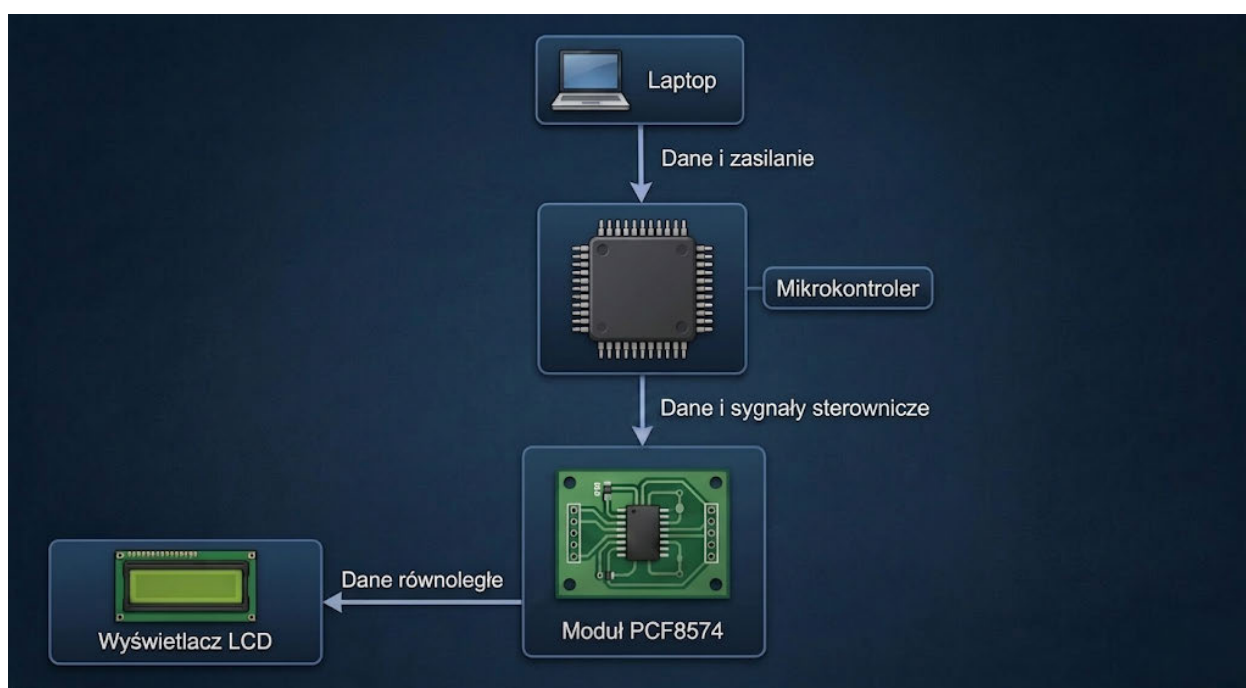
RYSUNEK 9.10: Kod programu testowego dla MCP23S17, ustawiający piny GPIO.
Część III

Rozdział 10

Uruchamianie modułu LCD z 8-bitowymi ekspanderami PCF8574

10.1 Środowisko pomiarowe

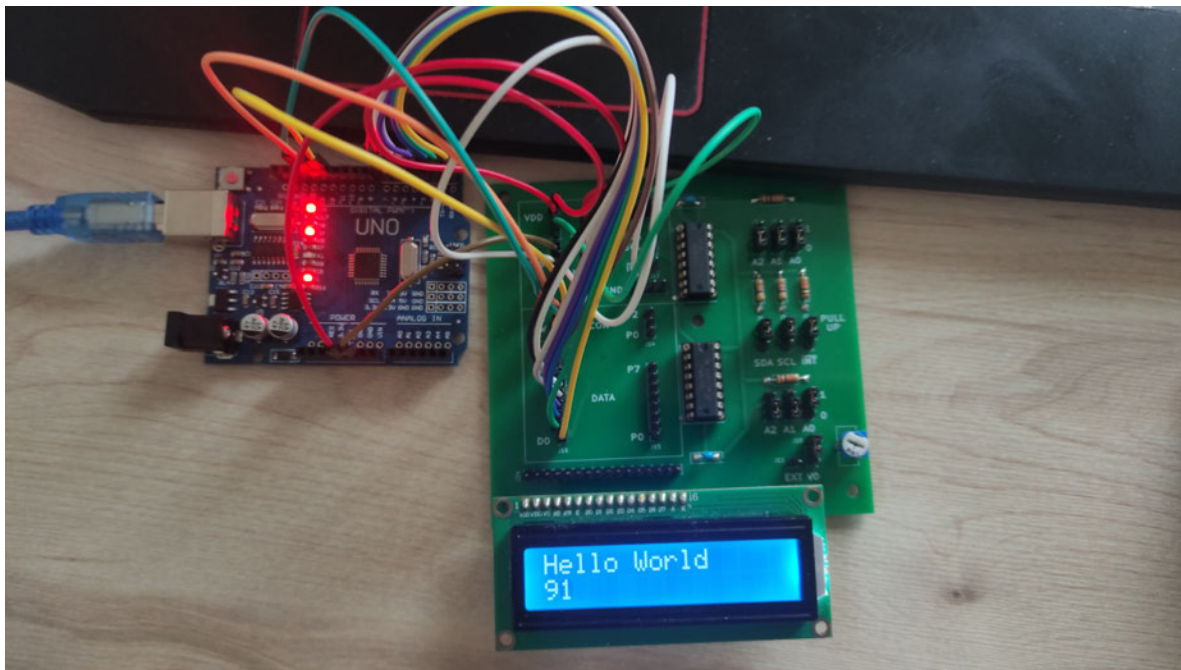
Przetestowanie ekspanderów pod kątem poprawności działania i projekt płytek drukowanych doprowadziły do wyprodukowania modułów. Gotowy obwód dał możliwość badania układów bez troski o jakość połączenia tak jak w przypadku płytek stykowych. Na rysunku 10.1 przedstawiono schemat blokowy środowiska testowego dla modułu z ekspanderami PCF8574. Wykonano zestaw badań, które zostaną opisane szczegółowo w następnych podrozdziałach.



RYSUNEK 10.1: Poglądowy schemat modułu z PCF8574

10.2 Testy modułu z 8-bitową transmisją równoległą danych

Zgodnie z projektem moduł obsługiwał dane i sygnały sterujące wysyłane wprost z mikrokontrolera na wyświetlacz LCD. W tym trybie ekspandery były całkowicie pominięte. Na rysunku 10.3 oraz 10.4 przedstawiono kod potrzebny do uruchomienia 8-bitowej transmisji równoległej. W kodzie znajdował się szereg funkcji konfiguracyjnych, inicjalizacyjnych, transmisyjnych i wykonawczych. Na rysunku 10.2 oraz 10.7 można zaobserwować wynik omówionych badań.



RYSUNEK 10.2: Zdjęcie przedstawiające pracę układu w trybie 8-bitowym, na wyświetlaczu 16x2

Przedstawiony kod realizuje 8-bitową transmisję danych. W jego strukturze można znaleźć funkcje realizujące sterowanie wyświetlaczem takie jak `lcd_enable()` do wysyłania impulsu informującego o gotowości wystawionych danych. Kod `lcd_send()` to funkcja, która ustawia pin RS, w zależności od wysyłania komendy lub danych, w dalszej kolejności wystawia 8 bitów na piny D7-D0 wyświetlacza. Funkcje `lcd_command()` oraz `lcd_data()` to funkcje pomocnicze rozgraniczające wysyłanie komend od danych. W `lcd_init()` odbywa się inicjalizacja wyświetlacza w trybie 8-bitowym zgodnie z dokumentacją. W linii 68 kodu znajduje się komenda ustawiająca tryb 8-bitowy transmisji oraz włączona obsługa 2 linii dla wyświetlacza 16x2 oraz wielkość czcionki jako 5x8 pikseli. W kolejnej linii jest włączenie wyświetlacza, wyłączenie widoczności kursora. Następna linia czyści wyświetlacz, a po 2 ms przerwie (czas na dokończenie czyszczenia) włączone jest przesuwanie kursora w prawo. Funkcja `lcd_set_cursor()` ustawia kursor według podanych koordynat - wiersz, kolumna. Kod `lcd_print` wysyła ciąg znaków na wyświetlacz. Funkcja `timer1_init()` włącza układ czasowy, wyzwalający przerwanie co 1 sekundę. W linii 117 jest obsługa wektora przerwania - inkrementacja zmiennej `seconds`. W funkcji głównej zostało zrealizowane wyzwolenie funkcji inicjalizujących ustawienie kursora na pozycję początkową, wyświetlenie napisu "Hello World" oraz w kolejnej linii cykliczne zliczanie pracy układu w sekundach. W kolejnych podrozdziałach zostały wymienione jedynie kluczowe różnice dla innych implementacji oraz trybów transmisji danych.

```

1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3  #include <util/delay.h>
4  #include <stdio.h>
5
6  //definicje pinow
7  #define RS PB5 //pin13
8  #define EN PB4 //pin12
9
10 #define D0 PD0 //pin0
11 #define D1 PD1
12 #define D2 PD2
13 #define D3 PD3
14 #define D4 PD4
15 #define D5 PD5
16 #define D6 PD6
17 #define D7 PD7 //pin7
18
19 uint16_t seconds = 0; //Zmienna globalna, licznik sekund
20
21 //Funkcja do wysylania impulsu na pin EN
22 void lcd_enable(){
23     PORTB |= (1<<EN);
24     _delay_us(1);
25     PORTB &= ~(1<<EN);
26     _delay_us(1);
27 }
28
29 //Funkcja do wysylania danych lub komend
30 void lcd_send(uint8_t data, uint8_t is_data){
31     if (is_data) PORTB |= (1<<RS);
32     else PORTB &= ~(1<<RS);
33
34     //wysylanie 8 bitow
35     PORTD = data;
36     lcd_enable();
37     _delay_us(50);
38 }
39
40 //Funkcja do wysylania komend
41 void lcd_command(uint8_t cmd){ //wysyłanie komendy
42     lcd_send(cmd, 0);
43 }
44
45 //Funkcja do wysylania znaku
46 void lcd_data(uint8_t data){ //wysyłanie znaku
47     lcd_send(data, 1);
48 }
49
50 //Funkcja inicjalizująca wyświetlacz
51 void lcd_init(){
52     //ustawienie pinow jako wyjscie
53     DDRB |= (1<<RS)|(1<<EN);
54     DDRD |= (1<<D0)|(1<<D1)|(1<<D2)|(1<<D3)|(1<<D4)|(1<<D5)|(1<<D6)|(1<<D7);
55     _delay_ms(40);
56     //tryb 8-bitowy
57     PORTD = 0b0000011 << 4;
58     lcd_enable();
59     _delay_ms(5);
60
61     lcd_enable();
62     _delay_us(100);
63
64     lcd_enable();
65     _delay_ms(1);
66
67     //ustawienia wyświetlacza
68     lcd_command(0b0011000); //8-bitowy tryb, 2 linie, 5x8 pikseli
69     lcd_command(0b0000100); // Włącz wyświetlacz, wyłącz kursor
70     lcd_command(0b0000001); // Wyczyść wyświetlacz
71     _delay_ms(2);
72     lcd_command(0b0000110); // Przesuwanie kursora w prawo
73 }
74
75 //Funkcja ustawiająca kursor na zadanej pozycji
76 void lcd_set_cursor(uint8_t row, uint8_t col){
77     uint8_t pos = 0;
78     if(row==0) pos = col;
79     else pos = 0b1000000 + col;
80     lcd_command(0b1000000 | pos);
81 }
82

```

RYSUNEK 10.3: Kod programu obsługujący tryb 8-bitowej transmisji danych do modułu wyświetlacza. Część I

```

83 //Funkcja wysylajaca tekst na wyswietlacz
84 void lcd_print(const char *str) {
85     while (*str) {
86         lcd_data(*str++);
87     }
88 }
89
90 //Funkcja inicjalizujaca Timer1
91 void timer1_init(){
92     TCCR1B |= (1<<WGM12)|(1<<CS12)|(1<<CS10); //tryb CTC, preskaler 1024
93     OCR1A = 15624; //Dla 16 MHz, 1 sekunda
94     TIMSK1 |= (1<<OCIE1A);
95     sei();
96 }
97
98 int main(void){
99     lcd_init();
100     timer1_init();
101
102     lcd_set_cursor(0, 0);
103     lcd_print("Hello World");
104
105     char buffer[16];
106     while (1){
107         lcd_set_cursor(1,0);
108         sprintf(buffer, "%u", seconds);
109         lcd_print(buffer);
110         _delay_ms(200);
111     }
112
113     return 0;
114 }
115
116 //przerwanie Timer1 do zwikszania liczby sekund
117 ISR(TIMER1_COMPA_vect){
118     seconds++;
119 }
120

```

RYSUNEK 10.4: Kod programu obsługujący tryb 8-bitowej transmisji danych do modułu wyświetlacza. Część II

Wymiana wyświetlacza na większy wymagała rozszerzenia funkcji *lcd_set_cursor*. Dodano 2 kolejne wiersze o maskach bitowych 0x14 oraz 0x54. Rozbudowano również funkcję *main*, tak aby wypełnić całe dostępne miejsce na wyświetlaczu.

```

76 //zmodyfikowana funkcja dla wyswietlacza 20x4
77 void lcd_set_cursor(uint8_t row, uint8_t col){
78     uint8_t row_offsets[] = {0x00, 0x40, 0x14, 0x54};
79     lcd_command(0x80 | (row_offsets[row] + col));
80 }

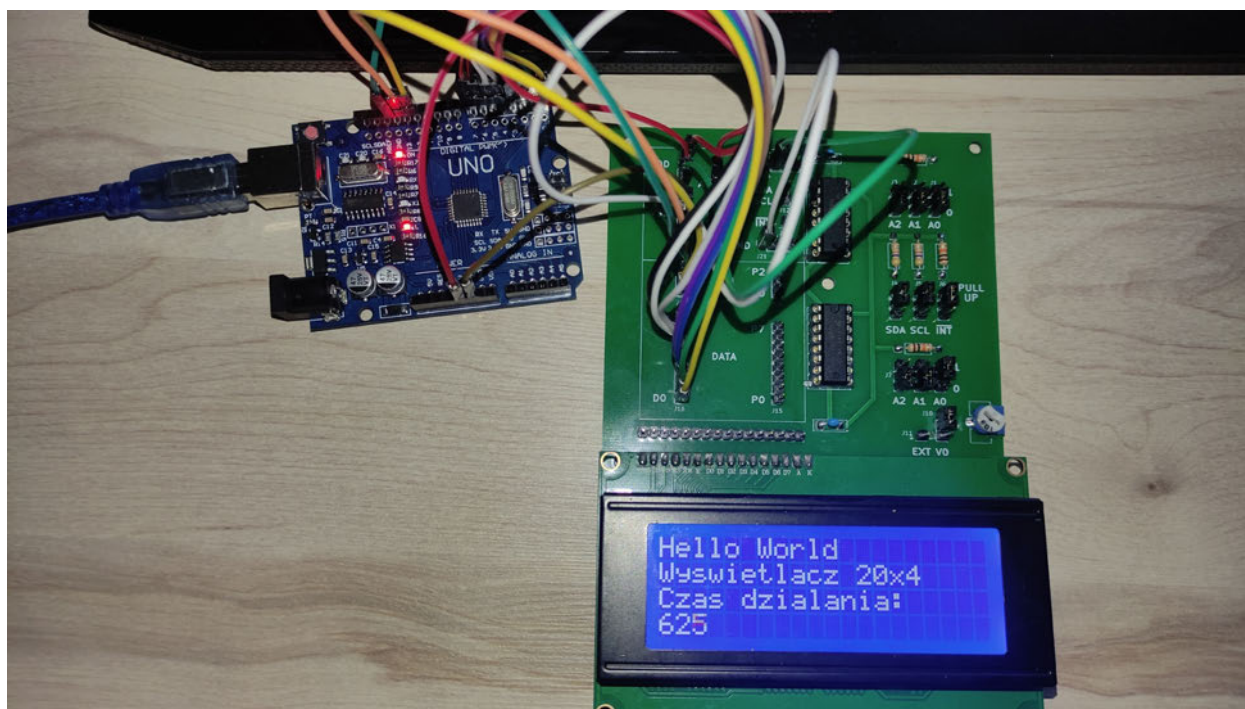
```

RYSUNEK 10.5: Zmodyfikowana funkcja *lcd_set_cursor* dla wyświetlacza 20x4

```
97  int main(){
98      lcd_init();
99      timer1_init();
100
101      lcd_set_cursor(0, 0);
102      lcd_print("Hello World");
103      lcd_set_cursor(1, 0);
104      lcd_print("Wyswietlacz 20x4");
105      lcd_set_cursor(2, 0);
106      lcd_print("Czas dzialania:");
107
108      char buffer[16];
109
110      while (1){
111          lcd_set_cursor(3,0);
112          sprintf(buffer, "%u", seconds);
113          lcd_print(buffer);
114          _delay_ms(200);
115      }
116      return 0;
117 }
```

RYSUNEK 10.6: Zmodyfikowana funkcja główna dla wyświetlacza 20x4

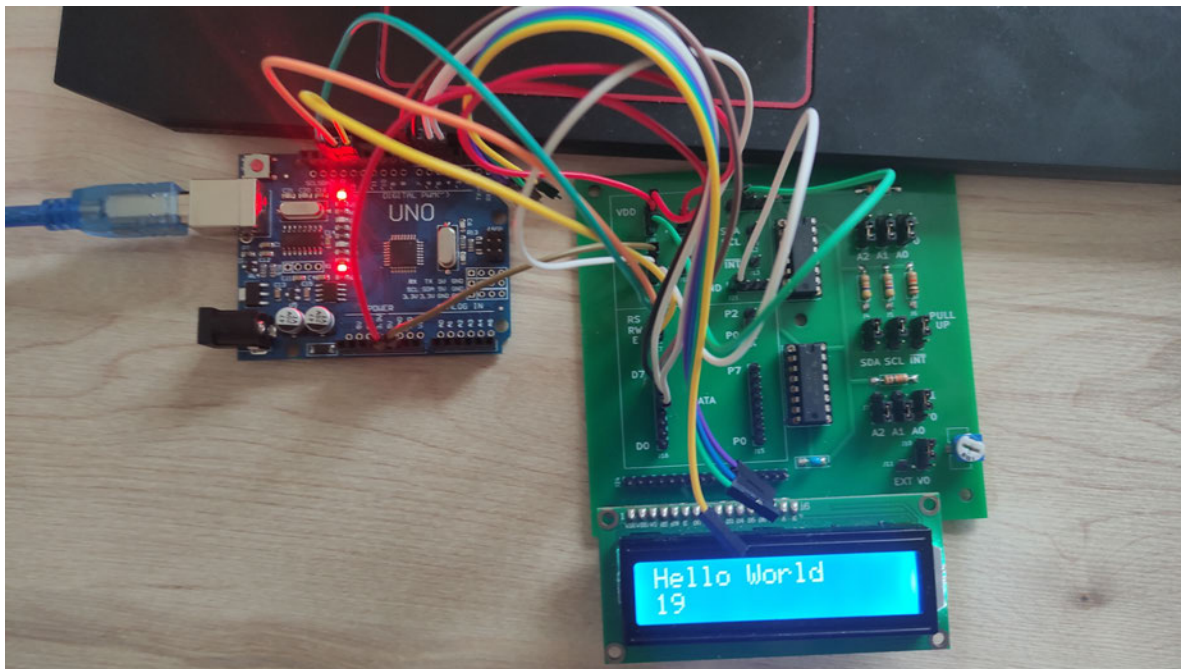
Wynik działania kodu jest widoczny na rysunku 10.7.



RYSUNEK 10.7: Zdjęcie przedstawiające pracę układu w trybie 8-bitowym, na wyświetlaczu 20x4

10.3 Testy modułu z 4-bitową transmisją równoległą danych

Sterownik HD44780 oprócz pracy 8-bitowej oferował możliwość dwukrotnego zredukowania liczby połączeń linii danych. Kluczowe funkcje różniące się od implementacji 8-bitowej zostały przedstawione na rysunku 10.9.



RYSUNEK 10.8: Zdjęcie przedstawiające pracę układu w trybie 4-bitowym

Dla implementacji 4-bitowej transmisji danych poddano ewolucji funkcję *lcd_send*. W zmodyfikowanej funkcji wymiany danych wysyłano starszy półbajt na piny P7 - P4. Po przesłaniu połowy danych czyszczono stan wyjść i sumowano z przesuniętymi o 4 pozycje najmłodszymi bitami. Takim sposobem realizowano komunikację 8-bitową przy ograniczonej liczbie połączeń.

```

23 void lcd_send(uint8_t data, uint8_t is_data){
24     if(is_data) PORTB |= (1<<RS);
25     else PORTB &= ~(1<<RS);
26
27     PORTD = (PORTD&0b00001111)|(data&0b11110000);
28     lcd_enable();
29
30     PORTD = (PORTD&0b00001111)|((data&0b00001111) << 4);
31     lcd_enable();
32
33     _delay_us(50);
34 }

```

RYSUNEK 10.9: Funkcja *lcd_send* dla implementacji 4-bitowej

Inicjalizacja wyświetlacza również musiała przejść proces renowacji. W linii 46 ustawiono tylko 4 najstarsze bity jako wyjścia mikrokontrolera do przesyłu danych. Od tej pory każda instrukcja musiała zostać przesunięta o 4 miejsca w lewo. Po procedurze resetu (linie 49 - 57) następowała konfiguracja sterownika graficznego. Moment nastawy trybu 4-bitowej wymiany danych znajdował się w linii nr. 59. Wraz z wysłaniem komendy i zatwierdzeniem jej przez sterownik układ przyjmował dane w sposób, który przedstawiono w poprzednim fragmencie kodu.

```

44 void lcd_init(){
45     DDRB |= (1<<RS)|(1<<EN);
46     DDRD |= (1<<D4)|(1<<D5)|(1<<D6)|(1<<D7);
47     _delay_ms(40);
48
49     PORTD = 0b00000011 << 4;
50     lcd_enable();
51     _delay_ms(5);
52
53     lcd_enable();
54     _delay_us(150);
55
56     lcd_enable();
57     _delay_ms(1);
58
59     PORTD = 0b00000010 << 4;
60     lcd_enable();
61     _delay_ms(1);
62     lcd_command(0b00101000);
63     lcd_command(0b00001100);
64     lcd_command(0b00000001);
65     _delay_ms(2);
66     lcd_command(0b00000110);
67 }

```

RYSUNEK 10.10: Funkcja *lcd_init* dla implementacji 4-bitowej

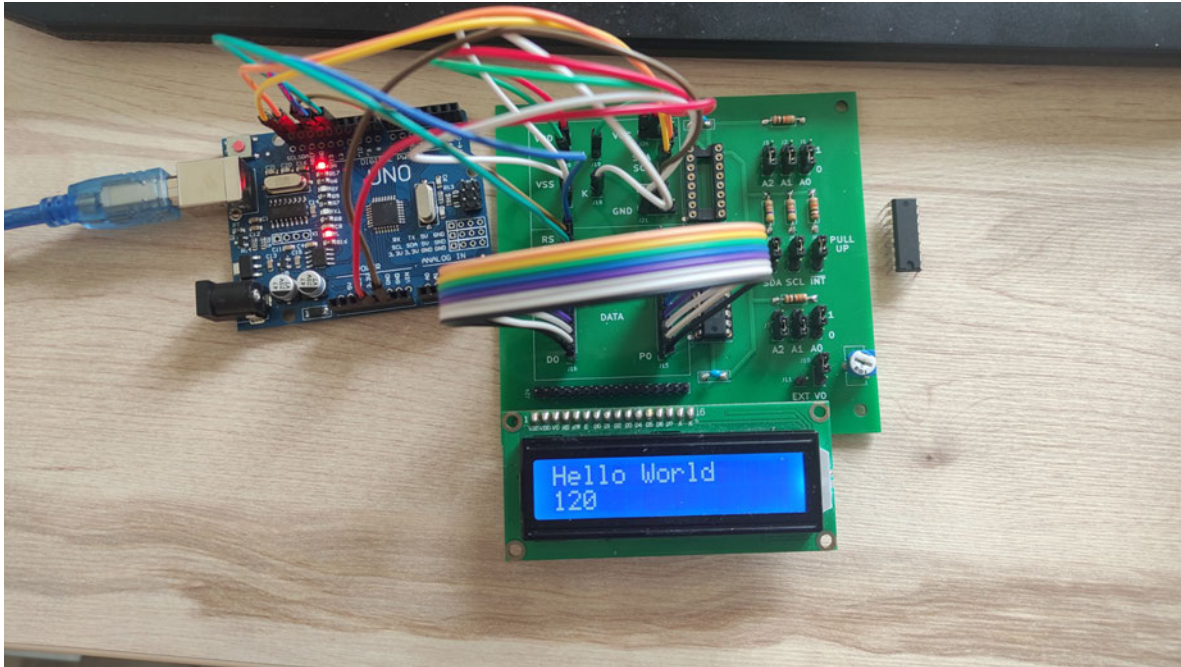
Badania przeprowadzono również dla wyświetlacza 20x4. Na rysunku 10.11 przedstawiono działanie całego systemu.



RYSUNEK 10.11: Zdjęcie przedstawiające pracę układu w trybie 4-bitowym, na wyświetlaczu 20x4

10.4 Testy modułu z transmisją szeregową danych

Pozytywny rezultat równoległej wymiany danych poprowadził do weryfikacji działania ekspanderów w szeregowej transmisji danych. Podczas tego badania magistrala I²C została wykorzystana tylko do przesyłu danych, natomiast sygnały sterujące transmitowano równolegle.



RYSUNEK 10.12: Zdjęcie przedstawiające pracę układu z jednym PCF8574 i wyświetlaczem 16x2



RYSUNEK 10.13: Zdjęcie przedstawiające pracę układu z jednym PCF8574 i wyświetlaczem 20x4

Prezentowany na rysunku 10.14 program implementuje sposób sterowania wyświetlaczem LCD. Rozwiązanie to łączy szeregową transmisję danych za pośrednictwem zewnętrznego ekspandera z bezpośrednim równoległym sterowaniem liniami kontrolnymi z poziomu mikrokontrolera AT-Mega328p. Zastosowanie takiego podejścia stanowi kompromis, pozwala na zaoszczędzenie zasobów sprzętowych (pinów GPIO) procesora nadrzędnego odpowiedzialnych za 8-bitową transmisję danych, przy jednoczesnym zachowaniu maksymalnej szybkości i precyzji sygnałów sterujących (RS, RW, E). Komunikacja z omawianym ekspanderem odbywa się za pomocą I²C. Transmisja danych opiera się na sekwencyjnym wywołaniu zestawu niskopoziomowych funkcji sterujących magistralą, do których należą *I2C_Init()*, *I2C_Start()*, *I2C_Write()*, *I2C_Stop()*. Sercem modułu odpowiedzialnego za translację poleceń systemowych na stany fizyczne ekspandera jest funkcja *PCF8574_Write()*. Przyjmuje ona jako argument jeden 8-bitowy bajt danych, który reprezentuje docelowy stan logiczny wszystkich ośmiu wyprowadzeń układu PCF8574. Wewnątrz tej funkcji realizowany jest kompletny cykl transmisyjny I²C, czyli wysłanie zdefiniowanego adresu sprzętowego ekspandera, a następnie przesłanie bajtu danych, który bezpośrednio przepisuje stany na piny wyjściowe układu scalonego.

Podczas gdy szyna danych (D0-D7) sterownika wyświetlacza połączona jest z wyjściami ekspandera PCF8574, sygnały sterujące RS, RW, E są obsługiwane asynchronicznie i bezpośrednio przez PORT B mikrokontrolera AT-Mega328p. Operacje te są realizowane w liniach 73 oraz 74. Wykorzystanie bitowych operatorów sumy logicznej oraz iloczynu logicznego z negacją pozwala na bezwzględną modyfikację stanów wybranych pinów bez naruszania konfiguracji pozostałych wyprowadzeń portu. Bezpośrednie połączenie linii E do pinu mikrokontrolera ma fundamentalne znaczenie dla stabilności działania systemu. Sterownik HD44780 wymaga, aby impuls Enable miał ściśle określony czas trwania oraz odpowiednio strome zbocza. Generowanie tego impulsu poprzez programowe opóźnienia *_delay_us()* bezpośrednio na pinie mikrokontrolera pozwala uniknąć narzutów czasowych magistrali I²C, które wystąpiłoby, gdyby sygnał E również był generowany przez ekspander. Zapobiega to powstawaniu błędów transmisji i gwarantuje poprawną interpretację instrukcji przez wewnętrzną maszynę stanów wyświetlacza.

Pozostała część struktury oprogramowania jest zgodna z klasycznym algorytmem transmisji w pełni równoległej, co ułatwia migrację kodu między różnymi platformami. W kodzie istnieją również zakomentowane funkcje sterujące wyświetlaczem 20x4, zlokalizowane w liniach 113-120 oraz 140-145. Zostały one wykorzystane przy testach większego wyświetlacza, którego wynik działania został zaprezentowany na rysunku 10.13. Wprowadzenie tych zmian jest podyktowane odmienną organizacją pamięci DDRAM (ang. *Display Data RAM*) w układach wielowierszowych. Podczas gdy standardowy wyświetlacz 16x2 adresuje wiersze w sposób ciągły (począwszy od adresów 0x00 dla linii pierwszej oraz 0x40 dla linii drugiej), sterownik formatu 20x4 dokonuje logicznego przesunięcia pamięci. Wiersz trzeci stanowi fizyczne przedłużenie wiersza pierwszego (adres początkowy 0x14), natomiast wiersz czwarty kontynuuje przestrzeń adresową wiersza drugiego (adres początkowy 0x54) [18].

```

1  #include <avr/io.h>
2  #include <avr/interrupt.h>
3  #include <util/delay.h>
4  #include <stdio.h>
5
6  //Adres I2C ekspandera PCF8574 (0x21 oznacza, że piny adresowe A0-A2 są odpowiednio ustawione)
7  #define PCF8574_ADDRESS 0x21
8
9  //Piny sterujące LCD podłączone bezpośrednio do mikrokontrolera (PORTB)
10 #define RS PB5 //Pin 13 w Arduino Uno (Register Select: 0 = komenda, 1 = dane)
11 #define EN PB4 //Pin 12 w Arduino Uno (Enable: aktywacja zapisu danych)
12
13 //Mapowanie wyjść ekspandera PCF8574 na piny danych wyświetlacza LCD (tryb 8-bitowy)
14 #define D0 0 //P0 ekspandera -> D0 wyświetlacza
15 #define D1 1
16 #define D2 2
17 #define D3 3
18 #define D4 4
19 #define D5 5
20 #define D6 6
21 #define D7 7 //P7 ekspandera -> D7 wyświetlacza
22
23 uint8_t to_send_B = 0; //Zmienna pomocnicza do przechowywania stanu danych
24
25 //Licznik sekund - dopisane 'volatile', ponieważ zmienna zmienia się w przerwaniu
26 volatile uint16_t seconds = 0;
27
28 //Inicjalizacja sprzętowego modułu I2C (TWI) w AVR
29 void I2C_Init() {
30     TWSR = 0x00; //Brak preskalera (częstotliwość dzielona przez 1)
31     TWBR = 0x48; //Ustawienie prędkości SCL na ok. 100 kHz przy zegarze 16 MHz
32     TWCR = (1 << TWEN); //Włączenie modułu TWI
33 }
34
35 //Generowanie sygnału START na magistrali I2C
36 void I2C_Start() {
37     TWCR = (1 << TWSTA) | (1 << TWEN) | (1 << TWINT);
38     while (!(TWCR & (1 << TWINT))); //Oczekiwanie na ustawienie flagi TWINT (koniec operacji)
39 }
40
41 //Generowanie sygnału STOP na magistrali I2C
42 void I2C_Stop() {
43     TWCR = (1 << TWSTO) | (1 << TWEN) | (1 << TWINT);
44 }
45
46 //Transmisja jednego bajtu danych przez I2C
47 void I2C_Write(uint8_t data) {
48     TWDR = data; //Wpisanie danych do rejestru przesuwanego TWI
49     TWCR = (1 << TWEN) | (1 << TWINT); //Rozpoczęcie wysyłania
50     while (!(TWCR & (1 << TWINT))); //Oczekiwanie na zakończenie nadawania
51 }
52
53 //Wysłanie pełnego bajtu danych bezpośrednio do portów wyjściowych ekspandera PCF8574
54 void PCF8574_Write(uint8_t data) {
55     I2C_Start();
56     I2C_Write(PCF8574_ADDRESS << 1); //Adresowanie układu w trybie zapisu (bit R/W = 0)
57     I2C_Write(data); //Wystawienie danych na piny P0-P7 ekspandera
58     I2C_Stop();
59 }
60
61 //Odczyt danych z ekspandera PCF8574
62 uint8_t PCF8574_Read() {
63     uint8_t data;
64     I2C_Start();
65     I2C_Write((PCF8574_ADDRESS << 1) | 1); //Adresowanie układu w trybie odczytu (bit R/W = 1)
66     TWCR = (1 << TWEN) | (1 << TWINT); //Odbiór bajtu z odpowiedzią NACK
67     while (!(TWCR & (1 << TWINT)));
68     data = TWDR; //Pobranie odczytanej wartości
69     I2C_Stop();
70     return data;
71 }
72
73 //Wygenerowanie impulsu na pinie EN (mikrokontrolera), który zatwierdza dane w LCD
74 void lcd_enable(){
75     PORTB |= (1<<EN); //Stan wysoki na EN
76     _delay_us(1); //Krótkie podtrzymanie stanu high
77     PORTB &= ~(1<<EN); //Opadające zbocze na EN - LCD w tym momencie zatrząskuje bity danych
78     _delay_us(1); //Czas na ustabilizowanie
79 }
80

```

RYSUNEK 10.14: Kod programu obsługujący transmisję 8 bitową danych jednym PCF8574. Część I

```

81 //Główna funkcja wysyłająca dane/komendy w trybie 8-bitowym
82 void lcd_send(uint8_t data, uint8_t is_data){
83     //Ustawienie pinu RS na mikrokontrolerze w zależności od rodzaju przesyłu
84     if (is_data) PORTB |= (1<<RS); //Dane (np. znak do wyświetlenia)
85     else PORTB &= ~(1<<RS); //Instrukcja/Komenda
86
87     PCF8574_Write(data); //Wysłanie całego bajtu danych na piny D0-D7 przez ekspander I2C
88     lcd_enable(); //Wygenerowanie impulsu zegarowego dla LCD
89
90     _delay_ms(2); //Bezpieczny czas na wykonanie operacji przez kontroler HD44780
91 }
92
93 //Wysłanie instrukcji sterującej do LCD
94 void lcd_command(uint8_t cmd){
95     lcd_send(cmd, 0); //is_data = 0
96 }
97
98 //Wysłanie pojedynczego znaku na ekran
99 void lcd_data(uint8_t data){
100     lcd_send(data, 1); //is_data = 1
101 }
102
103 //Inicjalizacja wyświetlacza LCD w trybie 8-bitowym
104 void lcd_init(){
105     DDRB |= (1<<RS) | (1<<EN); //Ustawienie pinów RS i EN mikrokontrolera jako wyjścia
106     _delay_ms(40); //Oczekiwanie na ustabilizowanie się napięcia zasilania LCD
107
108     //Sekwencja restartu kontrolera LCD (zgodna ze specyfikacją HD44780)
109     lcd_command(0b00110000);
110     _delay_ms(3);
111     lcd_command(0b00110000);
112     lcd_command(0b00110000);
113
114     //Właściwa konfiguracja:
115     lcd_command(0b00111000); //Tryb 8-bitowy, 2 linie wyświetlacza, czcionka 5x8 punktów
116     lcd_command(0b00001100); //Włączenie ekranu, wyłączenie kursora oraz jego migania
117     lcd_command(0b00000001); //Czyszczenie zawartości ekranu (Clear Display)
118     _delay_ms(2); //Czyszczenie wymaga dłuższego czasu wykonania
119     lcd_command(0b00000110); //Tryb pracy kursora - przesuwanie w prawo po zapisie znaku
120 }
121
122 //Ustawienie pozycji kursora dla standardowego wyświetlacza 16x2
123 void lcd_set_cursor(uint8_t row, uint8_t col){
124     uint8_t pos = 0;
125     if(row == 0) pos = col; // Pierwszy wiersz (adres DDRAM zaczyna się od 0x00)
126     else pos = 0b01000000 + col; // Drugi wiersz (adres DDRAM zaczyna się od 0x40)
127     lcd_command(0b10000000 | pos); // Wysłanie komendy "Set DDRAM Address"
128 }
129
130 // Zmodyfikowana funkcja ustawiania kursora dla dużego wyświetlacza 20x4 (zakomentowana)
131 /*void lcd_set_cursor(uint8_t row, uint8_t col){
132     // Mapowanie początkowych adresów pamięci DDRAM dla każdego z 4 wierszy
133     uint8_t row_offsets[] = {0x00, 0x40, 0x14, 0x54};
134     if (row >= 4) {
135         row = 0; // Zabezpieczenie przed wyjściem poza zakres wierszy
136     }
137     lcd_command(0x80 | (row_offsets[row] + col));
138 }*/
139
140 //Wyświetlenie całego ciągu tekstowego na ekranie
141 void lcd_print(const char *str) {
142     while (*str) {
143         lcd_data(*str++); //Wysyłanie kolejnych znaków w pętli do momentu znaku końca '\0'
144     }
145 }
146
147 //Konfiguracja Timera 1 do generowania przerwań co 1 sekundę
148 void timer1_init(){
149     //Tryb CTC (Clear Timer on Compare) oraz ustawienie preskalera na 1024
150     TCCR1B |= (1<<WGM12) | (1<<CS12) | (1<<CS10);
151
152     //Wartość TOP dla częstotliwości 1 Hz przy zegarze 16 MHz: (16000000 / (1024 * 1)) - 1
153     OCR1A = 15624;
154
155     TIMSK1 |= (1<<OCIE1A); //Włączenie przerwania Compare Match A dla Timera 1
156     sei(); //Globalne odblokowanie przerwań
157 }
158
159 int main() {
160     I2C_Init(); //Start sprzętowego I2C

```

RYSUNEK 10.15: Kod programu obsługujący transmisję 8 bitową danych jednym PCF8574. Część II

```

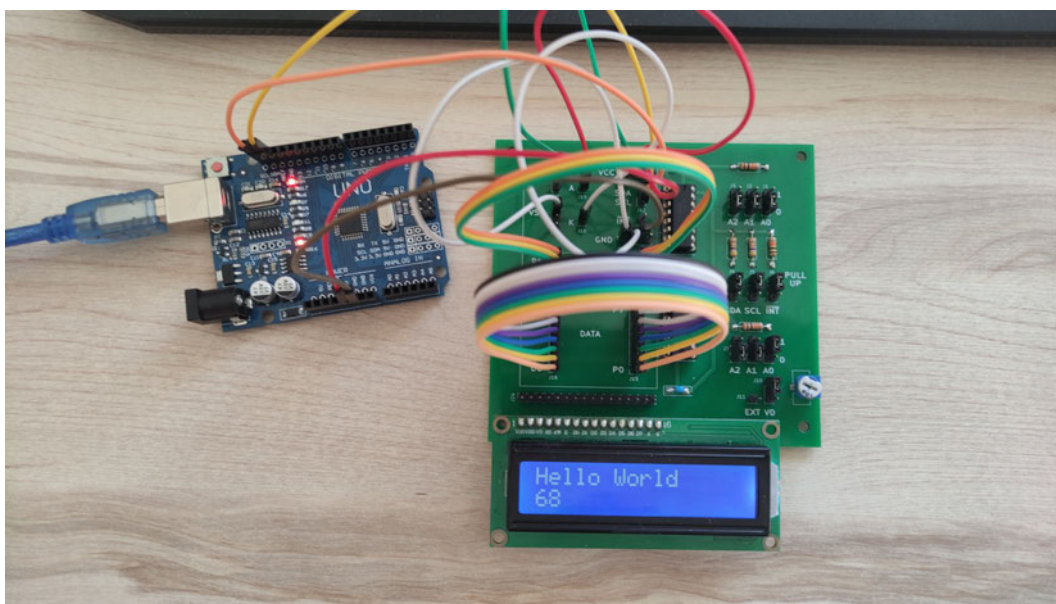
161  lcd_init(); //Konfiguracja ekranu LCD
162  timer1_init(); //Uruchomienie licznika sekund
163
164  /* Sekcja testowa dla ekranu 20x4 (zakomentowana)
165  lcd_set_cursor(0, 0);
166  lcd_print("Hello World");
167  lcd_set_cursor(1, 0);
168  lcd_print("Wyswietlacz 20x4");
169  lcd_set_cursor(2, 0);
170  lcd_print("Czas dzialania:"); */
171
172  lcd_set_cursor(0, 0);
173  lcd_print("Hello World"); //Wyswietlenie tekstu w pierwszym wierszu
174
175  char buffer[16]; //Bufor tekstowy do przechowywania sformatowanej liczby sekund
176
177  while (1){
178      lcd_set_cursor(1, 0); //Ustawienie kursora na początku drugiego wiersza
179      sprintf(buffer, "%u", seconds); //Konwersja zmiennej liczbowej 'seconds' na tekst
180      lcd_print(buffer); //Wypisanie aktualnego stanu licznika na LCD
181      _delay_ms(200); //Krótkie opóźnienie pętli głównej dla stabilności odświeżania
182  }
183
184  return 0;
185 }
186
187 //Wektor obsługi przerwania od Timera 1 - wywoływany dokładnie co sekundę
188 ISR(TIMER1_COMPA_vect){
189     seconds++; //Inkrementacja licznika sekund
190 }
191

```

RYSUNEK 10.16: Kod programu obsługujący transmisję 8 bitową danych jednym PCF8574. Część III

10.5 Testy modułu z transmisją szeregową danych i sygnałów sterujących

Ostatecznym testem uruchomieniowym dla zaprojektowanego modułu był przesył szeregowy zarówno danych, jak i sygnałów sterowniczych. Było to badanie z minimalną liczbą połączeń komunikacyjnych (standard I²C). Na rysunku 10.17 oraz rysunku 10.18 przedstawiono wyniki działania układu z dwoma rodzajami wyświetlaczy.



RYSUNEK 10.17: Zdjęcie przedstawiające pracę modułu z dwoma układami PCF8574



RYSUNEK 10.18: Zdjęcie przedstawiające współpracę modułu z dwoma układami PCF8574 z wyświetlaczem 20x4

Rysunek 10.19 przedstawia dwie funkcje realizujące komunikację z układami PCF8574. Wykorzystywane są funkcje zapisu *PCF8574_Write_A* oraz *PCF8574_Write_B*. Dane podobnie jak w poprzednim przypadku są wysyłane w 8-bitowych ramkach.

```

49 void PCF8574_Write_A(uint8_t data) {
50     I2C_Start();
51     I2C_Write(PCF8574_ADDRESS_A << 1);
52     I2C_Write(data);
53     I2C_Stop();
54 }
55 }
56
57 void PCF8574_Write_B(uint8_t data) {
58     I2C_Start();
59     I2C_Write(PCF8574_ADDRESS_B << 1);
60     I2C_Write(data);
61     I2C_Stop();
62 }
63 }
64
65 uint8_t PCF8574_Read_A() {
66     uint8_t data;
67     I2C_Start();
68     I2C_Write((PCF8574_ADDRESS_A << 1) | 1);
69     TWCR = (1 << TWEN) | (1 << TWINT);
70     while (!(TWCR & (1 << TWINT)));
71     data = TWDR;
72     I2C_Stop();
73     return data;
74 }
75
76 uint8_t PCF8574_Read_B() {
77     uint8_t data;
78     I2C_Start();
79     I2C_Write((PCF8574_ADDRESS_B << 1) | 1);
80     TWCR = (1 << TWEN) | (1 << TWINT);
81     while (!(TWCR & (1 << TWINT)));
82     data = TWDR;
83     I2C_Stop();
84     return data;
85 }

```

RYSUNEK 10.19: Funkcje odczytujące i zapisujące dane do dwóch układów PCF8574

Na obrazku 10.20 widoczne są zmodyfikowane funkcje *lcd_enable* oraz *lcd_send*. Sygnały sterujące nie są już wysyłane równolegle wprost z pinów mikrokontrolera, ale przekazywane za pośrednictwem dodatkowego układu PCF8574.

```
87 void lcd_enable(){
88     to_send_A |= (1<<EN);
89     PCF8574_Write_A(to_send_A);
90     _delay_us(1);
91     to_send_A &= ~(1<<EN);
92     PCF8574_Write_A(to_send_A);
93     _delay_us(50);
94 }
95
96 void lcd_send(uint8_t data, uint8_t is_data){
97     if(is_data) to_send_A |= (1<<RS);
98     else to_send_A &= ~(1<<RS);
99     PCF8574_Write_A(to_send_A);
100
101     PCF8574_Write_B(data);
102     lcd_enable();
103
104     _delay_ms(2);
105 }
```

RYSUNEK 10.20: Funkcje enable i send dla wyświetlacza sterowanego układami PCF8574

Rozdział 11

Uruchamianie modułu LCD z 16-bitowymi ekspanderami MCP23x17

11.1 Środowisko pomiarowe

Testy uruchomieniowe wykonano z użyciem zaprojektowanego modułu z dwoma ekspanderami z rodziny MCP23x17. Do komunikacji wykorzystano moduł Arduino Uno, natomiast dane wyświetlano na dwóch rodzajach ekranów LCD.



RYSUNEK 11.1: Schemat funkcjonalny systemu do testów modułu z układami MCP23x17

11.2 Testy modułu z 8-bitową transmisją równoległą danych

W badaniach zapewniono, możliwość bezpośredniego transmitowania danych z mikrokontrolera na wyświetlacz, z użyciem magistrali równoległej wpiętej do zaprojektowanego złącza danych przy ekranie LCD. Kod obsługujący równoległą wymianę danych z testów uruchomieniowych modułu z ekspanderami PCF8547 działał również dla tego projektu. Złącza danych w obu urządzeniach zostały stworzone podobnie, przez co nie powstały zmiany w implementacji komunikacji.

11.3 Testy modułu z 4-bitową transmisją równoległą danych

Po sprawdzeniu 8-bitowej równoległej transmisji danych naturalnym krokiem było zbadanie komunikacji z okrojoną liczbą bitów. Tak samo jak w poprzednio opisanym badaniu kod dla modułu z PCF8574 pasował do implementacji na płytce z ekspanderami MCP23x17.

11.4 Testy modułu z transmisją danych przez łącze I²C

Zgodnie z koncepcją modułu, w projekcie zastosowano magistralę szeregową I²C. Badania uruchomieniowe przeprowadzono w ten sam sposób, jak w przypadku testów płytki z PCF8574 tzn. najpierw sprawdzono sam przesył danych magistralą. Kod przeprowadzający transmisję był kopią programu do obsługi modułu z działu czwartego, wzbogaconą o obsługę rejestrów MCP23017. Mapę rejestrów urządzenia przedstawiono na rysunku 11.2.

```
16 //Definicje Rejestrów MCP23017 (BANK=0)
17 #define MCP0_IODIRA 0x00
18 #define MCP0_IODIRB 0x01
19 #define MCP0_GPIOA 0x12
20 #define MCP0_GPIOB 0x13
21 #define MCP0_OLATA 0x14
22 #define MCP0_OLATB 0x15
```

RYSUNEK 11.2: Mapa rejestrów ekspandera MCP23017

Sygnały sterujące wyświetlacz LCD były wysyłane poprzez odpowiednie ustawienia portów mikrokontrolera w Arduino Uno. Program zawierający funkcjonalności magistrali I²C, którą przesyłano dane zawarto na rysunku 11.3 oraz 11.4.

```

53 void I2C_Init() {
54     TWSR = 0x00;
55     TWBR = 0x19; //Prędkość transmisji
56     TWCR = (1 << TWEN);
57 }
58
59 void I2C_Start() {
60     TWCR = (1 << TWSTA) | (1 << TWEN) | (1 << TWINT);
61     while (!(TWCR & (1 << TWINT)));
62 }
63
64 void I2C_Stop() {
65     TWCR = (1 << TWSTO) | (1 << TWEN) | (1 << TWINT);
66
67     //ta poprawka naprawiła problem komunikacji z MCP, przy większych prędkościach znak start nadpisywał STOP i układ sie gubił
68     while(TWCR & (1 << TWSTO));
69     _delay_us(10);
70 }
71
72 void I2C_Write(uint8_t data) {
73     TWDR = data;
74     TWCR = (1 << TWEN) | (1 << TWINT);
75     while (!(TWCR & (1 << TWINT)));
76 }

```

RYSUNEK 11.3: Obsługa magistrali I²C. Część I

```

77
78 // --- Obsługa MCP23017 ---
79 void MCP0_Write(uint8_t reg, uint8_t data) {
80     I2C_Start();
81     I2C_Write(MCP0_OPCODE); //Wysłanie adresu urządzenia (z bitem zapisu)
82     I2C_Write(reg);         //Wysłanie adresu rejestru
83     I2C_Write(data);        //Wysłanie danych
84     I2C_Stop();
85 }
86
87 void MCP0_Init(void) {
88     // 1. Obsługa sprzętowa pinu RESET
89     DDRB |= (1 << MCP0_RESET); // Ustaw pin jako wyjście
90     PORTB &= ~(1 << MCP0_RESET); // TWARDE ZEROWANIE (Reset układu)
91     _delay_ms(20);
92     PORTB |= (1 << MCP0_RESET); // Wznowienie pracy (Stan wysoki)
93     _delay_ms(50);             // Czekamy na stabilizację napięć
94
95     // 2. Bezpośrednie (niesekwencyjne) ustawienie wszystkich pinów jako WYJŚCIA
96     MCP0_Write(MCP0_IODIRA, 0x00); // Port A w całości jako wyjścia
97     MCP0_Write(MCP0_IODIRB, 0x00); // Port B w całości jako wyjścia
98
99     // 3. Stan niski na start
100     MCP0_Write(MCP0_GPIOA, 0x00);
101     MCP0_Write(MCP0_GPIOB, 0x00);
102 }

```

RYSUNEK 11.4: Obsługa magistrali I²C. Część II

Moduł był przygotowany do współpracy z wyświetlaczami 2x16 oraz 4x20. Zasada działania sterowania ekranami pozostała identyczna jak w poprzednich programach.

11.5 Testy modułu z transmisją danych i sygnałów sterujących przez łącze I2C

Po pozytywnie przeprowadzonej transmisji danych, wykonano próbę kontroli wyświetlacza wykorzystując dodatkowy przesył sygnałów sterowniczych magistralą I²C. Omawiana funkcjonalność została przedstawiona na rysunku 11.5. W programie zdefiniowano indywidualne nazwy takie jak np. *LCD_RS_BIT*, które zawierały w sobie przypisaną pozycję w porcie GPIOB. W funkcjach inicjalizowano zmienną *portB_shadow* pozwalającą na manipulowanie sygnałami sterującymi. Następnie

za pomocą wcześniej omówionych funkcji aktualizowano stan na porcie GPIOB. W kodzie zawarto aktualizację linii sterujących dla interfejsu SPI, który został wykorzystany w dalszych badaniach. Różnicę można zaobserwować w nazwach własnych funkcji tzn. *MCP0_Write* odpowiadała za komunikację magistralą I²C, natomiast *MCPS_Write* kontrolowało interfejs SPI.

```

149 void LCD_ENABLE()
150 {
151     // E High
152     portB_shadow |= LCD_E_BIT;
153     MCP0_Write(MCP0_GPIOB, portB_shadow);
154     MCPS_Write(MCP0_GPIOB, portB_shadow);
155     _delay_us(1);
156
157     // E Low
158     portB_shadow &= ~LCD_E_BIT;
159     MCP0_Write(MCP0_GPIOB, portB_shadow);
160     MCPS_Write(MCP0_GPIOB, portB_shadow);
161     _delay_us(50);
162 }
163
164 void LCD_Send8Bit(uint8_t val, uint8_t is_cmd) {
165     // 1. Ustawienie RS (Port B)
166     if (is_cmd) portB_shadow &= ~LCD_RS_BIT; // RS = 0 (Command)
167     else portB_shadow |= LCD_RS_BIT; // RS = 1 (Data)
168
169     MCP0_Write(MCP0_GPIOB, portB_shadow); // Aktualizuj linie sterujące
170     MCPS_Write(MCP0_GPIOB, portB_shadow); // Aktualizuj linie sterujące
171
172     // 2. Wystawienie Danych (Port A)
173     MCP0_Write(MCP0_GPIOA, val);
174     MCPS_Write(MCP0_GPIOA, val);
175
176     // 3. Zatwierdzenie (Enable Pulse)
177     LCD_ENABLE();
178 }

```

RYSUNEK 11.5: Transmisja sygnałów sterowniczych magistralą I²C oraz SPI

11.6 Testy modułu z transmisją danych przez łącze SPI

Nowością względem poprzednich implementacji było zawarcie inicjalizacji i transmisji z wykorzystaniem znacznie szybszej magistrali SPI. Dla przejrzystości kodu dodano definicje portów interfejsu (rys. 11.6) oraz rejestrów ekspandera (rys. 11.7).

```

7 //Definicje pinów SPI
8 #define MISO PB4
9 #define MOSI PB3
10 #define SCK PB5
11 #define SS PB2

```

RYSUNEK 11.6: Definicje pinów ekspandera MCP23S17

```

24 //Definicje Rejestrów MCP23S17 (BANK=0)
25 #define MCPS_IODIRA 0x00
26 #define MCPS_IODIRB 0x01
27 #define MCPS_GPIOA 0x12
28 #define MCPS_GPIOB 0x13
29 #define MCPS_OLATA 0x14
30 #define MCPS_OLATB 0x15

```

RYSUNEK 11.7: Mapa rejestrów ekspandera MCP23S17

Inicjalizację interfejsu charakteryzowały dwa segmenty - ustawienia pinów i ustalenie prędkości sygnału zegarowego. Zgodnie z normami komunikacji magistrali SPI stworzono funkcje do transmi-

sji danych. Każdą sesję komunikacji rozpoczynało typowe ściągnięcie pinu CS do stanu niskiego oraz przesłanie adresu urządzenia, z którym chciano nawiązać łączność. Wymianę danych z użyciem SPI można porównać do taśmociągu, w którym jednocześnie adresat otrzymuje dane od nadawcy i na odwrót. Nie da się niczego nadać bez przyjęcia zwrotnej wiadomości. W kodzie stworzono funkcję *SPI_Transfer*, która odpowiadała za ten mechanizm. Posługując się rejestrem SPDR jednocześnie wypychano dane do odbiorcy, i zwracano wartość zachowaną w urządzeniu podrzędnym. Takim sposobem uzyskano warstwę fizyczną pod główną funkcję do komunikacji *MCPS_Write*, która służyła do nadpisywania danych rejestrów żądanymi wartościami. Finalną funkcją, kończącą zbiór instrukcji odpowiedzialnych za obsługę magistrali SPI była inicjalizacja ekspandera. Na początku deklarowano kierunek transmisji dla portów urządzenia. W tym celu na odpowiednie rejestry wysyłano wartość 0 zapisaną systemem szesnastkowym. Takim sposobem kończyła się pierwsza część inicjalizacji. Funkcję kończyło nadpisanie rejestrów portów GPIOA i GPIOB zerami, aby uniknąć odczytu nieprawidłowych danych w fazie początkowej. Kod przedstawiono na rysunku 11.8.

```

104 void SPI_INIT()
105 {
106     DDRB |= (1 << MOSI) | (1 << SCK) | (1 << SS) | (1 << MCPS_RESET);
107     DDRB &= ~(1 << MISO);
108     PORTB |= (1 << SS) | (1 << MCP0_RESET);
109
110     // F_CPU / 2 = 8 MHz
111     SPCR = (1 << SPE) | (1 << MSTR); // Brak bitów SPR1 i SPR0
112     SPSR |= (1 << SPI2X); // Włączenie bitu podwójnej prędkości!
113 }
114
115 uint8_t SPI_Transfer(uint8_t data)
116 {
117     SPDR = data; //wysłanie bajtu
118     while (!(SPSR & (1 << SPIF))); //czekanie na zakończenie transmisji
119     return SPDR;
120 }
121
122 // --- Obsługa MCP23S17 ---
123 void MCPS_Write(uint8_t reg, uint8_t data) {
124     PORTB &= ~(1 << SS); // CS Low (Start)
125     _delay_ms(1);
126     SPI_Transfer(MCPS_OPCODE);
127     SPI_Transfer(reg);
128     SPI_Transfer(data);
129     _delay_ms(1);
130     PORTB |= (1 << SS); // CS High (Stop)
131 }
132
133 void MCPS_Init(void) {
134     // 1. Ustawienie IODIRA i IODIRB na 0x00 (Wszystkie piny jako WYJŚCIA)
135     // Dzięki trybowi sekwencyjnemu (domyślnemu), można wysłać to jednym ciągiem!
136
137     PORTB &= ~(1 << SS); // CS Low
138     SPI_Transfer(MCPS_OPCODE);
139     SPI_Transfer(MCPS_IODIRA); // Startujemy od adresu 0x00
140     SPI_Transfer(0x00);        // Wpis do IODIRA (0x00) -> wskaźnik sam skoczy na 0x01
141     SPI_Transfer(0x00);        // Wpis do IODIRB (0x01)
142     PORTB |= (1 << SS); // CS High
143     _delay_ms(10);
144     MCPS_Write(MCPS_GPIOB, 0x00);
145     MCPS_Write(MCPS_GPIOA, 0x00);
146 }

```

RYSUNEK 11.8: Fragment kodu odpowiedzialny za transmisję SPI dla danych

11.7 Testy modułu z transmisją danych i sygnałów sterujących przez łącze SPI

W toku przeprowadzonych badań, równoległe z analizą magistrali I²C, dokonano empirycznej weryfikacji sprzętowego sterowania matrycą LCD z wykorzystaniem szybkiego interfejsu szerego-

wego SPI (ang. *Serial Peripheral Interface*). Mechanizm kontroli sygnałów sterowniczych wyświetlacza z poziomu warstwy logicznej aplikacji pozostał spójny z rozwiązaniem opartym na magistrali I²C (co zilustrowano na rysunku 11.5), jednakże konieczne było całkowite przeprojektowanie warstwy abstrakcji sprzętowej (HAL) odpowiedzialnej za fizyczny transfer danych do ekspandera portów MCP23S17.

Zasadnicza różnica między analizowanymi interfejsami leży w ich topologii oraz sposobie dystrybucji sygnałów. Inicjalizacja interfejsu SPI w mikrokontrolerze nadrzędnym wymagała przeprowadzenia precyzyjnej, dwuetapowej konfiguracji. W pierwszym kroku zdefiniowano kierunki wyprowadzeń sprzętowych modułu komunikacyjnego: linie MOSI (*Master Out Slave In*), SCK (*Serial Clock*) oraz CS (*Chip Select*) skonfigurowano jako wyjścia, natomiast linię MISO (*Master In Slave Out*) jako wejście. W drugim kroku, poprzez modyfikację rejestru kontrolnego SPCR (*SPI Control Register*), ustalono podział częstotliwości zegara systemowego (preskaler), definiując tym samym docelową przepływność magistrali, oraz wybrano odpowiedni tryb pracy SPI (tzw. *SPI Mode* określający polaryzację i fazę zegara, zazwyczaj Mode 0,0).

Zgodnie z rygorystycznymi normami synchronicznej komunikacji szeregowej, każdą sesję komunikacyjną (transakcję) rozpoczynało wygenerowanie opadającego zbocza i utrzymanie stanu niskiego na sprzętowej linii wyboru układu CS (zdefiniowanej w kodzie jako pin SS). Operacja ta wybudza dedykowany układ podrzędny i przygotowuje jego wewnętrzną maszynę stanów do odbioru ramki danych. Kluczową cechą magistrali SPI, odróżniającą ją od standardu I²C, jest praca w trybie pełnego duplexu (ang. *full-duplex*). W architekturze tej wymiana danych realizowana jest w oparciu o ideę sprzętowych rejestrów przesuwanych. Układ nadrzędny i podrzędny jednocześnie wymieniają między sobą informacje bit po bicie wraz z każdym taktem sygnału zegarowego SCK.

Ten zaawansowany mechanizm zaimplementowano w kodzie za pomocą autorskiej funkcji niskopoziomowej *SPI_Transfer()*. Zapisanie bajtu danych do rejestru danych SPDR (*SPI Data Register*) automatycznie inicjowało sprzętową transmisję – procesor sekwencyjnie ”wypychał” bity na linię MOSI, równocześnie próbując stany z linii MISO. Po zakończeniu cyklu transmisyjnego (8 taktów zegara), funkcja zwracała zawartość rejestru odebraną z ekspandera.

Tak skonstruowana procedura posłużyła jako warstwa fizyczna dla wyższego poziomu abstrakcji, zrealizowanego w funkcji *MCPS_Write()*. Zgodnie ze specyfikacją układu MCP23S17, nadpisywanie rejestrów wymaga wysłania ściśle określonej, trzybajtowej ramki. W pierwszej kolejności nadawany jest tzw. bajt kontrolny (*Opcode*), zawierający m. in. sprzętowy adres urządzenia oraz bit kierunku operacji (zapis/odczyt). Następnie przesyłany jest bajt z adresem docelowego rejestru wewnętrznego, a transakcję kończy właściwy bajt danych. Podniesienie linii CS do stanu wysokiego po wysłaniu trzeciego bajtu skutkuje ostatecznym zatrzaśnięciem danych w strukturze ekspandera.

Finalnym, lecz niezwykle krytycznym z punktu widzenia niezawodności systemu etapem inicjalizacji, była początkowa konfiguracja docelowych portów urządzenia (GPIOA oraz GPIOB). Za pomocą wywołania funkcji *MCPS_Write()* z wartością 0x00, odpowiednie rejestry kierunku (*IODIRA*, *IODIRB*) zostały ustawione w tryb wyjściowy. Sekwencję inicjalizacyjną kończyło wpisanie zerowych stanów logicznych (wartość szesnastkowa 0x00) bezpośrednio do rejestrów zatrzaśków portów. Działanie to stanowi kluczowe zabezpieczenie sprzętowe: zapobiega pojawieniu się stanów nieustalonych lub przypadkowych impulsów na pinie Enable (E) wyświetlacza LCD podczas fazy rozruchu zasilania, co w skrajnych przypadkach mogłoby doprowadzić do zawieszenia się matrycy lub błędnej interpretacji rozkazów.

Rozdział 12

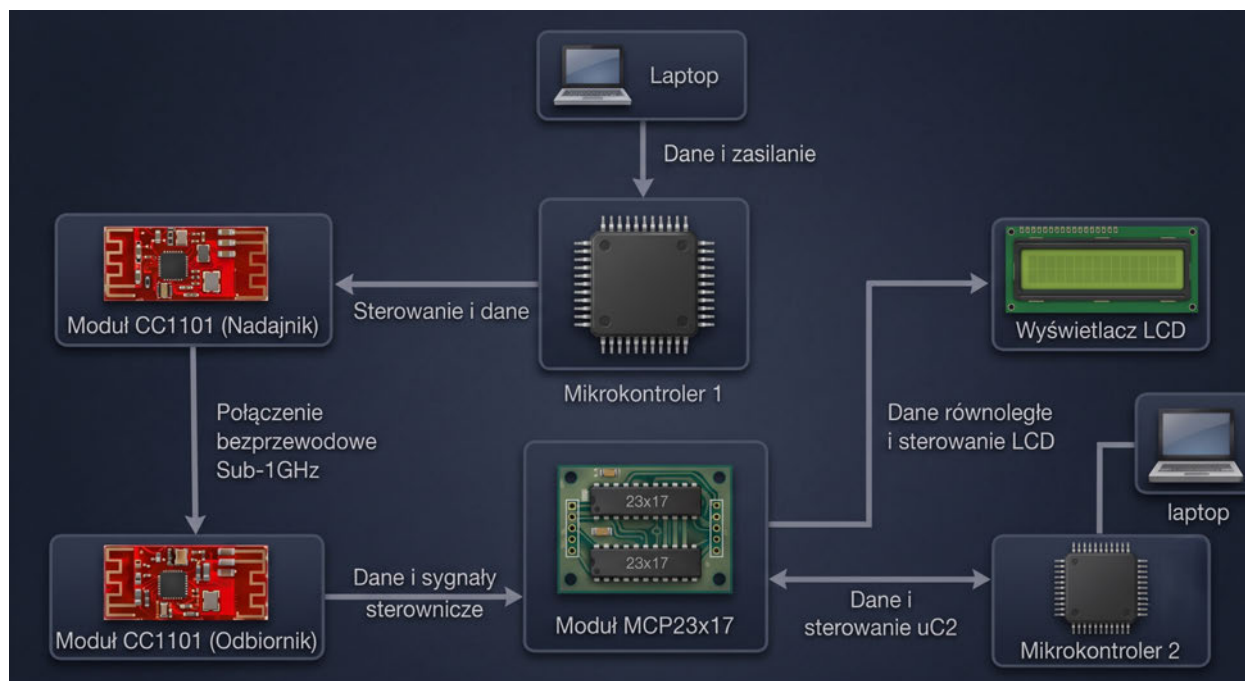
Uruchamianie modułu bezprzewodowego z układem CC1101

12.1 Środowisko pomiarowe

Zbudowane stanowisko pomiarowe zostało zaprojektowane w celu przeprowadzenia kompleksowych testów uruchomieniowych oraz weryfikacji poprawności działania opracowanych modułów transmisji bezprzewodowej. Architektura badanego systemu została fizycznie rozdzielona na dwa niezależne od siebie węzły – nadawczy oraz odbiorczy –, które komunikują się drogą radiową z wykorzystaniem łącza pracującego w paśmie częstotliwości Sub-1 GHz. Głównym celem tak sformułowanego eksperymentu była emulacja rzeczywistego scenariusza aplikacyjnego, polegającego na bezprzewodowym przesłaniu danych sterujących do oddalonego urządzenia peryferyjnego w celu oceny stabilności i niezawodności całego układu.

Węzeł odpowiedzialny za inicjację transmisji składa się ze stacji roboczej w postaci komputera, który pełni rolę nadrzędnego źródła danych testowych, generuje ładunek użyteczny (ang. *payload*) i jednocześnie zapewnia zasilanie układu. Dane te są przekazywane do pierwszej jednostki sterującej, czyli mikrokontrolera stanowiącego centralny element obliczeniowy tego segmentu, którego zadaniem jest akwizycja strumienia informacji, ich odpowiednie formatowanie oraz bezpośrednie wysterowanie układu radiowego. Za właściwą emisję sygnału odpowiada zintegrowany transceiver CC1101 skonfigurowany w trybie nadawczym, który dokonuje modulacji otrzymanego sygnału cyfrowego i przesyła go do medium bezprzewodowego.

Po stronie odbiorczej analogiczny moduł CC1101 realizuje ciągły nasłuch pasma, a po odebraniu fali radiowej dokonuje jej demodulacji i przekazuje odzyskane dane cyfrowe do drugiego mikrokontrolera zarządzającego tą częścią systemu. Jednostka ta jest połączona z osobną stacją roboczą dedykowaną do diagnostyki, monitorowania parametrów łącza oraz logowania wyników transmisji. Kluczowym elementem pośredniczącym w procesie wykonawczym jest ekspander wejść/wyjść MCP23S17, komunikujący się za pośrednictwem szybkiego interfejsu SPI, który pozwala na wielokrotnienie dostępnych linii GPIO (ang. *General-Purpose Input/Output*) mikrokontrolera. Dzięki temu możliwe jest zapewnienie odpowiedniej liczby linii sterujących oraz danych równoległych niezbędnych do poprawnej obsługi docelowego wyświetlacza LCD, na którym wizualizowane są odebrane informacje. Zaprezentowana topologia środowiska pomiarowego pozwala na całościową ocenę niezawodności toru transmisyjnego, potwierdzając zarówno stabilność samego łącza radiowego, jak i prawidłowe funkcjonowanie sprzętowej warstwy aplikacji. Na rysunku 12.1 przedstawiono omówiony system.



RYSUNEK 12.1: Środowisko pomiarowe

12.2 Kod testowy

Z punktu widzenia inżynierii oprogramowania, kod sterujący został zaprojektowany w oparciu o paradygmat programowania modularnego. Architektura projektu składała się łącznie z siedmiu plików źródłowych i nagłówkowych, tworzących spójny system inicjalizacji oraz dwukierunkowej komunikacji pomiędzy dwoma modułami radiowymi (w topologii nadajnik-odbiornik). Strukturę logiczną oprogramowania podzielono na cztery zintegrowane warstwy:

- Pliki nagłówkowe – zawierające definicje rejestrów i instrukcji strobuujących układu CC1101.
- Warstwę abstrakcji sprzętowej (HAL) – realizującą sprzętową inicjalizację magistrali SPI oraz dostarczającą niskopoziomowe funkcje transferu danych.
- Bibliotekę sterującą – odpowiadającą za wysterowanie transiwera radiowego, jego konfigurację RF oraz obsługę pakietów.
- Program główny (main) – integrujący wszystkie moduły i realizujący docelową logikę aplikacji.

Zgodnie z dokumentacją techniczną producenta, układ CC1101 charakteryzuje się bardzo wysokim stopniem konfigurowalności. Jego wewnętrzna maszyna stanów kontrolowana jest za pośrednictwem 62 rejestrów konfiguracyjnych oraz 17 unikalnych sygnałów sterujących, zwanych instrukcjami strobującymi (ang. Command Strobes). W przeciwieństwie do klasycznych operacji zapisu i odczytu, impulsy strobujące to specjalne, jednobajtowe komendy przesyłane za pomocą magistrali SPI, które nie przenoszą dodatkowego ładunku danych (payload). Ich wysłanie natychmiastowo wyzwała odpowiednie przejście w maszynie stanów układu (np. wejście w tryb nasłuchu, uruchomienie kalibracji syntezy częstotliwości czy przejście w stan uśpienia).

Fundamentalnym aspektem implementacji procedur SPI było właściwe sformatowanie bajtu nagłówkowego (adresowego) wysyłanego do transiwera na początku każdej transakcji. Protokół komunikacyjny układu CC1101 przewiduje cztery typy operacji, definiowane za pomocą dwóch najstarszych bitów wysyłanego bajtu: bit odczytu/zapisu (R/W) oraz bit trybu dostępu (pojedynczy – Single Access, lub ciągły – Burst Access).

Poprawne zarządzanie flagą trybu ciągłego okazało się krytyczne ze względu na specyficzną architekturę mapy pamięci urządzenia. W przestrzeni adresowej od 0x30 do 0x3F występuje fizyczne nakładanie się adresów (konflikt adresowy) pomiędzy instrukcjami strobuującymi a rejestrami statusowymi układu. Producent rozwiązał ten problem sprzętowo, dokonując multipleksowania dostępu w oparciu o wspomniany typ wymiany danych. Próba zapisu lub odczytu z adresów w tym przedziale w trybie pojedynczym (Single Access) jest automatycznie interpretowana przez mikrokontroler transiwera jako wywołanie instrukcji strobującej. Z kolei wywołanie operacji odczytu z tej samej puli adresowej, lecz z ustawionym bitem trybu ciągłego (Burst Access), pozwala na swobodny odczyt użytecznych rejestrów statusowych. W ten sposób zaimplementowano dostęp do kluczowych informacji diagnostycznych, takich jak wbudowany numer seryjny, wersja układu, a także bieżące parametry łącza radiowego (wskaźnik siły sygnału odbitego RSSI oraz wskaźnik jakości połączenia LQI).

Zastosowanie tego mechanizmu przez producenta znacząco optymalizuje mapę pamięci i minimalizuje czas transmisji po magistrali SPI, jednakże wymagało precyzyjnej i rygorystycznej implementacji po stronie mikrokontrolera nadrzędnego. Szczegółowe definicje makr preprocesora, odwzorowujące kompletną mapę adresową rejestrów konfiguracyjnych, rejestrów statusowych oraz instrukcji strobujących, zaprezentowano na rysunku 12.2.

```

1 // Typy transmisji
2
3 #define BURST_READ 0xC0 // odczyt ciągły (burst)
4 #define SINGLE_BYTE_READ 0x80 // pojedynczy odczyt
5 #define BURST_WRITE 0x40 // ciągłe nadpisywanie (burst)
6 #define SINGLE_BYTE_WRITE 0x00 // pojedyncze nadpisanie
7
8 // Polecenia stroboskopowe (Strobes)
9
10 #define SRES 0x30 // Reset CC1101
11 #define SFSTXON 0x31
12 #define SXOFF 0x32
13 #define SCAL 0x33
14 #define SRX 0x34
15 #define STX 0x35
16 #define SIDLE 0x36
17 #define SWOR 0x38
18 #define SPWD 0x39
19 #define SFRX 0x3A
20 #define SFTX 0x3B
21 #define SWORRST 0x3C
22 #define SNOP 0x3D
23
24 // Rejestry
25
26 #define IOCFG2 0x00
27 #define IOCFG1 0x01
28 #define IOCFG0 0x02
29 #define FIFOTHR 0x03
30 #define SYNC1 0x04
31 #define SYNC0 0x05
32 #define PKTLEN 0x06
33 #define PKTCTRL1 0x07
34 #define PKTCTRL0 0x08
35 #define ADDR 0x09
36 #define CHANNR 0x0A
37 #define FSCTRL1 0x0B
38 #define FSCTRL0 0x0C
39 #define FREQ2 0x0D
40 #define FREQ1 0x0E
41 #define FREQ0 0x0F
42 #define MDMCFG4 0x10
43 #define MDMCFG3 0x11
44 #define MDMCFG2 0x12
45 #define MDMCFG1 0x13
46 #define MDMCFG0 0x14
47 #define DEVIATN 0x15
48 #define MCSM2 0x16
49 #define MCSM1 0x17
50 #define MCSM0 0x18
51 #define FOCCFG 0x19
52 #define BSCFG 0x1A
53 #define AGCTRL2 0x1B
54 #define AGCTRL1 0x1C
55 #define AGCTRL0 0x1D
56 #define WOREVT1 0x1E
57 #define WOREVT0 0x1F
58 #define WORCTRL 0x20
59 #define FREND1 0x21
60 #define FREND0 0x22
61 #define FSCAL3 0x23
62 #define FSCAL2 0x24
63 #define FSCAL1 0x25
64 #define FSCAL0 0x26
65 #define RCCTRL1 0x27
66 #define RCCTRL0 0x28
67 #define FSTEST 0x29
68 #define PTEST 0x2A
69 #define AGCTEST 0x2B
70 #define TEST2 0x2C
71 #define TEST1 0x2D
72 #define TEST0 0x2E
73 #define PARTNUM 0x30
74 #define VERSION 0x31
75 #define FREQEST 0x32
76 #define LQI 0x33
77 #define RSSI 0x34
78 #define MARCSTATE 0x35
79 #define WORTIME1 0x36
80 #define WORTIME0 0x37
81 #define PKTSTATUS 0x38
82 #define VCO_VC_DAC 0x39
83 #define TXBYTES 0x3A
84 #define RXBYTES 0x3B
85 #define RCCTRL1_STATUS 0x3C
86 #define RCCTRL0_STATUS 0x3D
87 #define TX_RX_FIFO 0x3F // TX - zarówno zapis pojedynczy, jak i seryjny odbywają się pod tym samym adresem

```

RYSUNEK 12.2: Mapa rejestrów i impulsów sterowniczych CC1101

Po zdefiniowaniu mapy rejestrów i instrukcji strobuujących, prace programistyczne skupiły się na implementacji warstwy abstrakcji sprzętowej (ang. Hardware Abstraction Layer – HAL). Warstwa ta jest bezpośrednio odpowiedzialna za sprzętową inicjalizację interfejsu SPI w mikrokontrolerze nadrzędnym oraz rygorystyczną realizację niskopoziomowych procedur nadawczo-odbiorczych. Ponieważ jako jednostkę centralną (Master) wykorzystano mikrokontroler ESP32, kod źródłowy zrealizowano w oparciu o zaawansowane API środowiska programistycznego ESP-IDF.

Kod programu rozpoczęto od zdefiniowania portów wejścia/wyjścia oraz utworzenia dedykowanej struktury danych `cc1101_dev_t`. Struktura ta pełni funkcję wirtualnego odniesienia (uchwytu – ang. *device handle*) do fizycznego modułu radiowego. Przechowuje ona wskaźnik do przydzielonego urządzenia na magistrali SPI (`spi_device_handle_t`) oraz przypisany mu numer pinu wyboru układu CS. Takie modularne podejście znacząco zwiększa skalowalność kodu, pozwalając na jednoczesną, bezkolizyjną i przejrzystą obsługę wielu niezależnych modułów antenowych podłączonych do tej samej szyny sprzętowej mikrokontrolera.

Inicjalizacja magistrali SPI opierała się na konfiguracji dwóch kluczowych struktur środowiska ESP-IDF: `spi_bus_config_t` oraz `spi_device_interface_config_t`. W pierwszej z nich zmapowano fizyczne wyprowadzenia mikrokontrolera dla linii sygnałowych MOSI, MISO oraz SCLK. W drugiej strukturze zdefiniowano krytyczne parametry transmisyjne. Zegar magistrali ustawiono na częstotliwość 500 kHz, a protokół skonfigurowano do pracy w trybie zerowym (SPI Mode 0), wymagany przez układ CC1101.

Warto zwrócić szczególną uwagę na sposób zarządzania linią wyboru układu. W konfiguracji urządzenia zrezygnowano z wbudowanego, sprzętowego sterowania pinem CS (przypisując parametrowi `spics_io_num` wartość -1). Decyzja o implementacji ręcznego (programowego) sterowania tym pinem za pomocą standardowych funkcji biblioteki GPIO podyktowana była rygorystycznymi wymogami czasowymi transiweru. Sprzętowy sterownik SPI w układzie ESP32 mógłby wprowadzać niepożądane stany logiczne pomiędzy kolejnymi bajtami podczas transakcji wielobajtowych. Pełna kontrola programowa gwarantuje nienaruszoną integralność ramki danych. Ponadto, w konfiguracji określono rozmiar kolejki transakcji na 7 elementów, co optymalizuje pracę sprzętowego bufora FIFO i zwiększa ogólną wydajność komunikacji.

Na tak przygotowanym fundamencie fizycznym zbudowano funkcje narzędziowe obsługujące wymianę danych z docelowymi rejestrami i strobami (takie jak `cc1101_read_reg` oraz `cc1101_write_reg`). Korzystają one z wbudowanej struktury `spi_transaction_t`, do której przekazywane są wskaźniki na bufony nadawcze i odbiorcze oraz określana jest długość przesyłanej wiadomości.

Każda operacja zapisu i odczytu została precyzyjnie zsynchronizowana. Proces rozpoczyna się od programowego wygenerowania stanu niskiego na linii CS wybranego modułu. Następnie, wykorzystując zdefiniowane wcześniej maski bitowe (np. `CC1101_READ_SINGLE`), formowany jest bajt nagłówkowy, zawierający docelowy adres rejestru połączony z bitami kierunku oraz trybu operacji. Po synchronicznym przesłaniu danych za pomocą funkcji `spi_device_transmit()`, cykl kończy się przywróceniem linii CS do stanu wysokiego. Napisane w ten sposób funkcje wejścia/wyjścia umożliwiły bezpieczną enkapsulację operacji sprzętowych, tworząc stabilną bazę pod warstwę aplikacji odpowiedzialną za ostateczneysterowanie parametrów RF układu CC1101.

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <string.h>
4  #include "driver/gpio.h"
5  #include "driver/spi_master.h"
6  #include "esp_rom_sys.h"
7  #include "esp_log.h"
8  #include "cc1101_regs.h"
9  #include "cc1101_hal.h"
10
11 //Definicje pinów SPI
12
13 #define SCLK GPIO_NUM_18
14 #define MOSI GPIO_NUM_23
15 #define MISO GPIO_NUM_19
16 #define CS_1 GPIO_NUM_5
17 #define CS_2 GPIO_NUM_21
18 #define GD0_1 GPIO_NUM_13
19 #define GD0_2 GPIO_NUM_12
20
21 //Uchwyty SPI dla anten
22
23 spi_device_handle_t antena_1;
24 spi_device_handle_t antena_2;
25
26 //Inicjalizacja SPI
27
28 void spi_init(){
29
30     spi_device_interface_config_t spi_config = {
31         .mode = 0, //tryb 0
32         .clock_speed_hz = 500000, //500 kHz
33         .spics_io_num = -1, //ręczne sterowanie CS
34         .queue_size = 7, //rozmiar kolejki
35     };
36
37     spi_bus_config_t bus_config = {
38         .sclk_io_num = SCLK,
39         .mosi_io_num = MOSI,
40         .miso_io_num = MISO,
41         .quadhd_io_num = -1,
42         .quadwp_io_num = -1,
43         .max_transfer_sz = 32, //maksymalny rozmiar danych w bajtach
44     };
45
46     spi_bus_initialize(SPI3_HOST, &bus_config, SPI_DMA_CH_AUTO); //włączenie magistrali SPI
47     spi_bus_add_device(SPI3_HOST, &spi_config, &antena_1);
48     spi_bus_add_device(SPI3_HOST, &spi_config, &antena_2);
49     gpio_set_direction(CS_1, GPIO_MODE_OUTPUT);
50     gpio_set_direction(CS_2, GPIO_MODE_OUTPUT);
51     gpio_set_direction(GD0_1, GPIO_MODE_INPUT);
52     gpio_set_direction(GD0_2, GPIO_MODE_INPUT);
53     gpio_set_level(CS_1, 1); //początkowe ustawienie CS na 1, aby był nieaktywny
54     gpio_set_level(CS_2, 1);
55     static const char *TAG = "SPI";
56     ESP_LOGI(TAG, "SPI Initialization ready!");
57 }
58
59 //Funkcje dla trybu jednobajtowego
60
61 void cc1101_write_reg(uint8_t reg_adr, uint8_t value, int ant_num){
62
63     static const char *TAG = "CC1101 WRITE";
64
65     uint8_t tx_data[2] = {(reg_adr | SINGLE_BYTE_WRITE), value}; //dodanie maski do zapisu pojedynczej wiadomości
66     spi_transaction_t tx_transaction = {
67         .tx_buffer = &tx_data,
68         .length = 8*sizeof(tx_data),
69     };
70
71     spi_device_handle_t temp_handle; //zmienna, ponieważ obsługujemy dwie anteny
72
73     if (ant_num == 1){
74         gpio_set_level(CS_1, 0); //ściągnięcie CS w dół, aby wybudzić odbiornik
75         temp_handle = antena_1;
76         spi_device_polling_transmit(temp_handle, &tx_transaction); //wysłanie naszych danych
77         gpio_set_level(CS_1, 1); //zakończenie transmisji
78     }
79     else if (ant_num == 2){
80         gpio_set_level(CS_2, 0); //ściągnięcie CS w dół, aby wybudzić odbiornik

```

RYSUNEK 12.3: Program warstwy fizycznej. Część I

```

80     gpio_set_level(CS_2, 0); //ściągnięcie CS w dół, aby wybudzić odbiornik
81     temp_handle = antena_2;
82     spi_device_polling_transmit(temp_handle, &tx_transaction); // wysłanie naszych danych
83     gpio_set_level(CS_2, 1); //zakończenie transmisji
84 }
85 else { //podano zły numer
86     ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
87     return; //wyjście z funkcji
88 }
89 }
90
91 uint8_t cc1101_read_reg(uint8_t reg_adr, int ant_num){
92
93     static const char *TAG = "CC1101 READ";
94
95     uint8_t tx_data[2] = {(reg_adr | SINGLE_BYTE_READ), 0x00}; //odpytanie podanego rejestru
96     //w pierwszej rundzie, a następnie wysłanie 0x00, aby o nic nie pytać, ale odebrać wartość rejestru
97     uint8_t rx_data[2]; //bufor na odebrane dane z rejestru
98
99     spi_transaction_t rx_transaction = {
100         .tx_buffer = &tx_data,
101         .rx_buffer = &rx_data,
102         .length = 8*sizeof(tx_data),
103     };
104
105     spi_device_handle_t temp_handle; //zmienna, ponieważ obsługujemy dwie anteny
106
107     if (ant_num == 1){
108         gpio_set_level(CS_1, 0);
109         temp_handle = antena_1;
110         spi_device_polling_transmit(temp_handle, &rx_transaction); //wysłanie naszych danych
111         gpio_set_level(CS_1, 1);
112         return rx_data[1]; //pobranie danych z drugiej rundy transmisji.
113         //W pierwszej wysłaliśmy adres rejestru i odbiornik odesłał śmieci.
114         //W drugiej otrzymujemy właściwą wartość rejestru i wysyłamy 0
115     }
116     else if (ant_num == 2){
117         gpio_set_level(CS_2, 0);
118         temp_handle = antena_2;
119         spi_device_polling_transmit(temp_handle, &rx_transaction); //wysłanie naszych danych
120         gpio_set_level(CS_2, 1);
121         return rx_data[1];
122     }
123     else { //podano zły numer
124         ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
125         return 0; //wyjście z funkcji
126     }
127 }
128
129 //Funkcje dla trybu seryjnego (BURST)
130 void cc1101_write_burst(uint8_t reg_adr, uint8_t *buffer, uint8_t length, int ant_num){
131     static const char *TAG = "CC1101 BURST WRITE";
132
133     //Utworzenie bufora większego o 1 bajt (na adres rejestru)
134     uint8_t tx_data[length + 1];
135
136     //Umieszczenie adresu z poleceniem BURST_WRITE na pierwszym miejscu
137     tx_data[0] = (reg_adr | BURST_WRITE);
138
139     //Skopiowanie naszych właściwych danych zaraz po adresie (zaczynając od indeksu 1)
140     memcpy(&tx_data[1], buffer, length);
141
142     //Konfiguracja JEDNEJ wspólnej transakcji
143     spi_transaction_t tx_transaction = {
144         .tx_buffer = tx_data,
145         .length = 8 * (length + 1), //Całkowita długość: 1 bajt adresu + długość danych
146     };
147
148     spi_device_handle_t temp_handle;
149
150     if (ant_num == 1){
151         gpio_set_level(CS_1, 0);
152         temp_handle = antena_1;
153         //Wysłanie wszystkiego za jednym razem!
154         spi_device_polling_transmit(temp_handle, &tx_transaction);
155         gpio_set_level(CS_1, 1);
156     }
157     else if (ant_num == 2){
158         gpio_set_level(CS_2, 0);
159         temp_handle = antena_2;
160         //Wysłanie wszystkiego za jednym razem!

```

RYSUNEK 12.4: Program warstwy fizycznej. Część II

```

161     spi_device_polling_transmit(temp_handle, &tx_transaction);
162     gpio_set_level(CS_2, 1);
163 }
164 else {
165     ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
166     return;
167 }
168 }
169
170 uint8_t cc1101_read_burst(uint8_t reg_addr, uint8_t *buffer, uint8_t length, int ant_num) {
171     static const char *TAG = "CC1101 BURST READ";
172
173     //Utworzenie powiększonych buforów: +1 bajt na sam adres rejestru
174     uint8_t tx_temp[length + 1];
175     uint8_t rx_temp[length + 1];
176
177     //Wypełnienie bufora nadawczego zerami (puste cykle zegara do odczytu)
178     memset(tx_temp, 0, length + 1);
179
180     //Umieszczenie adresu rejestru z flagą BURST_READ na pierwszym miejscu
181     tx_temp[0] = (reg_addr | BURST_READ);
182
183     //Konfiguracja JEDNEJ nieprzerwanej transakcji
184     spi_transaction_t transaction = {
185         .length = 8 * (length + 1),
186         .tx_buffer = tx_temp,
187         .rx_buffer = rx_temp,
188     };
189
190     spi_device_handle_t temp_handle;
191
192     if (ant_num == 1) {
193         gpio_set_level(CS_1, 0);
194         temp_handle = antena_1;
195         spi_device_polling_transmit(temp_handle, &transaction);
196         gpio_set_level(CS_1, 1);
197     }
198     else if (ant_num == 2) {
199         gpio_set_level(CS_2, 0);
200         temp_handle = antena_2;
201         spi_device_polling_transmit(temp_handle, &transaction);
202         gpio_set_level(CS_2, 1);
203     }
204     else {
205         ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
206         return 0;
207     }
208
209     //Wyodrębnienie czystych danych
210     //rx_temp[0] to status (śmiec), więc kopiujemy dane zaczynając od rx_temp[1]
211     memcpy(buffer, &rx_temp[1], length);
212
213     return rx_temp[0]; //Zwrócenie bajtu statusu na wypadek, gdyby był potrzebny w przyszłości
214 }
215
216 //Konfiguracja poleceń stroboskopowych (strobe)
217
218 uint8_t cc1101_set_strobe(uint8_t strb_cmd, int ant_num){
219
220     static const char *TAG = "CC1101 STROBE";
221
222     uint8_t tx_data = strb_cmd;
223     uint8_t rx_data;
224
225     spi_transaction_t rx_transaction = {
226         .tx_buffer = &tx_data,
227         .rx_buffer = &rx_data,
228         .length = 8*sizeof(tx_data),
229     };
230
231     spi_device_handle_t temp_handle; //zmienna, ponieważ obsługujemy dwie anteny
232
233     if (ant_num == 1){
234         gpio_set_level(CS_1, 0);
235         temp_handle = antena_1;
236         spi_device_polling_transmit(temp_handle, &rx_transaction);
237         gpio_set_level(CS_1, 1);
238         return rx_data; //pobranie statusu po stroboskopie
239     }
240     else if (ant_num == 2){

```

RYSUNEK 12.5: Program warstwy fizycznej. Część III

```
241     gpio_set_level(CS_2, 0);
242     temp_handle = antena_2;
243     spi_device_polling_transmit(temp_handle, &rx_transaction);
244     gpio_set_level(CS_2, 1);
245     return rx_data;
246 }
247 else { //podano zły numer
248     ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
249     return 0; //wyjście z funkcji
250 }
251 }
```

RYSUNEK 12.6: Program warstwy fizycznej. Część IV

Wdrożenie warstwy abstrakcji sprzętowej (HAL) pozwoliło na sprawną implementację warstwy aplikacji, zoptymalizowanej pod kątem minimalizacji narzutu obliczeniowego układu ESP32 oraz maksymalizacji niezawodności łącza. Podstawą rygorystycznej konfiguracji transiwera CC1101 była tablica struktur `cfg_regiser`, przechowująca docelowe adresy rejestrów i ich zadaną wartość. Parametry te wygenerowano za pomocą narzędzia SmartRF Studio, co wyeliminowało ryzyko błędów przy ręcznym przeliczaniu nastaw toru RF.

Proces rozruchu, zrealizowany w funkcji `cc1101_init()`, rozpoczął się od sprzętowo-programowego resetu (`cc1101_power_on_reset()`). Wykorzystano tu strob SRES połączony z mechanizmem aktywnego oczekiwania (ang. *polling*). Mikrokontroler odpytywał układ instrukcją SNOP, badając siódmy bit statusowy (maska `0b10000000`). Jego wyzerowanie potwierdzało osiągnięcie stanu IDLE. Następnie weryfikowano szynę SPI poprzez odczyt rejestru VERSION i iteracyjnie przesyłano zawartość matrycy konfiguracyjnej. Dla optymalizacji energetycznej zaimplementowano funkcję wybudzania `cc1101_wake_up()`, a także ustalono sprzętowe wzmocnienie toru nadawczego na poziomie +10 dBm.

Procedurę nadawczą (`cc1101_send_packet()`) zoptymalizowano pod kątem szybkości. Przed każdą transmisją bufor TX FIFO czyszczono strobem SFTX. Ramkę transferowano w trybie ciągłym (BURST), a emisję wyzwalano natychmiastowym strobem STX. Z kolei mechanizm odbiorczy (`cc1101_receive_packet()`) po aktywacji nasłuchu (SRX) oczekiwał na przerwanie z pinu GDO. Program pobierał bajt długości z bufora RX FIFO; jeśli przekraczał on bezpieczną granicę 63 bajtów, pakiet odrzucał, a kolejkę czyszczono strobem SFRX.

W celu podniesienia niezawodności wykorzystano sprzętową funkcję dołączania dwóch bajtów diagnostycznych na końcu odebranego pakietu: RSSI (ang. *Received Signal Strength Indicator*) oraz LQI. W bajcie LQI producent zaimplementował sprzętową flagę poprawności sumy kontrolnej (CRC). W kodzie zastosowano filtr binarny weryfikujący ten warunek: `(status_buffer[1] & 0x80) != 128`. Jego niespełnienie oznaczało przekłamanie danych – uszkodzony pakiet był definitywnie odrzucany, błąd logowano makrem `ESP_LOGW`, a bufor sprzętowy czyszczono przed przyjęciem kolejnej transmisji.

```

1  #include<stdio.h>
2  #include <stdint.h>
3  #include "esp_log.h"
4  #include "driver/gpio.h"
5  #include "esp_rom_sys.h"
6  #include "FreeRTOSConfig.h"
7  #include "freertos\FreeRTOS.h"
8  #include "cc1101_regs.h"
9  #include "cc1101_hal.h"
10 #include "cc1101.h"
11
12 //Wartości rejestrów potrzebne do konfiguracji
13
14 static const cc1101_cfg cfg_regiser[] =
15 {
16     { IOCFG0, 0x06 }, //iocfg0 - Konfiguracja pinu wyjściowego GD00
17     { FIFOTHR, 0x47 }, //fifothr - Progi (Thresholds) RX FIFO i TX FIFO
18     { SYNC1, 0x2D }, //sync1 - Słowo synchronizacji, starszy bajt (High Byte)
19     { SYNC0, 0xD4 }, //sync0 - Słowo synchronizacji, młodszy bajt (Low Byte)
20     { PKTLEN, 0x3D }, //pktlen - Długość pakietu
21     { PKTCTRL1, 0x04 }, //pktctrl1 - Sterowanie automatyzacją pakietów
22     { PKTCTRL0, 0x45 }, //pktctrl0 - Sterowanie automatyzacją pakietów
23     { FSCTRL1, 0x06 }, //fsctrl1 - Sterowanie synteizatorem częstotliwości
24     { FREQ2, 0x21 }, //freq2 - Słowo sterujące częstotliwością, starszy bajt (High Byte)
25     { FREQ1, 0x65 }, //freq1 - Słowo sterujące częstotliwością, środkowy bajt (Middle Byte)
26     { FREQ0, 0x6A }, //freq0 - Słowo sterujące częstotliwością, młodszy bajt (Low Byte)
27     { MDMCFG4, 0xCA }, //mdmcf4 - Konfiguracja modemu
28     { MDMCFG3, 0xB3 }, //mdmcf3 - Konfiguracja modemu
29     { MDMCFG2, 0x16 }, //mdmcf2 - Konfiguracja modemu
30     { DEVIATN, 0x40 }, //deviatn - Ustawienia dewiacji modemu
31     { MCSM0, 0x18 }, //mcsm0 - Konfiguracja głównej maszyny stanów sterowania radiem
32     { FOCCFG, 0x16 }, //foccf - Konfiguracja kompensacji przesunięcia częstotliwości
33     { AGCTRL2, 0x43 }, //agcctrl2 - Sterowanie AGC (Automatyczna kontrola wzmacnienia)
34     { AGCTRL1, 0x49 }, //agcctrl1 - Sterowanie AGC
35     { WORCTRL, 0xFB }, //worctrl - Sterowanie Wake On Radio (wybudzaniem radiowym)
36     { FSCAL3, 0xE9 }, //fscal3 - Kalibracja syntezy częstotliwości
37     { FSCAL2, 0x2A }, //fscal2 - Kalibracja syntezy częstotliwości
38     { FSCAL1, 0x00 }, //fscal1 - Kalibracja syntezy częstotliwości
39     { FSCAL0, 0x1F }, //fscal0 - Kalibracja syntezy częstotliwości
40     { TEST2, 0x81 }, //test2 - Różne ustawienia testowe
41     { TEST1, 0x35 }, //test1 - Różne ustawienia testowe
42     { TEST0, 0x09 }, //test0 - Różne ustawienia testowe
43 };
44
45 //Konfiguracja pinów
46
47 #define GD0_1 GPIO_NUM_13
48 #define GD0_2 GPIO_NUM_12
49 #define CS_1 GPIO_NUM_5
50 #define CS_2 GPIO_NUM_21
51
52 void cc1101_power_on_reset(int ant_num){
53     static const char *TAG = "RESET";
54     uint8_t temp_rx_data; //zmienna do przechowywania logu statusu ze stroboskopu
55     if(ant_num == 1){
56         temp_rx_data = cc1101_set_strobe(SRES, 1); //resetowanie anteny numer 1
57     }
58     else if(ant_num == 2){
59         temp_rx_data = cc1101_set_strobe(SRES, 2); //resetowanie anteny numer 2
60     }
61     else {
62         ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
63         return;
64     }
65     while ((temp_rx_data & 0b10000000) != 0)
66     {
67         ESP_LOGW(TAG, "The chip has not reset yet");
68         //dopóki bit statusu CHIP_RDYN wynosi 1, układ jeszcze się nie zresetował
69         temp_rx_data = cc1101_set_strobe(SNOP, ant_num);
70         //wygenerowanie pustego stroboskopu, aby zaktualizować wartość statusu
71     }
72     ESP_LOGI(TAG, "Antenna %d reset successfully", ant_num);
73 }
74
75 void cc1101_wake_up(int ant_num){
76     static const char *TAG = "CC1101 WAKE UP";
77     gpio_num_t temp_gpio;
78     if (ant_num == 1) temp_gpio = CS_1;
79     else if (ant_num == 2) temp_gpio = CS_2;
80     else {

```

```

81     ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
82     return;
83 }
84
85 gpio_set_level(temp_gpio, 0);
86 esp_rom_delay_us(1000);
87 gpio_set_level(temp_gpio, 1);
88 }
89
90 void cc1101_init(int ant_num){
91     static const char *TAG = "INIT";
92     cc1101_power_on_reset(ant_num); //reset układu
93     uint8_t buffer = 0xFF;
94     cc1101_read_burst(VERSION, &buffer, 1, ant_num);
95
96     if((int)buffer != 20){ //sprawdzenie komunikacji z CC1101
97         ESP_LOGW(TAG, "Expected 20, but read: %d", buffer);
98         ESP_LOGE(TAG, "Incorrect register value! Check connections!");
99     }
100     else ESP_LOGI(TAG, "Register value correct. Communication works");
101
102     int i = 0;
103     while(i<(sizeof(cfg_regiser)/sizeof(cfg_regiser[0])))
104         //sprawdzenie całego rozmiaru tablicy
105     {
106         cc1101_write_reg(cfg_regiser[i].addr, cfg_regiser[i].data, ant_num);
107         //zapis pod adres
108         i++;
109     }
110     ESP_LOGI(TAG, "Initialized CC1101 antenna number %d", ant_num);
111 }
112
113 void cc1101_send_packet(uint8_t *payload, uint8_t length, int ant_num){
114     //wysyłanie pakietu
115     cc1101_set_strobe(SFTX, ant_num);
116     //resetowanie bufora danych nadawczych
117     uint8_t tx_data [length + 1];
118     //bufor stworzony specjalnie dla danych o zmiennej długości
119     tx_data[0] = length;
120     //najpierw wysyłana jest informacja o długości danych
121     memcpy(&tx_data[1], payload, length);
122     //rozpoczęcie wklejania naszych danych do sztucznie utworzonej ramki
123     cc1101_write_burst(TX_RX_FIFO, tx_data, sizeof(tx_data), ant_num);
124     //wysyłanie danych do bufora nadawczego
125     cc1101_set_strobe(STX, ant_num);
126     //ustawienie stroboskopu na wysyłanie danych
127 }
128
129 bool cc1101_receive_packet(uint8_t *data_buffer, uint8_t *status_buffer,
130                             uint8_t *data_length, uint8_t *status_length, int ant_num){
131     static const char *TAG = "CC1101 RECEIVE";
132     gpio_num_t temp_gpio;
133     if (ant_num == 1) temp_gpio = GD0_1;
134     else if (ant_num == 2) temp_gpio = GD0_2;
135     else {
136         ESP_LOGE(TAG, "WRONG ANTENNA NUMBER");
137         return false;
138     }
139
140     cc1101_set_strobe(SRX, ant_num);
141     //ustawienie w tryb odczytu - teraz nasłuchujemy
142     uint32_t time_cnt = 0;
143     while(gpio_get_level(temp_gpio) == 0){
144         //nic nie rób do czasu zakończenia przerwania, początkowo jest to 0 zgodnie z dokumentacją
145         if (time_cnt == 5000000) return false;
146         else time_cnt++;
147     }
148     time_cnt = 0;
149     while(gpio_get_level(temp_gpio) == 1){
150         //nic nie rób do czasu zakończenia przerwania
151         //tutaj odbieramy dane i czekamy aż w całości trafią do bufora
152         if (time_cnt == 500000) return false;
153         else time_cnt++;
154     }
155     uint8_t length = 0;
156     cc1101_read_burst(TX_RX_FIFO, &length, 1, ant_num);
157     //chcemy odczytać 1 bajt, którym jest długość
158     if(length > 63){
159         //gdy pakiet jest niebezpiecznie duży
160         ESP_LOGW(TAG, "Detected packet length too large. Packet discarded. Length %d", length);

```

RYSUNEK 12.8: Program warstwy funkcjonalnej. Część II

```

161     cc1101_set_strobe(SFRX, ant_num);
162     //wyczyszczenie danych z bufora
163     return false;
164     //nie chcemy przekazywać tego pakietu dalej
165 }
166 *data_length = length;
167 //zapisanie rozmiaru do zewnętrznej zmiennej w celu użycia w dalszej części kodu
168 cc1101_read_burst(TX_RX_FIFO, data_buffer, length, ant_num);
169 //zapisanie danych do naszej tablicy i przekazanie ich dalej w kodzie
170 cc1101_read_burst(TX_RX_FIFO, status_buffer, 2, ant_num); //pobranie metadanych
171 *status_length = 2;
172 //znana stała wartość, która nigdy się nie zmienia
173
174 if((status_buffer[1] & 0x80) != 128){
175     ESP_LOGW(TAG, "CRC error detected. Packet discarded. Parameters: Length %d, Data: %d,
176     LQI: %d", length, data_buffer[0], status_buffer[1]);
177     cc1101_set_strobe(SFRX, ant_num); //Wyczyszczenie bufora
178     return false;
179 }
180
181 cc1101_set_strobe(SFRX, ant_num); //Wyczyszczenie bufora
182 return true; //odebrano dane, zakończono
183 }

```

RYSUNEK 12.9: Program warstwy funkcjonalnej. Część III

Finalną częścią kodu był program główny, zrealizowany z uwzględnieniem architektury i wymagań mikrokontrolera ESP32. W celu zapewnienia determinizmu czasowego oraz optymalnego wykorzystania zasobów sprzętowych, oprogramowanie zostało oparte na systemie operacyjnym czasu rzeczywistego FreeRTOS. Zastosowanie tego rozwiązania pozwoliło na implementację współbieżności, dzieląc logikę aplikacji na niezależne wątki robocze (zadania). Kod główny stanowił zwieńczenie wszystkich funkcji konfiguracyjnych i sterujących napisanych w wcześniejszych etapach prac.

Punktem wejścia programu jest funkcja `app_main()`, która w pierwszej kolejności odpowiada za konfigurację sprzętową. Inicjalizowana jest magistrala SPI (`spi_init()`), zapewniająca szybką i niezawodną wymianę danych, a następnie przeprowadzana jest konfiguracja samego układu radiowego CC1101 (`cc1101_init()`). Po przygotowaniu peryferiów, kontrolę nad wykonywaniem programu przejmuje planista (ang. *scheduler*) FreeRTOS, który zarządza utworzonymi zadaniami transmisyjnymi i odbiorczymi. Każdemu z zadań przydzielono niezależny stos pamięci o rozmiarze 2048 bajtów oraz odpowiedni priorytet, co gwarantuje bezkolizyjną pracę modułów.

Zadanie nadawcze (`CC1101_send_task`) odpowiada za cykliczne formowanie i wysyłanie pakietów danych. Szczególną uwagę zwrócono na optymalizację bilansu energetycznego urządzenia. Mikrokontroler tuż przed rozpoczęciem nadawania wybudza układ radiowy, transmituje przygotowany bufor danych, po czym niezwłocznie wprowadza transceiver z powrotem w stan głębokiego uśpienia (ang. *Power Down*) przy pomocy stroboskopu SPWD. Takie zarządzanie zasilaniem drastycznie redukuje pobór prądu w oknach czasowych pomiędzy kolejnymi transmisjami.

Z kolei zadanie odbiorcze (`CC1101_receive_task`) realizuje nasłuchiwanie oraz ekstrakcję przychodzących danych. W przypadku detekcji poprawnego pakietu, algorytm kopiuje zawartość użyteczną oraz pobiera metadane z bufora statusu. Istotnym elementem analizy sygnału zaimplementowanym w tym wątku jest programowe przeliczanie surowej wartości parametru RSSI (ang. *Received Signal Strength Indicator*) na fizyczne jednostki dBm, z uwzględnieniem zdefiniowanego przesunięcia sprzętowego (tzw. offsetu). Ponadto funkcja odczytuje wskaźnik jakości łącza (LQI). Rejestracja tych parametrów w czasie rzeczywistym pozwala na bieżącą diagnostykę, kalibrację oraz ocenę stabilności zaprojektowanego toru radiowego.

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include "freertos\FreeRTOS.h"
4  #include "cc1101.h"
5  #include "cc1101_hal.h"
6  #include "cc1101_regs.h"
7
8  #define RSSI_OFFSET 74
9
10 void CC1101_send_task(void *pvParameters){
11     uint8_t payload[5] = {10, 20, 30, 40, 50};
12     while (1)
13     {
14         cc1101_wake_up(1); //wybudzenie anteny
15         cc1101_send_packet(payload, sizeof(payload), 1); //antena wysyła dane
16         cc1101_set_strobe(SPWD, 1);
17         //usypianie anteny podczas oczekiwania -> oszczędza energię
18         vTaskDelay(1000/portTICK_PERIOD_MS);
19     }
20 }
21
22 void CC1101_receive_task(void *pvParameters){
23     uint8_t data_length = 0;
24     //wskaźnik pomocniczy, z którego później odzyskujemy nieznaną długość pakietu,
25     //a następnie jej używamy
26     uint8_t data_buffer[64] = {0};
27     uint8_t status_buffer[2] = {0};
28     uint8_t status_length = 0;
29     int RSSI_dBm = 0;
30
31     while(1)
32     {
33         if(cc1101_receive_packet(data_buffer, status_buffer, &data_length, &status_length, 2) == true){
34             //gdy odebrano dane
35             if(status_buffer[0] < 128) //obliczanie dBm
36                 RSSI_dBm = (status_buffer[0] / 2) - RSSI_OFFSET;
37             else
38                 RSSI_dBm = ((status_buffer[0] - 256) / 2) - RSSI_OFFSET;
39
40             for(int i = 0; i < data_length; i++)
41                 printf("Packet! Length: %d, Byte[%d]: %d RSSI Metadata: %d dBm, LQI: %d\n",
42                     data_length, i, data_buffer[i], RSSI_dBm, status_buffer[1]);
43         }
44         vTaskDelay(1000/portTICK_PERIOD_MS);
45     }
46 }
47
48 void app_main(void)
49 {
50     spi_init();
51     //cc1101_init(1);
52     cc1101_init(2);
53     //xTaskCreate(CC1101_send_task, "CC1101_SEND", 2048, NULL, 5, 0);
54     xTaskCreate(CC1101_receive_task, "CC1101_RX", 2048, NULL, 5, 0);
55 }

```

RYSUNEK 12.10: Program główny

Rozdział 13

Badania parametrów podsystemu z modułem LCD

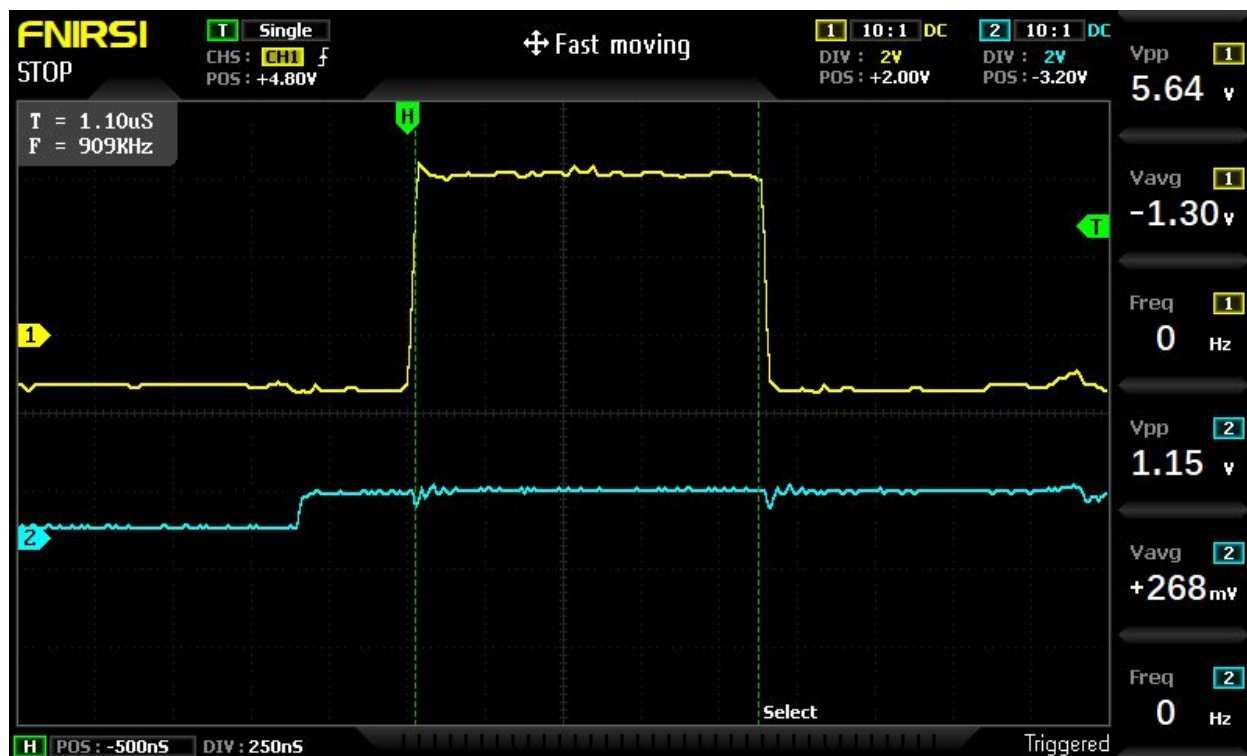
13.1 Analiza porównawcza opóźnień programowych i charakterystyk dynamicznych wyświetlaczy LCD

Kluczowym elementem determinującym stabilność pracy alfanumerycznych modułów LCD opartych na sterowniku HD44780U jest bezwzględne zachowanie reżimu czasowego narzucanego przez producenta. Weryfikacja charakterystyk dynamicznych układu wymaga dogłębnej analizy zaimplementowanych opóźnień programowych oraz zestawienie ich z ograniczeniami sprzętowymi, takimi jak czasy propagacji sygnałów w ekspanderach I/O oraz czasy przetwarzania instrukcji przez układ sterownika wyświetlacza ciekłokrystalicznego [6].

Fundamentalnym sygnałem warunkującym poprawny odczyt danych przez kontroler wyświetlacza jest wygenerowanie prawidłowego impulsu na linii E ang. *Enable*). Zgodnie z notą katalogową układu HD44780U, minimalny czas trwania stanu wysokiego na tej linii wynosi 450 ns dla zasilania napięciem 5 V. We wszystkich opracowanych programach testowych sterujących wyświetlaczem, zarówno w transmisji równoległej, jak i szeregowej zaimplementowano funkcję *lcd_enable()*, która wymusza stan wysoki na linii E, oczekuje 1 μ s, a następnie wraca do stanu niskiego. Zastosowanie takiego opóźnienia daje ponad dwukrotny, bezpieczny margines błędu. Rozwiązanie to całkowicie eliminuje ryzyko zignorowania komendy przez układ na skutek pasożytniczych pojemności ścieżek sygnałowych. Przy pomocy oscyloskopu cyfrowego zweryfikowano rzeczywisty czas omówionego opóźnienia. Pomiar z użyciem makr wyniósł 1.1 μ s.

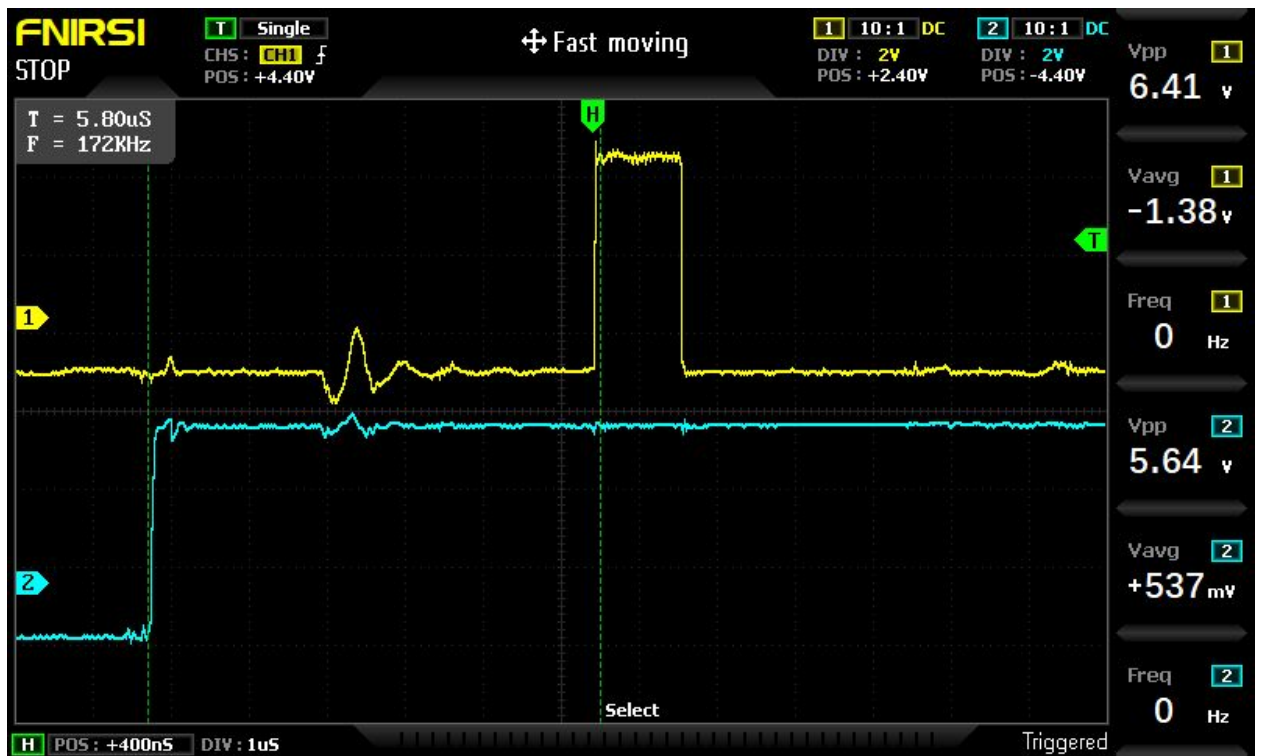
Podczas pomiarów zaobserwowano znaczący wpływ zjawiska dzwonienia (ang. *ringing*) na wyznaczane automatycznie wartości napięcia międzyszczytowego V_{pp} . Algorytm pomiarowy użytego oscyloskopu FNIRSI-1014D wyznaczał tę wartość jako bezwzględną różnicę pomiędzy absolutnym maksimum a minimum dla całej przechwyconej ramki sygnału, przez co włączał do wyniku krótkotrwałe przerzuty napięcia (ang. *overshoot* oraz *undershoot*). Zjawisko to było potęgowane przez specyfikę prototypowego środowiska testowego oraz nieidealność samego układu pomiarowego. Wykorzystanie uniwersalnej płytki stykowej oraz standardowych przewodów połączeniowych wprowadziło do badanego obwodu rozproszone pojemności oraz indukcyjności pasożytnicze. Dodatkowo, pojemność wejściowa pasywnej sondy w połączeniu z indukcyjnością standardowego przewodu masy (tzw. krokodyłka) stworzyła obwód rezonansowy RLC. Przy sygnałach cyfrowych o stromych zboczach prowadzi to do powstawania oscylacji tłumionych, zniekształcających rzeczywisty kształt

przebiegu. Należy jednak zaznaczyć, że głównym celem przeprowadzonych pomiarów było wyznaczenie opóźnienia sygnału, a nie analiza jego precyzyjnych poziomów napięciowych. Z tego względu automatyczne wskazania V_{pp} dla całej ramki potraktowano wyłącznie poglądowo. Właściwa analiza opóźnień została przeprowadzona w sposób manualny, z wykorzystaniem kursorów oscyloskopu, co pozwoliło całkowicie uniezależnić uzyskane wyniki czasowe od zarejestrowanego zjawiska dzwonięcia.



RYSUNEK 13.1: Pomiar opóźnienia stanu wysokiego sygnału enable

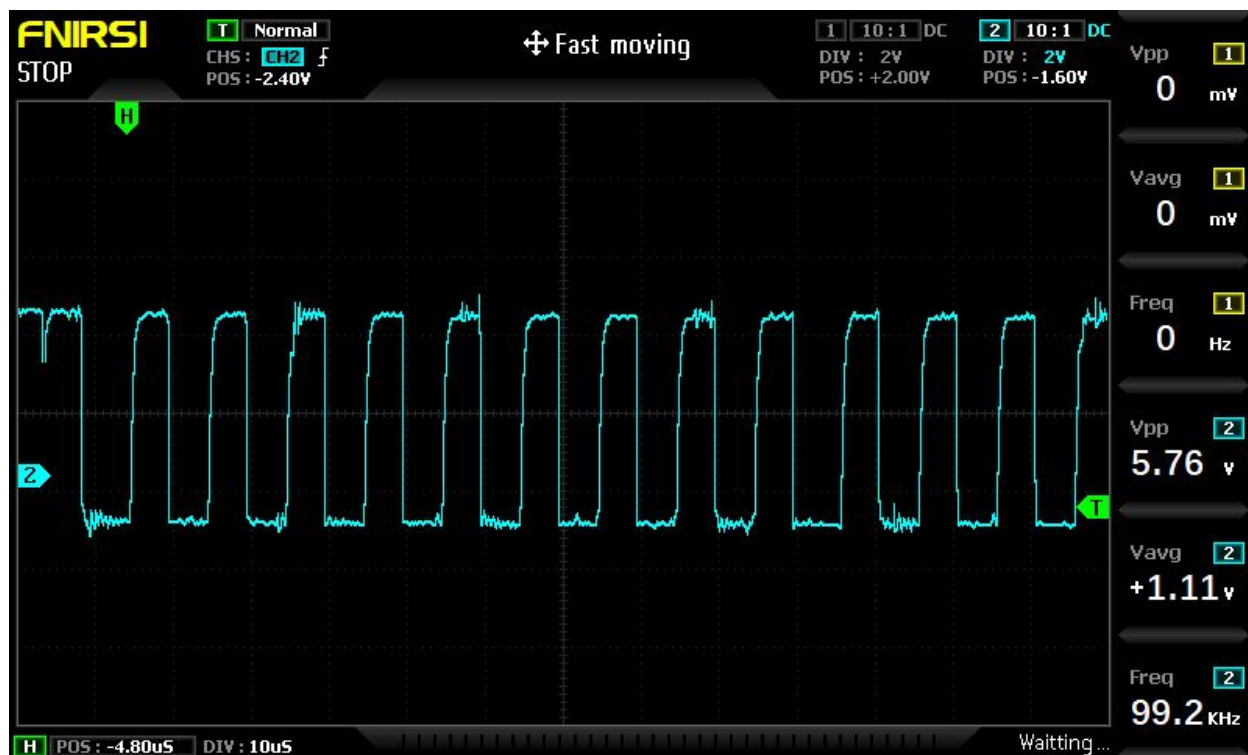
Kolejnym krytycznym elementem jest czas niezbędny na przetworzenie instrukcji przez wewnętrzny układ sterujący matrycą LCD. Standardowe komendy, takie jak wpisanie znaku do pamięci DDRAM czy przesunięcie kursora, wymagają nominalnie około $37 \mu s$ do $43 \mu s$. W kodach źródłowych na końcu funkcji wysyłającej dane `lcd_send()` zastosowano opóźnienie blokujące o wartości $50 \mu s$. Jest to wartość optymalna, gwarantująca ukończenie operacji przed nałożeniem kolejnego rozkazu. Wyjątek stanowią instrukcje czyszczenia ekranu ang. *Clear Display* oraz powrotu na początek linii ang. *Return Home*, których czas oscyluje w granicach $1,52 ms$. W procesie inicjalizacji oraz przy specyficznych poleceniach zaimplementowano wyższe wartości opóźnień, rzędu $2 ms$ oraz sekwencje $40 ms$, $5 ms$ i $1 ms$ w początkowej fazie wybudzania układu, zgodnie z wymogami oczekiwania na stabilizację napięcia zasilania VCC wyższego niż $4,5 V$ [11].



RYSUNEK 13.2: Okres stabilizacji danych przed sygnałem Enable

Istotne wnioski płyną z porównania konfiguracji równoległej z układami wykorzystującymi serializację danych. W przypadku sterowania bezpośredniego¹, czas jest determinowany wyłącznie częstotliwością taktowania mikrokontrolera (16 MHz) i jest pomijalnie mały. O całkowitym czasie transakcji decyduje wprost opóźnienie 50 μs . W konfiguracji hybrydowej, gdzie dane są przesyłane szeregowo, a sygnały sterujące równolegle wprost z wyjść mikrokontrolera, sytuacja ulega zmianie z powodu charakterystyki dynamicznej ekspanderów wejść/wyjść. Magistrala I²C taktowana standardowym zegarem o częstotliwości 100 kHz potrzebuje około 180 μs na przesłanie ramki (uwzględniając adres, dane i bit ACK). Wynika z tego, że samo opóźnienie sprzętowe wynikające z przesyłu danych do układu PCF8574 znacząco przekracza czas przetworzenia rozkazu przez wyświetlacz LCD (37 μs). Pomiary oscyloskopem przeprowadzone na rzeczywistym układzie (rys. 13.3), wykazały, że częstotliwość linii SCL oscyluje w granicach 96-99 kHz. Protokół I²C wymusza przesłanie minimum dwóch bajtów, w każdej transakcji zapisu (adres oraz właściwy bajt danych). Jak wynika z zarejestrowanych przebiegów, pełny cykl przesłania danych zajmuje w rzeczywistości 200 μs (rys. 13.4). Oznacza to, że sam narzut czasowy wynikający z protokołu komunikacyjnego ponad pięciokrotnie przekracza czas przetworzenia instrukcji przez wyświetlacz, stanowiąc głównie wąskie gardło tej metody sterowania.

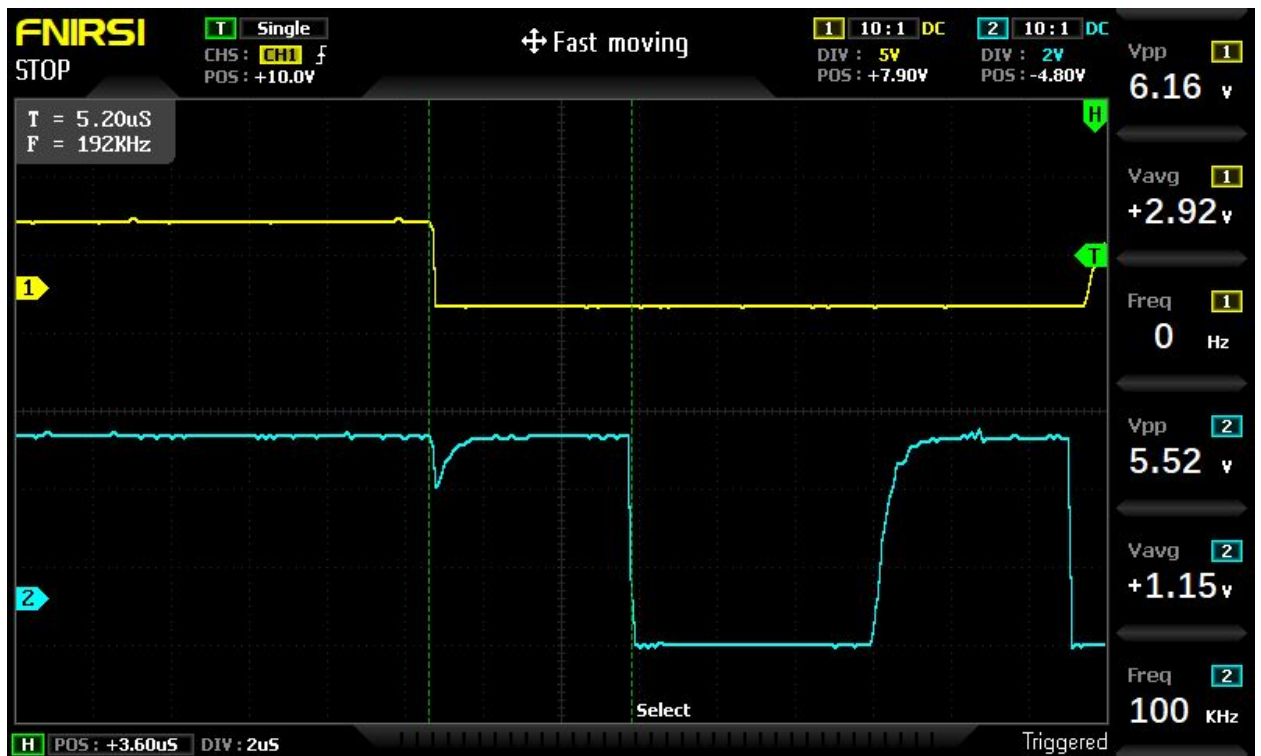
¹8 bądź 4-bitowego poprzez porty mikrokontrolera ATmega 328p



RYSUNEK 13.3: Częstotliwość linii zegarowej magistrali I2C



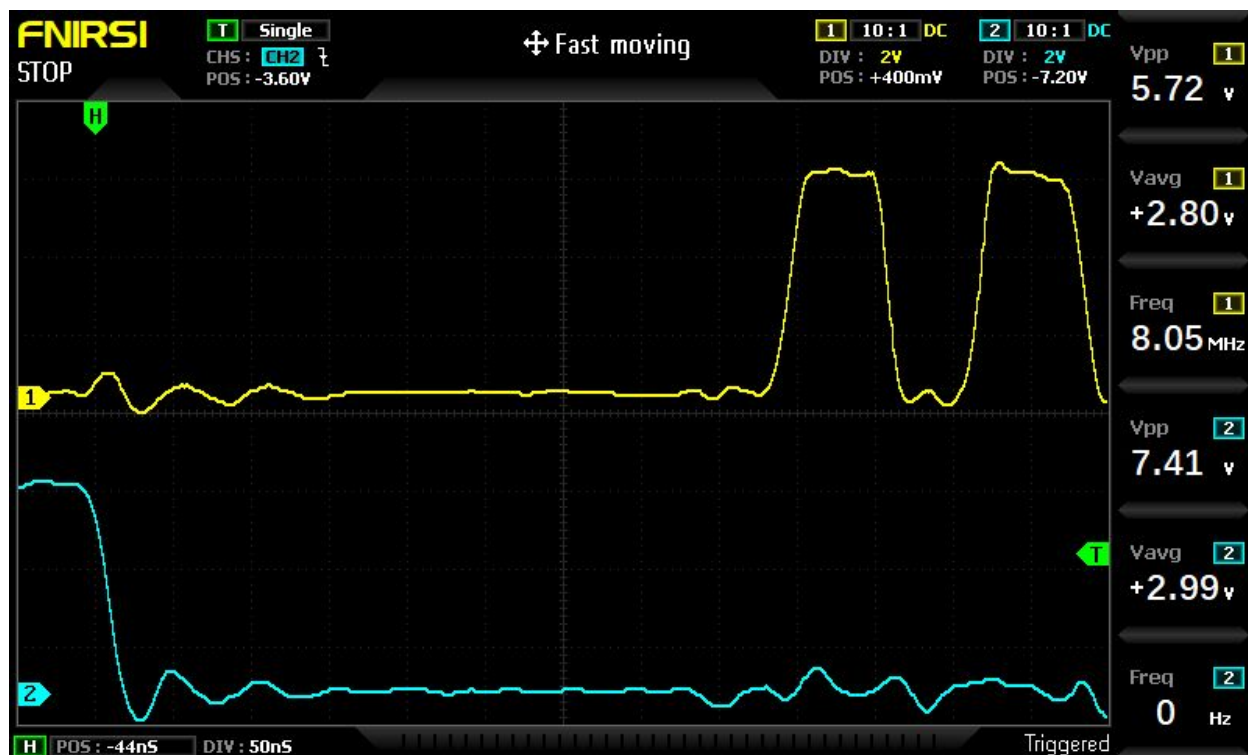
RYSUNEK 13.4: Czas jednej sesji I2C



RYSUNEK 13.5: Wyznaczenie czasu od sygnału START do faktycznej transmisji

W implementacji z ekspanderami I/O programowe funkcje blokujące przez $50 \mu s$ stają się teoretycznie nadmiarowe, ponieważ sam fizyczny czas transmisji instrukcji po magistrali gwarantuje, że sterownik HD44780U zdążył przetworzyć poprzednią operację. Niemniej jednak pozostawienie tych opóźnień w strukturze hybrydowej jest uzasadnioną praktyką, gwarantującą niezawodność przy migracjach na szybsze magistrale, gdzie przepływność sprzętowa przestaje maskować wymogi czasowe matrycy. Za pomocą oscyloskopu zweryfikowano parametry sygnałów linii I²C.

W przypadku interfejsu SPI, cechującego się wyższą przepustowością niż I²C, częstotliwość taktowania magistrali ustalono na 8 MHz. Poprawność tej konfiguracji została zweryfikowana pomiarem na rzeczywistym układzie, jak przedstawiono na przebiegu z oscyloskopu (rys. 13.6), gdzie rzeczywista częstotliwość linii SCK wynosi około 8,05 MHz. Zgodnie ze specyfikacją układu MCP23S17, pojedyncza instrukcja zapisu wymaga przesłania trzech bajtów (kod operacji, adres rejestru oraz dane). Analiza zarejestrowanego przebiegu sprzętowej transmisji danych przedstawionych na rysunku 13.7 wykazuje, że sekwencyjne nadanie paczki czterech bajtów zajmuje zaledwie $8,62 \mu s$. Dowodzi to, że sprzętowy czas wysłania pojedynczej instrukcji wraz z niezbędnym narzutem programowym mikrokontrolera (obsługa pętli wysyłającej) wynosi optymalnie około $4\text{--}5 \mu s$. Niemniej jednak, na potrzeby zapewnienia stabilnej komunikacji z wolniejszym sterownikiem wyświetlacza LCD, procedura zapisu została celowo spowolniona przez warstwę programową. Wprowadzenie dodatkowego opóźnienia o wartości 1 ms powoduje, że pojedyncza transakcja zapisu na magistrali wydłuża się do ponad 2 ms. Zestawienie dowodów z oscyloskopu (rys. 13.6 rys. 13.7) z ostatecznym czasem wykonania procedury jednoznacznie udowadnia, że sama warstwa sprzętowa SPI działa z maksymalną, zakładaną wydajnością i bez opóźnień między bitami. W tej specyficznej konfiguracji wąskie gardło systemu zostało sztucznie przesunięte z protokołu komunikacyjnego na oprogramowanie, w celu dostosowania się do ograniczeń czasowych elementu końcowego.



RYSUNEK 13.6: Częstotliwość linii zegarowej magistrali SPI



RYSUNEK 13.7: Okres transmisji pakietu danych SPI

13.2 Badania poprawności transmisji przewodowej w systemie rozproszonym

Prace związane ze stabilnością komunikacji zaprowadziły nas do pytania, jaka kombinacja długości łącza i częstotliwości transmisji była optymalna. Poddano badaniom użyte wcześniej interfejsy I²C oraz SPI, korzystając z ekspanderów serii MCP23x17. W pracach badawczych zastosowano dwa mikrokontrolery, rezydujące w Arduino Uno oraz ESP32. W przypadku, gdy wykorzystano układ firmy Microchip tabela ma dopisek "dla napięcia 5 V". Jest to związane z poziomem logicznym wystawianym przez mikrokontroler. Dla ESP32 w tytułach tabel dopisano "dla napięcia 3,3 V". Pomiary wykonano dla kilku długości przewodów transmisyjnych: 1 m, 3 m, 5 m, 7 m i 10 m. Zależnie od badanej magistrali zmieniano częstotliwość taktowania zegara. Dla interfejsu I²C korzystano z częstotliwości od 100 kHz do 1,1 MHz z krokiem 100 kHz. Zgodnie z dokumentacją techniczną ekspandera MCP23017 jego maksymalna częstotliwość transmisji wynosiła 1 MHz.



RYSUNEK 13.8: Testy na magistrali I²C dla przewodów 3 metrowych

TABELA 13.1: Wyniki pomiarów z wykorzystaniem modułu ESP32 dla transmisji I²C przy napięciu zasilania 3,3 V oraz rezystora 1 k Ω . Część I

Długość / Częstotliwość	100 kHz	200 kHz	300 kHz	400 kHz	500 kHz	600 kHz
1,0 m	+	+	+	+	+	+
3,0 m	+	+	+	+	+	+
5,0 m	+	+/-	–	–	–	–
7,0 m	+	+	–	–	–	–
10,0 m	+	+	–	–	–	–

TABELA 13.2: Wyniki pomiarów z wykorzystaniem modułu ESP32 dla transmisji I²C przy napięciu zasilania 3,3 V oraz rezystora 1 k Ω . Część II

Długość / Częstotliwość	700 kHz	800 kHz	900 kHz	1000 kHz	1100 kHz
1,0 m	+	+	+	+/-	-
3,0 m	+	+	+	+/-	–
5,0 m	–	–	–	–	–
7,0 m	–	–	–	–	–
10,0 m	–	–	–	–	–

TABELA 13.3: Wyniki pomiarów z wykorzystaniem modułu Arduino Uno dla transmisji I²C przy napięciu zasilania 5 V oraz rezystora 1 k Ω . Część I

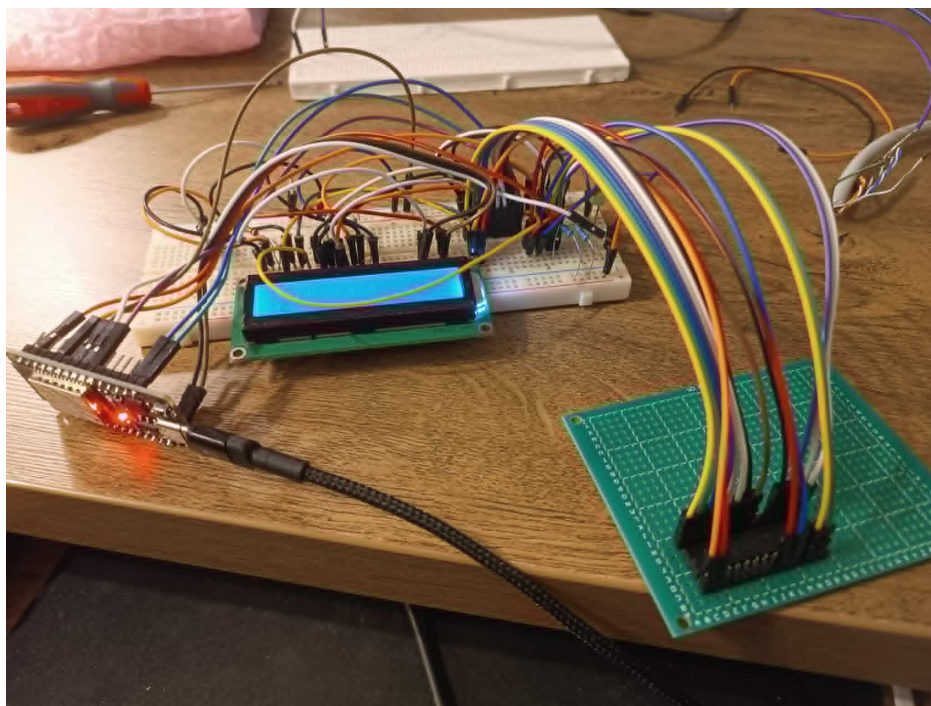
Długość / Częstotliwość	100 kHz	200 kHz	300 kHz	400 kHz	500 kHz
1,0 m	+	+	+	+	+
3,0 m	+	+	+	+	+
5,0 m	+	+	+	+	+
7,0 m	+	+	–	–	–
10,0 m	+	+	–	–	–

TABELA 13.4: Wyniki pomiarów z wykorzystaniem modułu Arduino Uno dla transmisji I²C przy napięciu zasilania 5 V oraz rezystora 1 k Ω . Część II

Długość / Częstotliwość	600 kHz	700 kHz	800 kHz	900 kHz	1000 kHz
1,0 m	+	+	+	–	–
3,0 m	+	+	+	–	–
5,0 m	+	+	+	–	–
7,0 m	–	–	–	–	–
10,0 m	–	–	–	–	–

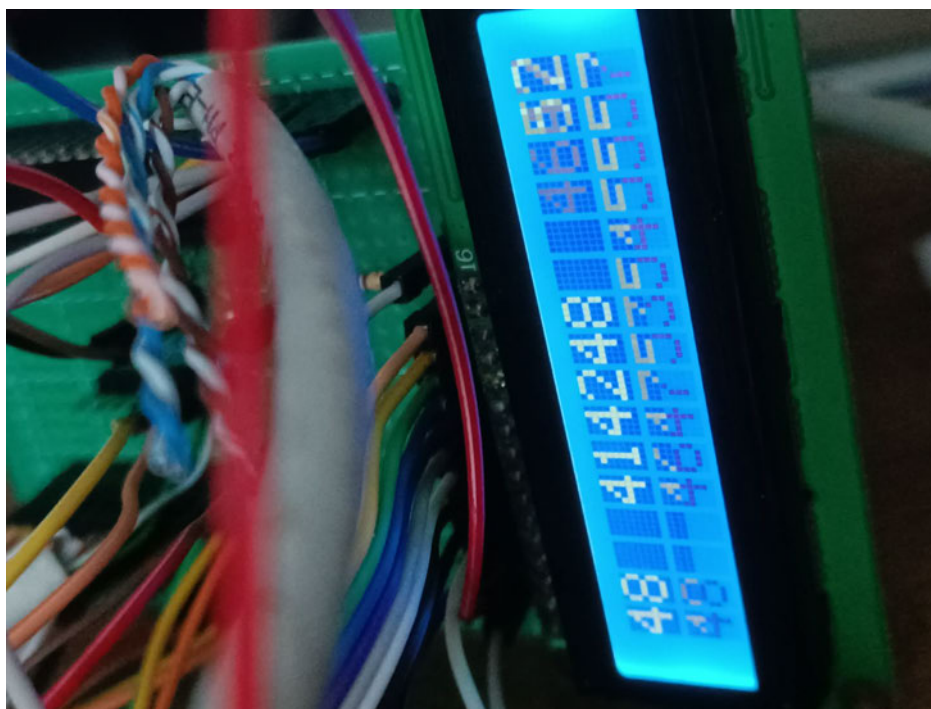
Na podstawie wyników (Tabele 13.1–13.4) przeanalizowano wpływ napięcia logicznego i długości linii na stabilność magistrali I²C (przy rezystorach 1 k Ω). Pomiary wykazały, że układ ESP32 (3,3 V) osiąga najwyższą przepływność (do 900 kHz) na krótkich odcinkach (1–3 m). Jednak ze względu na niższe napięcie stanu wysokiego jest podatny na pojemność pasożytniczą przewodów, co przy 5–10 m wymusza redukcję prędkości do 100–200 kHz.

Z kolei platforma Arduino Uno (5 V), mimo nieco niższej maksymalnej częstotliwości (800 kHz), wykazuje znacznie większą odporność na degradację sygnału, utrzymując tę prędkość aż do 5 metrów. Dla najdłuższych linii (7–10 m) fizyka transmisji ogranicza oba układy do pasma 100–200 kHz. Podsumowując: standard 3,3 V jest optymalny dla szybkich, scentralizowanych połączeń, natomiast 5 V gwarantuje niezawodną komunikację na większych dystansach.



RYSUNEK 13.9: Układ pomiarowy z mikrokontrolerem ESP32

W przypadku SPI częstotliwości testowe były dostosowywane do użytego mikrokontrolera. Dla Arduino są to 1 MHz, 4 MHz, 8 MHz i 9 MHz. Mikrokontroler firmy Espressif miał możliwość na sprawdzenie wyższych taktowań zegara tj. 10 MHz, 20 MHz, 30 MHz i 40 MHz [25]. Ekspander MCP23S17 zgodnie z notą katalogową zapewniał poprawność transmisji maksymalnie do 10 MHz.



RYSUNEK 13.10: Niestabilna komunikacja przy SPI

TABELA 13.5: Wyniki pomiarów z wykorzystaniem modułu ESP32 dla transmisji SPI przy napięciu zasilania 3,3 V oraz rezystora 1 k Ω . Część I

Długość / Częstotliwość	1 MHz	4 MHz	8 MHz	9 MHz
1,0 m	+	+	+	+
3,0 m	+	+	+	+
5,0 m	+	+	+	+
7,0 m	+	+	+	+
10,0 m	–	–	–	–

TABELA 13.6: Wyniki pomiarów z wykorzystaniem modułu ESP32 dla transmisji SPI przy napięciu zasilania 3,3 V oraz rezystora 1 k Ω . Część II

Długość / Częstotliwość	10 MHz	20 MHz	30 MHz	40 MHz
1,0 m	+/-	–	–	–
3,0 m	+/-	–	–	–
5,0 m	+/-	–	–	–
7,0 m	+/-	–	–	–
10,0 m	–	–	–	–

TABELA 13.7: Wyniki pomiarów z wykorzystaniem modułu Arduino Uno dla transmisji SPI przy napięciu zasilania 5 V oraz rezystora 1 k Ω . Część I

Długość / Częstotliwość	1 MHz	4 MHz	8 MHz	9 MHz
1,0 m	+	+	+	+
3,0 m	+	+	+	+
5,0 m	+	+	+	+
7,0 m	+	+	+	+/-
10,0 m	+	+	+	–

TABELA 13.8: Wyniki pomiarów z wykorzystaniem modułu Arduino Uno dla transmisji SPI przy napięciu zasilania 5 V oraz rezystora 1 k Ω . Część II

Długość / Częstotliwość	10 MHz	20 MHz	30 MHz	40 MHz
1,0 m	+	–	–	–
3,0 m	+	–	–	–
5,0 m	+/-	–	–	–
7,0 m	–	–	–	–
10,0 m	–	–	–	–

Pomiary ujawniły kilka ciekawych zjawisk zachodzących w badanym systemie. Zwiększone napięcie stanów logicznych Arduino Uno pozwoliły na stabilną i daleką transmisję do około 8 MHz. Przy szybszej komunikacji został ukazany potencjał ESP32. Stała pojemność przewodów i rezystor tworzyły filtr RC, który tłumił sygnały SPI. W tym momencie znacznie nowocześniejszy układ jakim niewątpliwie jest ESP32 pokazał swoje możliwości. Mikrokontroler pozwalał na wygenerowanie bardzo stromych zboczy o niższej amplitudzie, przez co przy większych częstotliwościach odbiór poszczególnych stanów logicznych był dalej możliwy. Dla podobnej sytuacji odbiór sygnałów z Arduino nie był już osiągalny, ponieważ napięcie rosło lecz zbyt wolno żeby odczytać stan wysoki [6, 8, 12].

Rozdział 14

Podsumowanie

Przedstawiona praca udowodniła, że tematyka komunikacji z wyświetlaczami to bardzo rozległy dział elektroniki. Uruchomienie ekranu łączy się z zapoznaniem się z notami katalogowymi wszystkich wykorzystywanych urządzeń, weryfikacją działania na płytce stykowej, poznaniem magistrali komunikacyjnej i przeprowadzeniem testów. Kolejnym wyzwaniem było stworzenie modułów, które miały obsługiwać cały system współpracujący z wyświetlaczem. Proces tworzenia płytek PCB składał się z określenia materiałów, z których miał składać się moduł, doboru komponentów, rozłożenia elementów, optymalizacji płytki oraz polutowania wszystkich modułów z weryfikacją działania. Sumienna, ciężka praca przyniosła pozytywne efekty - dwa w pełni działające projekty modułów z ekspanderami portów wejścia/wyjścia. Praca z stworzonymi urządzeniami miała dużą wartość edukacyjną. Powielenie modułów dało możliwość, żeby każdy student elektroniki zdobył doświadczenie z tematyki komunikacji z wyświetlaczem, oraz pracy z różnymi magistralami. Prostota ekspandera PCF8574 pokazywała, że odpowiednio dostosowując projekt do swoich potrzeb można w łatwy i tani sposób manipulować ekranem. Projektowanie modułu wykazało, że ograniczenie liczby połączeń jest możliwe i niesie za sobą wiele korzyści. PCF8574 komunikował się interfejsem I²C, co dało możliwość lepszego zrozumienia szeregowej wymiany danych. Drugi moduł pozwalał na pracę z bardziej zaawansowanymi ekspanderami z rodziny MCP23x17. Urządzenia cechowały się 16-bitową szyną portów wejść/wyjść, rejestrami konfiguracyjnymi i obsługą dwóch magistral szeregowych: I²C oraz SPI. Poza komunikacją szeregową wszystkie moduły miały możliwość transmisji równoległej. Dzięki temu student może na jednej płytce zweryfikować wady i zalety danych typów komunikacji. W celu urozmaicenia pracy oraz mając świadomość ciągle rozwijającego się świata elektroniki postanowiono zaprojektować moduły do bezprzewodowej wymiany danych. Układ scalony CC1101 posługiwał się magistralą SPI, dlatego jest w pełni kompatybilny z jednym ze stworzonych modułów. Takim sposobem pokazano, że zaprojektowane i wykonane płytki posiadają ogromny potencjał do rozwoju. Zaprojektowane urządzenia znajdują swoje zastosowanie w systemach pomiarowych, systemach dozorowych, IoT, SmartHome i wielu innych.

Literatura

- [1] ATMel. Nota katalogowa mikrokontrolera atmega328p, [online 12.04.2026].
https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.
- [2] Jeremy Blum. *Odkrywanie Arduino. Narzędzia i techniki inżynierii pełnej czaru. Wydanie II.* wyd. Helion, ul. Kościuszki 1c, 44-100 GLIWICE, 2020.
- [3] Botland. Konwerter i2c dla wyświetlacza lcd hd44780, [online 14.05.2026].
<https://botland.com.pl/konwertery-pozostale/2352-konwerter-i2c-dla-wyswietlacza-lcd-hd44780-5903351248693.html>.
- [4] Botland. Wyświetlacz 16x2 z górnymi wyprowadzeniami, [online 12.04.2026].
<https://botland.com.pl/wyswietlacze-alfanumeryczne-i-graficzne/19738-wyswietlacz-lcd-2x16-znakow-zielony-justpi-5903351243063.html>.
- [5] Robert H. Chen. *Liquid Crystal Displays: Fundamental Physics and Technology.* John Wiley and Sons, Inc., Hoboken, New Jersey, 2011.
- [6] Augustyn Chwaleba, Maciej Poniński, Andrzej Siedlecki. *Metrologia Elektryczna.* Wydawnictwo Naukowo-Techniczne, Warszawa, 2003.
- [7] Peter Dalmaris. *KiCad Like a Pro.* Tech Explorations, Berowra 2081, Australia, 2021.
- [8] Jacek Dusza, Grażyna Gortat, Antoni Leśniewski. *Podstawy miernictwa.* Oficyna Wydawnicza Politechniki Warszawskiej, 2002.
- [9] Tomasz Francuz. *Mikrokontrolery AVR i ARM. Sterowanie wyświetlaczami LCD.* wyd. Helion, ul. Kościuszki 1c, 44-100 GLIWICE, 2017.
- [10] Paweł Hadam. *Projektowanie systemów mikroprocesorowych.* wyd. BTC, ul. Lwowska 5 05-120 Legionowo, 2004.
- [11] Hitachi. Nota katalogowa sterownika hd44780u, [online 12.04.2026].
<https://www.alldatasheet.pl/datasheet-pdf/view/63673/HITACHI/HD44780.html>.
- [12] Paul Horowitz, Winfield Hill. *Sztuka elektroniki.* Wydawnictwa Komunikacji Łączności, Warszawa, 2018.
- [13] Ilitek. Nota katalogowa sterownika ili9341, [online 12.04.2026].
<https://www.alldatasheet.com/datasheet-pdf/view/1131760/ETC2/ILI9341.html>.
- [14] Microchip Technology Inc. Nota katalogowa ekspanderów mcp23017 oraz mcp23s17, [online 17.04.2026]. <https://ww1.microchip.com/downloads/en/devicedoc/20001952c.pdf>.
- [15] Texas Instruments. Cc1101 - low-power sub-1 ghz rf transceiver [online 15.06.2026].
<https://www.ti.com/lit/ds/symlink/cc1101.pdf>.
- [16] Texas Instruments. Nota katalogowa ekspandera pcf8574, [online 17.04.2026].
<https://www.ti.com/lit/ds/symlink/pcf8574.pdf>.

- [17] Horward W. Johnson, Martin Graham. *High-Speed Digital Design*. Prentice Hall, Englewood Cliffs, New Jersey, 1993.
- [18] Mirosław Kardaś. *Język C Pasja Programowania Mikrokontrolerów 8-bitowych*. wyd. Atnel, ul. Uczniowska 5A, 70-893 Szczecin, 2014.
- [19] Graham Keeth, Jon Evans, Glenn Peterson. Wprowadzenie do programu kicad, [online 11.06.2026].
https://docs.kicad.org/10.0/pl/getting_started_in_kicad/getting_started_in_kicad.html.
- [20] Altium Limited. Szeregowe protokoły komunikacyjne część 5 - spi, [online 07.05.2026].
<https://resources.altium.com/p/serial-communications-protocols-part-5-spi>.
- [21] Altium Limited. Szeregowe protokoły komunikacyjne część 6 - i2c, [online 07.05.2026].
<https://resources.altium.com/p/6-serial-communications-protocols-i2c>.
- [22] mSalamon. Wyświetlacz tft oparty na sterowniku ili9341, [online 12.04.2026].
<https://sklep.msalamon.pl/produkt/wyswietlacz-tft-lcd-32%E2%80%B3-ili9341-240x320/>.
- [23] Waldemar Nawrocki. *Sensory i systemy pomiarowe*. wyd. Politechniki Poznańskiej, ul. Piotrowo 5 61-138 Poznań, 2006.
- [24] Waldemar Nawrocki. *Projektowanie systemów mikroprocesorowych*. wyd. Politechniki Poznańskiej, ul. Piotrowo 5 61-138 Poznań, 2010.
- [25] Vedat Ozan Oner. *Developing IoT Projects with ESP32*. Packt Publishing, 11 St Paul's Square, Birmingham, 2021.
- [26] Zachariah Peterson. Spi vs. i2c: How to choose the best protocol for your memory chips, [online 11.06.2026]. <https://resources.altium.com/p/spi-versus-i2c-how-choose-best-protocol-your-memory-chips>.
- [27] Paul Scherz, Simon Mon. *Practical Electronics for Inventors*. Mc Graw Hill Education, New York, Chicago, 2016.
- [28] NXP Semiconductors. Specyfikacja magistrali i2c i instrukcja obsługi, [online 07.05.2026].
https://www.nxp.com/documents/user_manual/UM10204.pdf.
- [29] TME. Wyświetlacz 16x2 z dolnymi wyprowadzeniami, [online 12.04.2026]. <https://www.tme.eu/pl/details/rc1602e-biw-esx/wyswietlacze-lcd-alfanumeryczne/raystar-optronics/>.
- [30] TME. Wyświetlacz 40x2 z wyprowadzeniami po lewej stronie wyświetlacza, [online 12.04.2026].
<https://www.tme.eu/pl/details/rc4002a-biw-jsx/wyswietlacze-lcd-alfanumeryczne/raystar-optronics/>.



© 2026 Szymon Dobrowolski, Rafał Ryske

Instytut Telekomunikacji Multimedialnej, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.